A Tutorial of CS Tools
Vim, the Unix Shell, Make, GDB and Git

Kon Yi

October 16, 2026

Preface

Every computer science course teaches you a language. Almost none of them teach you the tools you will spend the next decade typing into. You are expected to absorb Vim, the shell, `make`, a debugger and Git by osmosis, from watching someone else's screen or from whichever Stack Overflow answer happened to work — and the result is that most people plateau early, using perhaps five percent of each tool for years.

This note is an attempt to fix that for five tools at once. It is not comprehensive; each of the five has a manual running to hundreds of pages, and those manuals are excellent and free. What this note offers instead is the *model*: the handful of ideas inside each tool that make the rest of its commands predictable rather than arbitrary.

Those models turn out to rhyme. Vim composes small operators with small motions. The shell composes small programs with pipes. `make` composes small rules into a dependency graph. Git composes small commits into a history you can rearrange. GDB is the odd one out — it composes nothing — but it earns its place by being the tool that answers the question the other four provoke: *what is this program actually doing?* In every case the tool is not a bag of features but a grammar, and in every case the reward for learning the grammar is that you stop memorising commands and start deriving them. That shared shape is the real argument for putting these five in one document.

How to read this. The five parts are largely independent and can be read in any order. They do lean on each other where it helps, and those places are usually the interesting ones: `sed`'s substitution command and Vim's `:s` are the same command with the same ancestor; a unified diff is read identically in Section 11.4 and Section 31.5; `make` is at its best when it delegates to a tool that already solves the problem; and GDB runs perfectly well inside the editor of Part I. Within a part, the chapters are ordered and do build on one another.

Everything here was written and checked on one ordinary Linux machine, and the versions are listed in Appendix G. Where a part describes a configuration — Chapter 8 in particular — it describes the one actually installed there, rather than an idealised one.

Do not try to memorise the tables. Every part explains its own notation before using it — a whole chapter for Vim, whose notation is the densest, and a first section for the rest — and every part ends with material meant to be used rather than read. The reference cards in the appendices exist so that you can *stop* holding it all in your head. Pick three or four commands per chapter, use them until they become involuntary, and come back for more. These tools have a learning curve that is not steep so much as *long*, and the only approach that reliably fails is trying to climb it all at once.

Sources. Where this note is thin, the manuals are not, and all of them are free. Vim's own help files [Vim] and Neovim's [Neo] are readable inside the editor with `:help`; `man` and `info` cover the shell and its tools; and the GNU manuals for `make` [SMS] and GDB [SPS] are the authoritative references for Parts III and IV. For book-length treatments, Drew Neil's *Practical Vim* [Nei15] and Chacon and Straub's *Pro Git* [CS14] are the standard recommendations, and Miller's *Recursive Make Considered Harmful* [Mil98] is worth the twenty minutes it takes to read.

Contents

I. Vim and Neovim **11**

1. Reading Vim Keystrokes **12**
1.1. Sequences, Not Chords . . . 12
1.2. Keycaps, Literal Characters and Case . . . 12
1.3. Modifier Notation: `<C-x>`, `<S-x>`, `<M-x>` . . . 13
1.4. Named Keys and Their Aliases . . . 13
1.5. `<leader>` and `<localleader>` . . . 14
1.6. The Shape of a Normal-Mode Command . . . 15
1.7. The Shape of an Ex Command . . . 15
1.8. Placeholders . . . 16
1.9. How Vim Names Keys in `:help` and `:map` . . . 17
1.10. Five Commands, Read Aloud . . . 17
1.11. Legend for Part I . . . 18

2. The Modal Mindset **19**
2.1. Why Modes . . . 19
2.2. The Modes . . . 19
2.3. Survival Kit: Open, Edit, Save, Quit . . . 20
2.4. Reading the Manual . . . 20
2.5. Options . . . 21

3. Motions **23**
3.1. Characters and Lines . . . 23
3.2. Words and WORDs . . . 23
3.3. Finding a Character on the Line . . . 23
3.4. Around the File, and Scrolling . . . 24
3.5. Search as a Motion . . . 24
3.6. Jumps, Marks and the Change List . . . 25

4. The Grammar: Operators and Text Objects **26**
4.1. Operator + Motion = Action . . . 26
4.2. The Operators . . . 26
4.3. Counts . . . 27
4.4. Text Objects . . . 27
4.5. Repeat: the Dot Command . . . 28
4.6. Composition Table . . . 28

5. Editing in Practice **30**
5.1. Entering and Using Insert Mode . . . 30
5.2. Undo, Redo and the Undo Tree . . . 30
5.3. Registers and the Clipboard . . . 31
5.4. Macros . . . 32
5.5. Visual and Blockwise Editing . . . 32
5.6. Substitution . . . 33
5.7. The Global Command . . . 33

6. Files, Buffers, Windows and Tabs **34**
6.1. Buffers, Windows and Tab Pages . . . 34
6.2. Working with Buffers . . . 34
6.3. Splits and Window Commands . . . 34
6.4. Tab Pages . . . 35
6.5. The Quickfix and Location Lists . . . 35
6.6. Files, Filters and the Shell . . . 36
6.7. Diff Mode . . . 36
6.8. Sessions . . . 36

7. Neovim **37**
7.1. What Neovim Changes . . . 37
7.2. Configuration in Lua . . . 37
7.3. Mappings and Autocommands . . . 38
7.4. Talking to Vim from Lua . . . 39
7.5. The Built-in Terminal . . . 40
7.6. Treesitter . . . 40
7.7. LSP: Language Servers . . . 40
7.8. Completion and Snippets . . . 41
7.9. Plugins and `lazy.nvim` . . . 42

8. The Setup on This Machine **43**
8.1. LazyVim . . . 43
8.2. The Keymap Worth Learning First . . . 43
8.3. VimTeX . . . 45
8.4. LuaSnip . . . 46
8.5. The LaTeX Workflow, End to End . . . 47

II. The Unix Shell **49**

9. The Shell and the Command Line **50**
9.1. Reading Shell Transcripts . . . 50
9.2. What a Shell Is . . . 51
9.3. Bash, Zsh, and Which One You Are Running . . . 52
9.4. Anatomy of a Command . . . 52
9.5. Quoting and Escaping . . . 53
9.6. Globbing and Brace Expansion . . . 54
9.7. Line Editing and History . . . 55
9.8. Getting Help . . . 56

10. The Filesystem **57**
10.1. Paths . . . 57
10.2. Navigating . . . 57
10.3. Listing and Inspecting . . . 58
10.4. Creating and Destroying . . . 59
10.5. Hard and Symbolic Links . . . 59
10.6. Permissions and Ownership . . . 60
10.7. Finding Files . . . 60

11. Viewing and Processing Text **62**
11.1. Reading Files . . . 62

11.2. Slicing Lines . . . 63
11.3. Ordering . . . 63
11.4. Comparing . . . 64
11.5. Regular Expressions . . . 64
11.6. `grep` . . . 65
11.7. `sed` . . . 66
11.8. `awk` . . . 66

12. Streams, Pipes and Redirection **68**
12.1. Standard Input, Output and Error . . . 68
12.2. Redirection . . . 68
12.3. Pipes and Filters . . . 69
12.4. `tee` and `xargs` . . . 69
12.5. Here-Documents and Here-Strings . . . 70
12.6. Command and Process Substitution . . . 71
12.7. Exit Status and the Logical Operators . . . 71

13. Processes and the Environment **73**
13.1. Processes . . . 73
13.2. Job Control . . . 73
13.3. Signals . . . 74
13.4. Environment Variables . . . 75
13.5. Startup Files . . . 76
13.6. Aliases and Functions . . . 76
13.7. Measuring . . . 77

14. Shell Scripting **78**
14.1. Shebangs and Executable Scripts . . . 78
14.2. Variables, Expansion and Arrays . . . 78
14.3. Arguments . . . 79
14.4. Conditionals . . . 80
14.5. Loops and Reading Input . . . 81
14.6. Functions . . . 82
14.7. Writing Robust Scripts . . . 82
14.8. Debugging Scripts . . . 83

15. Everyday Tools **84**
15.1. Archives and Compression . . . 84
15.2. Remote Work . . . 84
15.3. Fetching Things . . . 85
15.4. Terminal Multiplexing . . . 85
15.5. A Handful of One-Liners . . . 86

III. Make and Makefiles **88**

16. Why a Build System **89**
16.1. Notation for Part III . . . 89
16.2. Compiling by Hand, and Where It Breaks . . . 89
16.3. Dependency Graphs and Incremental Builds . . . 90
16.4. Make's Model . . . 90
16.5. Where Make Sits . . . 91

17. Anatomy of a Makefile **92**
17.1. The Rule . . . 92
17.2. The Tab Character . . . 92
17.3. The Default Goal . . . 93
17.4. How Recipes Are Run . . . 94
17.5. A First Makefile . . . 94

18. Variables and Expansion **96**
18.1. Recursive versus Simple Assignment . . . 96
18.2. Conditional and Appending Assignment . . . 96
18.3. Automatic Variables . . . 97
18.4. Functions and Substitution References . . . 97
18.5. Conventional Variables . . . 98
18.6. Overriding from Outside . . . 99

19. Pattern and Implicit Rules **101**
19.1. The Built-in Rule Database . . . 101
19.2. Pattern Rules . . . 101
19.3. Static Pattern Rules . . . 102
19.4. `VPATH` and Out-of-Tree Builds . . . 103
19.5. Automatic Dependency Generation . . . 103

20. Phony Targets and Project Conventions **105**
20.1. `.PHONY` and Why It Matters . . . 105
20.2. The Standard Target Names . . . 105
20.3. Order-Only Prerequisites and Directories . . . 106
20.4. Recursive Make, and Its Problems . . . 107

21. Running and Debugging Make **109**
21.1. Command-Line Options . . . 109
21.2. Parallel Builds . . . 109
21.3. Printing from Inside a Makefile . . . 110
21.4. Inspecting the Database . . . 111
21.5. Error Messages You Will Meet . . . 112

22. Worked Makefiles **113**
22.1. A Multi-File C Project . . . 113
22.2. A LaTeX Document . . . 115
22.3. A Small Data Pipeline . . . 115

IV. GDB **117**

23. Preparing to Debug **118**
23.1. Notation for Part IV . . . 118
23.2. Compiling with `-g` . . . 118
23.3. Debug Symbols and Stripped Binaries . . . 119
23.4. Core Dumps . . . 120
23.5. Ways to Start GDB . . . 120
23.6. The Prompt, Abbreviations and Help . . . 121

24. Controlling Execution **123**
24.1. Starting and Stopping . . . 123

24.2. Stepping . . . 123
24.3. Source Lines versus Instructions . . . 124
24.4. Changing the Flow . . . 125

25. Breakpoints and Watchpoints **126**
25.1. Setting Breakpoints . . . 126
25.2. Conditions and Ignore Counts . . . 126
25.3. Listing, Disabling and Deleting . . . 127
25.4. Watchpoints . . . 127
25.5. Catchpoints . . . 128
25.6. Breakpoint Command Lists . . . 129

26. The Stack and the Source **130**
26.1. Reading a Backtrace . . . 130
26.2. Selecting Frames . . . 130
26.3. Viewing Source . . . 131
26.4. Locals and Arguments . . . 132
26.5. Inlined and Optimised Frames . . . 132

27. Inspecting and Changing Data **134**
27.1. `print` and Format Letters . . . 134
27.2. `x`: Examining Memory . . . 135
27.3. Types . . . 136
27.4. `display` . . . 136
27.5. Changing Variables and Registers . . . 137
27.6. Pointers, Arrays and Structs . . . 137
27.7. Convenience Variables and Value History . . . 138

28. Debugging Real Failures **139**
28.1. Segmentation Faults . . . 139
28.2. Stack Overflow and Runaway Recursion . . . 140
28.3. Uninitialised Memory and Heap Corruption . . . 140
28.4. Multi-Threaded Programs . . . 141
28.5. Handling Signals . . . 142
28.6. Companion Tools . . . 142

29. Making GDB Comfortable **143**
29.1. The TUI . . . 143
29.2. `.gdbinit` and User Commands . . . 143
29.3. Pretty-Printers and Python . . . 145
29.4. Front Ends, and GDB from Neovim . . . 145
29.5. A Worked Debugging Session . . . 146

V. Git and GitHub **148**

30. Version Control and Git's Model **149**
30.1. Notation for Part V . . . 149
30.2. Why Version Control . . . 149
30.3. Snapshots, Not Diffs . . . 150
30.4. The Object Database . . . 150
30.5. The Three Trees . . . 151
30.6. Refs, `HEAD`, and What a Branch Really Is . . . 152

30.7. First-Time Setup . . . 153

31. The Everyday Loop **154**
31.1. Starting a Repository . . . 154
31.2. `status` and the Index . . . 154
31.3. Staging . . . 155
31.4. Committing . . . 156
31.5. `diff`: Three Comparisons . . . 157
31.6. Reading History . . . 157
31.7. `.gitignore` . . . 158
31.8. Undoing . . . 158

32. Branching and Merging **160**
32.1. Creating and Switching . . . 160
32.2. Fast-Forward and Three-Way Merges . . . 160
32.3. Conflicts . . . 161
32.4. Rebase . . . 162
32.5. Interactive Rebase . . . 163
32.6. `cherry-pick` . . . 164
32.7. Stashing . . . 164
32.8. Worktrees . . . 164

33. Remotes and Collaboration **165**
33.1. Remotes . . . 165
33.2. `fetch` versus `pull` . . . 165
33.3. `push` and Tracking . . . 166
33.4. Merge or Rebase on Pull . . . 166
33.5. Tags and Releases . . . 166
33.6. Submodules and Subtrees . . . 167

34. GitHub **168**
34.1. Repositories, Forks and Clones . . . 168
34.2. Authentication . . . 168
34.3. Pull Requests . . . 169
34.4. Reviewing Code . . . 169
34.5. Issues and Projects . . . 170
34.6. The `gh` Command-Line Tool . . . 170
34.7. GitHub Actions . . . 170
34.8. Pages, Releases and `README` Conventions . . . 171

35. Working on a Team **173**
35.1. Branching Models . . . 173
35.2. Commit Hygiene . . . 173
35.3. Protecting `main` . . . 174
35.4. Keeping a Fork in Sync . . . 174
35.5. Hooks . . . 175
35.6. Licences and Repository Hygiene . . . 175

36. When Things Go Wrong **176**
36.1. `reflog`: the Safety Net . . . 176
36.2. Recovering Lost Commits . . . 176
36.3. Fixing the Last Commit . . . 177

36.4. Splitting and Reordering . . . 177
36.5. `bisect` . . . 177
36.6. Archaeology . . . 178
36.7. Rewriting History Safely . . . 179
36.8. Large Files and Secrets . . . 179

A. Vim Reference Card **181**
A.1. Modes . . . 181
A.2. Motions . . . 181
A.3. Operators and Text Objects . . . 181
A.4. Registers, Macros and Search . . . 182
A.5. Files, Buffers and Windows . . . 182
A.6. Neovim . . . 182

B. Shell Reference Card **184**
B.1. Files and Navigation . . . 184
B.2. Text Processing . . . 184
B.3. Redirection and Pipes . . . 185
B.4. Processes and Jobs . . . 185
B.5. Scripting Idioms . . . 185

C. Make Reference Card **187**
C.1. Rule Syntax . . . 187
C.2. Variables . . . 187
C.3. Automatic Variables . . . 187
C.4. Built-in Functions . . . 188
C.5. Command-Line Options . . . 188
C.6. Diagnosing a Makefile . . . 188

D. GDB Reference Card **190**
D.1. Starting and Stopping . . . 190
D.2. Breakpoints and Watchpoints . . . 190
D.3. Stepping and Frames . . . 191
D.4. Examining Data . . . 191
D.5. Threads, Signals and Comfort . . . 191
D.6. Choosing the Right Tool . . . 192

E. Git Reference Card **193**
E.1. The Everyday Loop . . . 193
E.2. Branching and Merging . . . 193
E.3. Remotes . . . 194
E.4. Inspecting History . . . 194
E.5. Undo and Recovery . . . 194
E.6. Diagnosing Git . . . 195

F. Configuration Files **196**
F.1. A Minimal `init.lua` . . . 196
F.2. A Reusable Makefile Skeleton . . . 198
F.3. A `.gdbinit` Worth Having . . . 198
F.4. A `.gitconfig` Worth Having . . . 199

G. This Machine **200**
G.1. Versions . . . 200

G.2. Installed and Not Installed 200
G.3. The Editor Configuration 201

Part I.

Vim and Neovim

Vim is not an editor with a strange set of shortcuts. It is a small *language* for describing edits, and the keyboard is its alphabet. Once you can read `c` `i` `(` as *"change inside parentheses"*, the hundreds of commands collapse into a grammar of perhaps thirty words.

Chapter 1 is that alphabet, and it is not optional — everything afterwards, and every Vim answer you will ever read online, is written in the notation it defines. Chapters 2 to 6 cover Vim proper, which works unchanged in Vim, Neovim, and the Vim emulation layer of almost every other editor, so the investment never expires. Chapter 7 covers what Neovim adds on top, and Chapter 8 walks through the configuration actually installed on this machine, so the abstractions have something concrete to point at.

1. Reading Vim Keystrokes

Vim's manual, every tutorial, every plugin README and every answer you will ever read online describe commands in a compact notation — `dw`, `ci"`, `<C-w>v`, `"a3dd`, `:%s/old/new/gc` — and almost none of them explain it. That notation is not decoration. It is a precise description of what your fingers do, and once you can read it, most of Vim's documentation becomes usable without a tutorial in between.

This chapter contains no commands worth memorising. It is the key to the rest of Part I, and it is short. Read it once now, and come back to Section 1.11 whenever a line of this note looks like line noise.

§1.1. Sequences, Not Chords

The single most important thing to understand: **Vim commands are sequences of key-presses, not simultaneous chords.**

In a conventional editor, "delete line" might be `<C-S-k>` — hold Control, hold Shift, tap `k`, release. In Vim it is `d` `d`: press `d`, release it, press `d` again. Nothing is held down.

> **Notation .** When this note prints two or more keycaps side by side, like `d` `w` or `g` `U` `i` `w`, it means *press them one after another, in that order*. The gaps between keycaps are there for legibility; they do not mean pause.

Remark 1.1.1 (Does timing matter?)**.** For built-in commands, no — Vim will wait indefinitely between `d` and its motion. For *user-defined mappings* it can: if `j` `k` is mapped to `<Esc>`, then after you press `j` Vim waits `'timeoutlen'` milliseconds (1000 by default) to see whether a `k` is coming. That delay is the only place typing speed changes the meaning of a sequence. See `:help 'timeout'`.

§1.2. Keycaps, Literal Characters and Case

A keycap in this note shows the *character you must produce*, not the physical key you press. `$` means "produce a dollar sign", which on a US layout is Shift held with `4`; we never write that as `<S-4>`, because Vim does not think of it that way and neither should you.

> **Case is significant, always. `d` and `D` are different commands, not the same command shouted.**

The relationship between a lowercase command and its uppercase partner is not random. Two patterns cover most of them:

"to the end of the line"	`D` = `d` `$`, `C` = `c` `$`, `Y` = `y` `$` in Neovim
"the bigger version"	`w` moves a word, `W` a whitespace-delimited WORD; `f` searches forward, `F` backward; `p` pastes after, `P` before

Digits are a special case. In Normal mode `1` through `9` are never commands — they begin a **count** (Section 1.6). `0` is the exception: on its own it is the motion "go to column zero", but typed after another digit it is just part of the number, which is why `1` `0` `j` means "down ten lines" and not "down one line, then column zero".

The punctuation keys used throughout Part I, with the character they stand for:

`$` `^` `%` `&` `*`	Produced with Shift on a US layout
`{` `}` `[` `]` `(` `)`	Braces, brackets, parentheses — always distinguished
`;` `,` `.` `:`	Semicolon, comma, period, colon — four different commands
`‘` and `’`	Backtick and apostrophe — also two different commands
`"`	One double-quote character, the register prefix

Note 1.2.1. `;` and `:` are a common beginner trap, as are `‘` and `’`. If a command “does nothing”, check which of the pair you actually typed.

§1.3. Modifier Notation: <C-x>, <S-x>, <M-x>

Modifiers *are* chords, and they are the only chords Vim uses. Vim writes them inside angle brackets, and so does this note — because that spelling is also exactly what you type into a mapping.

`<C-x>`	Hold **Control**, press `x`. Sometimes written `CTRL-X` in the manual.
`<S-x>`	Hold **Shift**, press `x` — i.e. `X`
`<M-x>`	Hold **Meta / Alt**. `<A-x>` is a synonym
`<D-x>`	Hold **Command**. macOS GUI builds only
`<C-S-x>`	Control *and* Shift together

Note 1.3.1 (Case inside a modifier)**.** `<C-a>` and `<C-A>` are the *same* key. Control strips the case bit, so a terminal cannot tell them apart, and Vim treats both spellings as `CTRL-A`. `<C-S-a>` is genuinely different — but most terminal emulators cannot transmit it at all, which is why you rarely see it in configurations meant to be portable.

Remark 1.3.2 (Why some Control keys are secretly other keys)**.** This is inherited from ASCII, not invented by Vim. Control-plus-letter produces the control code at that letter’s position in the alphabet, and four of those codes already have names:

`<C-i>`	*is* `<Tab>`
`<C-m>`	*is* `<CR>`
`<C-[>`	*is* `<Esc>`
`<C-h>`	*is* `<BS>`

They are indistinguishable at the terminal, so remapping one remaps the other. This is why `<C-i>` (jump forward) cannot be given a new meaning without breaking `<Tab>`, and it is a question that comes up constantly on mailing lists. Neovim in a modern terminal can sometimes separate them; see `:help tui-modifyOtherKeys`.

§1.4. Named Keys and Their Aliases

Keys with no printable character get a name in angle brackets. Vim accepts several spellings for some of them; this note picks one and uses it everywhere.

`<Esc>`	Escape. Also `<C-[>`
`<CR>`	Enter / Return. Also spelled `<Enter>`, `<Return>`, `<NL>`
`<Tab>`	Tab. Also `<C-i>`
`<BS>`	Backspace. Also `<C-h>`
`<Space>`	The space bar
`<Del>`	Forward delete
`<Up>` `<Down>` `<Left>` `<Right>`	Arrow keys
`<Home>` `<End>` `<PageUp>` `<PageDown>`	As labelled
`<F1>`–`<F12>`	Function keys
`<lt>`	A literal `<`, needed when `<` would otherwise open a key name
`<Bar>`	A literal `|`, needed where `|` separates commands
`<Nop>`	"Do nothing" — the target for disabling a key

Example 1.4.1. To make `<Space>` do nothing before using it as a leader, a configuration writes `vim.keymap.set("n", "<Space>", "<Nop>")`. The angle-bracket names are literal strings there — you type the characters `<`, `S`, `p`, `a`, `c`, `e`, `>`, not the space bar.

Angle brackets have two jobs, and context tells them apart: inside a mapping or a config file they are *literal text* describing a key; in this note's keycaps they are a *picture* of the key you press.

§1.5. `<leader>` and `<localleader>`

Definition 1.5.1 (Leader)**.** `<leader>` is not a key. It is a placeholder for whatever key the variable `mapleader` currently holds, expanded when a mapping is *defined*. Its purpose is to give you a private prefix that no built-in command uses, so your own mappings cannot collide with Vim's.

Vim's factory default is `\`. Essentially every modern configuration — LazyVim included — changes it to `<Space>`, on the grounds that it is the largest, most easily reached key on the board and does nothing useful in Normal mode.

`<localleader>` is the same idea for mappings that only make sense in one kind of file: LaTeX commands in a `.tex` buffer, test-running commands in a `.py` buffer. It stays at `\` in most setups, including this machine's (Chapter 8).

`:echo mapleader`	What is my leader? (errors if never set)
`:echo maplocalleader`	Likewise for the local leader
`:verbose nmap <Space>`	Every mapping starting with Space, and where each was defined

Notation . Throughout Part I, `<leader>` means `<Space>` and `<localleader>` means `\`, matching the configuration described in Chapter 8. So `<leader>` `f` `f` is three presses: space, `f`, `f`.

Note 1.5.2. Because the leader is expanded at definition time, changing `mapleader` halfway through a configuration leaves earlier mappings on the old key and later ones on the new. This is why every config sets it on the first line, before any plugin loads.

§1.6. The Shape of a Normal-Mode Command

Almost every Normal-mode command fits one template. Learn the template and you can parse commands you have never seen.

```
["{reg}]  [{count}]  {operator}  [{count}]  {motion}
```

Square brackets mark optional slots; braces mark something you must supply. Reading left to right:

`"{reg}`	*Optional.* Which register to use — a double-quote followed by one character. Omitted, the unnamed register is used (Section 5.3).
`[{count}]`	*Optional.* A number, repeating what follows
`{operator}`	The verb: `d`, `c`, `y`, `>`, `g U`, …
`[{count}]`	*Optional again.* A second count multiplies with the first
`{motion}`	The noun: a motion (Chapter 3) or a text object (Section 4.4)

Example 1.6.1 (Parsing `"a3d2w`)**.**

`" a`	into register `a`
`3`	three times …
`d`	… delete …
`2`	… two …
`w`	… words

Six keypresses: *delete six words into register* `a`. The counts multiply, $3 \times 2 = 6$.

Two abbreviations complete the picture, and both were mentioned in Section 1.2:

Doubling	An operator typed twice acts on the whole line: `d d`, `y y`, `> >`
Capitalising	The uppercase form usually means “to end of line”: `D`, `C`

Commands that are not operators take only a count:

```
[{count}]  {command}
```

so `5 j` is “down five lines” and `3 x` is “delete three characters”.

Remark 1.6.2 (Operator-pending mode)**.** Between the operator and the motion, Vim is in a mode of its own, waiting for the noun. You never enter it deliberately, but its existence explains two things: why `d` alone appears to hang (it is waiting), and why plugins can invent new nouns like `i f` that work with every operator at once.

§1.7. The Shape of an Ex Command

Commands typed after `:` follow a different template:

```
:[{range}]  {command}[!]  [{arguments}]  <CR>
```

`{range}`	*Optional.* Which lines to act on. Omitted, most commands act on the current line
`{command}`	The command name, which may be abbreviated
`!`	*Optional.* "Force" — overwrite, discard, do not ask
`{arguments}`	Whatever the command takes
`<CR>`	Required. Nothing runs until you press Enter

Ranges are worth knowing in their own right:

`.`	The current line (the default)
`$`	The last line of the file
`%`	The whole file — shorthand for `1,$`
`1,5`	Lines 1 to 5
`.,+3`	From here to three lines on
`'<,'>`	The last Visual selection; typed for you by pressing `:` in Visual mode
`'a,'b`	From mark `a` to mark `b`
`/pat/`	The next line matching `pat`

> **Notation** (Abbreviations in the manual)**.** Vim's help writes Ex commands as `:s[ubstitute]` or `:w[rite]`. The bracketed tail is the part you may omit: `:s`, `:su`, `:subs` and `:substitute` are all the same command. This note writes out whichever form is clearer in context and does not use the bracket convention.

Example 1.7.1 (Parsing `:%s/old/new/gc`)**.**

`:`	enter Command-line mode
`%`	range: every line in the file
`s`	command: `substitute`
`/old/new/`	arguments: pattern, then replacement
`g`	flag: every match on each line, not just the first
`c`	flag: confirm each one
`<CR>`	run it

§1.8. Placeholders

Where a command needs an argument that varies, this note follows Vim's own convention: **braces mean required, square brackets mean optional.**

`{motion}`	Replace with any motion: `w`, `$`, `i` `(`, …
`{char}`	Replace with one literal character
`{reg}`	Replace with one register name
`{pattern}`	Replace with a search pattern
`[count]`	Optionally, a number
`x`	In examples only: stands for "some particular character"
…	Elision: keystrokes omitted because they vary

Example 1.8.1. `f{char}` is the general form; `f,` is one instance of it. `<C-v>` … `I text <Esc>` means: start a blockwise selection, extend it however you like, then press `I`, type the

text, and press `<Esc>`.

§1.9. How Vim Names Keys in `:help` and `:map`

Vim's help files are organised by *mode plus key*, and the mode is a prefix on the topic name. This is the single most useful thing to know about `:help`, and Section 2.4 returns to it:

`:help x`	The Normal-mode command `x` — no prefix
`:help i_CTRL-r`	`<C-r>` in **I**nsert mode
`:help v_o`	`o` in **V**isual mode
`:help c_CTRL-r`	`<C-r>` on the **c**ommand line
`:help o_v`	`v` in **o**perator-pending mode
`:help t_CTRL-\`	`<C-\>` in **t**erminal mode
`:help 'number'`	An *option* — single quotes
`:help :write`	An *Ex command* — leading colon
`:help CTRL-W_v`	A two-key command, joined with an underscore

Note that the manual writes `CTRL-R` where this note prints `<C-r>`. They are the same key; `CTRL-R` is the prose spelling and `<C-r>` is the spelling you type into a mapping.

The `:map` family prints mappings in a table whose first column is a set of mode letters, and whose annotations are easy to misread:

`n i v x s o t c`	The mode: Normal, Insert, Visual+Select, Visual, Select, operator-pending, Terminal, Command-line
`*`	The right-hand side is *not* remappable (`noremap`)
`&`	Only script-local mappings are remapped
`@`	The mapping is buffer-local

Example 1.9.1. `:verbose nmap <leader>w` prints the mapping *and* the file and line that defined it. When a key does something you did not ask for, this is the command that names the culprit. The same `:verbose` prefix works on `:set` (`:verbose set shiftwidth?`) and on `:autocmd`.

§1.10. Five Commands, Read Aloud

Nothing below needs to be understood yet — the chapters that follow explain every piece. The exercise is purely to show that the notation always decomposes.

Example 1.10.1 (1. `dw`)**.** `d` `w` = operator `d` (delete) + motion `w` (to the start of the next word). *Delete a word.*

Example 1.10.2 (2. `ci"`)**.** `c` `i` `"` = operator `c` (change) + text object `i` `"` (inside the double quotes). *Replace the contents of a string, leaving the quotes.* Note that `i` here is not "insert" — inside a text object it means *inner*.

Example 1.10.3 (3. `"a3dd`)**.** `"` `a` (register `a`) + `3` (count) + `d` `d` (delete, doubled, so linewise). *Cut three lines into register* `a`.

Example 1.10.4 (4. `<C-w>v`)**.** `<C-w>` is a chord — Control held with `w` — and it is the prefix for every window command. Then `v`. *Split the window vertically.* Two presses, the first of which is a chord and the second of which is not.

Example 1.10.5 (5. `qqA;<Esc>jq` then `5@q`).

`q` `q`	start recording a macro into register `q`
`A`	append at end of line (enters Insert mode)
`;`	type a semicolon
`<Esc>`	leave Insert mode
`j`	move down one line
`q`	stop recording
`5` `@` `q`	replay the macro five times

Put a semicolon at the end of six consecutive lines. Eleven keypresses, none of them held down, and every one of them a command you will meet again.

§1.11. Legend for Part I

Every typographic device used from here on:

`d` `w`	A keystroke sequence. Press in order, nothing held
`<C-w>`	A chord: Control held with `w`
`:write`	An Ex command, typed after `:`
`'ignorecase'`	An option name — Vim writes these in single quotes
`~/.config/nvim/`	A file path
`vim.opt.number`	Lua or Vimscript code, inline
text object	A term being defined for the first time
`{motion}`, `[count]`	Required and optional placeholders

Boxes carry a consistent meaning:

Definition	A term you will need again
Note	A caveat or a detail that is easy to get wrong
Remark	Background: why something is the way it is
Intuition	The mental model, stated plainly
Example	A concrete case, usually with keystrokes
Try it	A drill to do at the keyboard before reading on
Moral	The one sentence worth remembering from the section

If a line of Vim notation looks unreadable, it is almost always because you are trying to read it as one symbol. Split it into register, count, verb, noun — and it will read as a sentence.

2. The Modal Mindset

§2.1. Why Modes

Every other editor you have used is **modeless**: the keyboard always inserts text, and commands are reached by holding modifiers (`<C-c>`, `<C-v>`, `<C-S-k>`) or by leaving the keyboard for the mouse. This is a reasonable design, but it wastes the most valuable real estate on the machine — the twenty-six letter keys — on a single operation.

> **Intuition .** **Modal editing** observes that you spend far more time *reading and restructuring* code than typing it fresh. So Vim gives the letter keys a second life: in **Normal mode** they are commands, and you enter **Insert mode** only for the short bursts when you are genuinely producing new characters.

The consequence is that commands become *composable* in a way that modifier-key shortcuts never can be. `<C-S-k>` deletes a line and that is all it will ever do; `d` is an operator that combines with any motion, so `d` `w`, `d` `$`, `d` `i` `(` and `d` `/` `f` `o` `o` `<CR>` all follow from one key. We will make this precise in Chapter 4 — it is the single idea worth taking away from this note.

Normal mode is home. You should be in Insert mode only while text is actually appearing on screen, and you should leave it the moment you stop.

§2.2. The Modes

Definition 2.2.1 (Normal mode)**.** The default mode, entered with `<Esc>`. Keys are **commands**: they move the cursor, delete, copy, paste and repeat. If you are ever unsure what mode you are in, press `<Esc>` twice.

Definition 2.2.2 (Insert mode)**.** Typed characters are inserted into the buffer, as in any ordinary editor. Entered with `i`, `a`, `o` and friends (Section 5.1), left with `<Esc>` or `<C-[>`.

Definition 2.2.3 (Visual mode)**.** Select a region first, then act on it — the reverse of Normal mode's verb-then-object order. Three flavours:

`v`	**charwise**: select character by character
`V`	**linewise**: select whole lines
`<C-v>`	**blockwise**: select a rectangle, for column edits

Definition 2.2.4 (Command-line mode)**.** Entered with `:` (Ex commands such as `:write`), `/` or `?` (search), or `!` (filter through a shell command). One line is typed at the bottom of the screen and executed with `<CR>`; `<Esc>` aborts.

Definition 2.2.5 (Replace mode)**.** Entered with `R`. Typed characters overwrite existing ones instead of pushing them right. Rarely needed; `r` (replace exactly one character) is the version you will actually use.

Definition 2.2.6 (Terminal mode)**.** A Neovim addition: inside a `:terminal` buffer, keystrokes go to the shell. Return to Normal mode with `<C-\>` `<C-n>`. See Section 7.5.

Remark 2.2.7. Vim also has **Operator-pending mode** — the sliver of time after you press `d` but before you have said *what* to delete. You will never enter it deliberately, but it is why `d` `d` works: the second `d` is parsed as "the whole line" only because Vim was waiting for a motion.

---

§2.3. Survival Kit: Open, Edit, Save, Quit

Enough to open a file, change it, and get out alive.

`nvim file.tex`	Open a file from the shell
`i`	Enter Insert mode; type something
`<Esc>`	Back to Normal mode
`:w<CR>`	**Write** (save) the file
`:q<CR>`	**Quit**
`:wq<CR>`	Write and quit (`Z` `Z` is the Normal-mode shorthand)
`:q!<CR>`	Quit, discarding all changes
`:w newname<CR>`	Write to a different file
`u`	Undo
`<C-r>`	Redo

Note 2.3.1 (The `!` suffix)**.** Appending `!` to an Ex command means *force*, or "yes, I am sure". `:q` refuses to quit a modified buffer; `:q!` throws the changes away. `:qa!` does it for every open buffer at once.

Note 2.3.2 (If you are stuck)**.** Three situations trap beginners, and all three have the same escape:

(i) You pressed `<C-s>` out of habit and the terminal froze — press `<C-q>` to resume.

(ii) You are in the middle of a half-typed command — `<Esc>`.

(iii) You entered Ex mode (`Q` in Vim) and see a `:` prompt that will not go away — type `:visual<CR>`. Neovim removed this trap entirely.

Try it

Open a scratch file with `nvim /tmp/scratch.txt`, type a few lines, save with `:w`, quit with `:q`, reopen it, and delete everything you wrote with `u`. Do this until saving and quitting requires no thought at all — the rest of the note assumes it.

§2.4. Reading the Manual

Vim's documentation is unusually good, and it is available offline, inside the editor, with no network round trip. Learning to navigate it is worth more than any cheat sheet.

`:help`	The table of contents
`:help dd`	Help for the Normal-mode command `d` `d`
`:help i_CTRL-r`	Help for `<C-r>` *in Insert mode*
`:help v_o`	Help for `o` *in Visual mode*
`:help 'number'`	Help for an *option* — note the single quotes
`:help :write`	Help for an Ex command — note the leading colon
`:helpgrep <pattern>`	Full-text search across all help files
`<C-]>`	Follow the tag under the cursor
`<C-o>`	Jump back where you came from

So the mode you are asking about is encoded in the prefix: `i_` for Insert, `v_` for Visual, `c_` for the command line, `o_` for operator-pending, and no prefix for Normal. Tab completion works on the help argument, which is the fastest way to discover the prefixes.

Example 2.4.1. You have forgotten what `g` `U` does. Type `:help gU` and read. You want to know why your search is case-insensitive: `:help 'ignorecase'`. You saw `<C-a>` in someone's config and want to know if it is safe to remap: `:help CTRL-A`.

Note 2.4.2 (`vimtutor`)**.** Vim ships with a thirty-minute interactive tutorial. Run `vimtutor` in a shell (or `:Tutor` inside Neovim). It is the single highest-return half hour available to a beginner, and this chapter is not a substitute for it.

§2.5. Options

Behaviour is controlled by **options**, set with `:set`. Three forms matter:

`:set number`	Turn a boolean option on
`:set nonumber`	Turn it off (prefix `no`)
`:set number!`	Toggle it (suffix `!`)
`:set number?`	Ask its current value (suffix `?`)
`:set tabstop=4`	Set a numeric or string option
`:setlocal ...`	Apply to the current buffer/window only

A handful you will want on day one:

`'number'`, `'relativenumber'`	Absolute and relative line numbers; together they give the **hybrid** display that makes counts like `7` `j` easy to read off
`'ignorecase'`, `'smartcase'`	Case-insensitive search, unless the pattern contains a capital
`'expandtab'`, `'shiftwidth'`	Insert spaces instead of tabs; how far `>` `>` indents
`'wrap'`, `'linebreak'`	Soft-wrap long lines, and wrap at word boundaries
`'scrolloff'`	Keep n lines of context above and below the cursor
`'clipboard'`	Set to `unnamedplus` to share yanks with the system clipboard (Section 5.3)

Typed at the `:` prompt, an option lasts for the session only. To make it permanent it goes in your configuration file — `~/.vimrc` for Vim, `~/.config/nvim/init.lua` for Neovim, where the Lua spelling is `vim.opt.number = true` (Section 7.2).

Remark 2.5.1. Neovim already switches on many options that Vim leaves off for compatibility with 1976: `'nocompatible'`, `'incsearch'`, `'hlsearch'`, `'autoindent'`, syntax highlighting and filetype plugins are all default. A Neovim configuration is therefore much shorter than the equivalent `.vimrc`; see `:help nvim-defaults` for the full list.

3. Motions

A **motion** is any command that moves the cursor. Motions matter far beyond navigation, because Chapter 4 will feed every one of them to an operator: learning a new motion automatically teaches you a new way to delete, change, copy and indent. Read this chapter as vocabulary building, not as a tour of the arrow keys.

§3.1. Characters and Lines

`h` `j` `k` `l`	Left, down, up, right
`0`	First character of the line (column zero)
`^`	First *non-blank* character — usually what you want
`$`	End of the line
`g` `_`	Last non-blank character
`+` / `-`	First non-blank of the next/previous line

> **Intuition .** `h` `j` `k` `l` sit under the right hand in the home row, which is the entire argument for them: your hand never leaves its resting position. The usual advice to disable the arrow keys is sound, but temporary — once the habit forms you will find you were never reaching for them anyway.

Remark 3.1.1 (Soft-wrapped lines)**.** With `'wrap'` on, `j` moves a whole logical line, which may be several screen rows. Prefix with `g` — `g` `j`, `g` `k`, `g` `0`, `g` `$` — to move by *display* line instead. Indispensable when editing prose or LaTeX.

§3.2. Words and WORDs

`w` / `b`	Forward / backward to the start of a word
`e` / `ge`	Forward / backward to the end of a word
`W` `B` `E`	The same, by **WORD**

Definition 3.2.1 (word vs. WORD)**.** A lowercase **word** is a run of letters, digits and underscores, or a run of punctuation. An uppercase **WORD** is any run of non-blank characters, delimited only by whitespace. In `foo_bar.baz()` there are five words but one WORD.

Example 3.2.2. On the line `self.config[key] = value`, pressing `w` repeatedly stops at `self`, `.`, `config`, `[`, `key`, `]`, `=`, `value` — eight stops. `W` makes three: `self.config[key]`, `=`, `value`. Use `W` when punctuation is noise, `w` when it is structure.

§3.3. Finding a Character on the Line

The most under-used motions in Vim, and the ones that most reward practice.

`f` `x`	Jump *onto* the next `x` on this line
`F` `x`	Jump onto the previous `x`
`t` `x`	Jump *till* just before the next `x`
`T` `x`	Jump till just after the previous `x`
`;`	Repeat the last `f`/`t` search forward
`,`	Repeat it backward

> **Intuition .** Think **f**ind and **t**ill. The distinction exists entirely so that operators land in the right place: `d` `f` `,` deletes up to and including the comma, while `d` `t` `,` leaves the comma behind.

Example 3.3.1. Cursor at the start of `foo(alpha, beta)`. To change the first argument, `c` `t` `,` — "change till comma". To delete both arguments and the parentheses' contents, `d` `i` `(` (Section 4.4).

§3.4. Around the File, and Scrolling

`g` `g`	First line of the file
`G`	Last line of the file
`42` `G`	Line 42 (equivalently `:42<CR>`)
`{` / `}`	Previous / next blank-line-delimited paragraph
`(` / `)`	Previous / next sentence
`%`	Jump to the matching bracket, or with a count, to that percentage of the file
`H` `M` `L`	**H**ighest, **M**iddle, **L**owest line on screen

Scrolling is distinct from motion — these move the window, dragging the cursor along only when it would otherwise leave the screen:

`<C-d>` / `<C-u>`	Half a screen down / up
`<C-f>` / `<C-b>`	A full screen forward / back
`<C-e>` / `<C-y>`	One line down / up
`z` `z`	Redraw with the cursor line centred
`z` `t` / `z` `b`	. . . at the top / bottom

Note 3.4.1. `z` `z` deserves to be a reflex. After any long jump, centring the view costs two keystrokes and saves you from reading code at the very bottom edge of the terminal.

---

§3.5. Search as a Motion

`/` `pat` `<CR>`	Search forward for `pat`
`?` `pat` `<CR>`	Search backward
`n` / `N`	Next / previous match, in the same direction
`*` / `#`	Search for the word under the cursor, forward / backward
`:nohlsearch`	Clear the highlighting until the next search

Search is itself a motion, so it composes: `d` `/` `end` `<CR>` deletes from the cursor up to the next occurrence of `end`. Offsets refine where it lands — `/` `pat` `/` `e` `<CR>` puts the cursor on the last character of the match rather than the first.

Remark 3.5.1 (Regular expressions)**.** Vim's regex dialect is its own. By default `+`, `?`, `|`, `(` and `)` are *literal* and must be backslash-escaped, which is a constant irritation if you come from PCRE. Prefix the pattern with `\v` ("very magic") to get the familiar behaviour: `/` `\v(foo|bar)+` `<CR>`. See `:help /magic`.

Example 3.5.2. Useful atoms: `\<` and `\>` anchor word boundaries, so `/` `\<if\>` will not match `diff`. `*` is exactly this — it searches for `\<word\>` under the cursor. Use `g` `*` for the unanchored version.

§3.6. Jumps, Marks and the Change List

Vim records every "long" move — searches, `G`, `%`, `{` — in a per-window **jump list**, so you can retrace your steps.

`<C-o>`	Go back to the previous position in the jump list
`<C-i>`	Go forward again
`‘` `‘`	Back to the position before the latest jump
`‘` `.`	To the position of the last change
`:jumps`	Show the jump list
`g` `;` / `g` `,`	Walk backward/forward through the **change list**

Definition 3.6.1 (Mark)**.** A named bookmark. `m` `a` sets mark `a` at the cursor; `‘` `a` returns to the exact position, and `’` `a` to the first non-blank of that line. Lowercase marks are local to the buffer, uppercase marks are global — `m` `T` in a file and `‘` `T` anywhere later will open it again.

Try it

Open a long file. Jump to the end with `G`, search for something with `/`, then press `<C-o>` three times to walk back to where you started and `<C-i>` to return. This pair replaces most uses of a mouse and a scrollbar.

Prefer the motion that expresses your *intent*. "Down eleven lines" is a description of the screen; "to the next `return`" is a description of the code, and it keeps working after the code moves.

4. The Grammar: Operators and Text Objects

This is the chapter that makes Vim worth learning. Everything so far has been vocabulary; here is the syntax that multiplies it.

§4.1. Operator + Motion = Action

Almost every editing command in Vim parses as

$$\underbrace{\texttt{[count]}}_{\text{how many}}\ \underbrace{\texttt{operator}}_{\text{verb}}\ \underbrace{\texttt{[count]}}_{\text{how many}}\ \underbrace{\texttt{motion or text object}}_{\text{noun}}$$

Reading `d` `2` `w` as "delete two words" is not a mnemonic invented after the fact — it is how the command is actually parsed. The payoff is combinatorial: n operators and m nouns give you nm commands to memorise, but only $n + m$ things to learn.

> **Never memorise a Vim command as an atom. Decompose it. `c` `i` `"` is not "the change-string command", it is *change* + *inside* + *quotes*, and each piece is reusable.**

§4.2. The Operators

`d`	**Delete** (and store in a register)
`c`	**Change**: delete, then enter Insert mode
`y`	**Yank** (copy)
`>` / `<`	Indent / dedent by `'shiftwidth'`
`=`	Re-indent using the current indent rules
`g` `u` / `g` `U`	Lowercase / uppercase
`g` `~`	Toggle case
`g` `q`	Reformat (hard-wrap) to `'textwidth'`
`!`	Filter through an external shell command

Doubling an operator applies it to the current line: `d` `d` deletes a line, `y` `y` yanks one, `>` `>` indents one, `g` `q` `q` rewraps one. Capitalising usually means "to the end of the line": `D` = `d` `$`, `C` = `c` `$`, `Y` = `y` `y` in Vim but `y` `$` in Neovim, which is the more consistent choice.

Example 4.2.1. Combining the operators above with the motions of Chapter 3:

`d` `w`	Delete to the start of the next word
`d` `e`	Delete to the end of this word
`c` `$`	Change to end of line
`y` `}`	Yank to the end of the paragraph
`d` `G`	Delete from here to the end of the file
`>` `i` `{`	Indent everything inside the braces
`g` `U` `i` `w`	Uppercase the word under the cursor
`g` `q` `a` `p`	Rewrap the current paragraph
`!` `i` `p` `sort<CR>`	Sort the current block of lines

§4.3. Counts

A count multiplies the noun: `3` `w` moves three words, `d` `3` `w` deletes three. Counts placed before the operator multiply too, and if both are given they are multiplied together — `2` `d` `3` `w` deletes six words.

`5` `j`	Down five lines
`3` `d` `d`	Delete three lines
`2` `f` `,`	Jump to the second comma ahead
`4` `>` `>`	Indent four lines
`20` `i` `-` `<Esc>`	Insert a rule of twenty hyphens

Note 4.3.1. Counts are where `'relativenumber'` pays for itself: the line you want is labelled with exactly the number you need to type.

---

§4.4. Text Objects

Motions are relative to the cursor. **Text objects** instead name a structure the cursor happens to be *inside*, which makes them independent of where in that structure you are standing.

Definition 4.4.1 (Text object)**.** A noun of the form `i`x (**inner**: the contents) or `a`x (**a**, or **around**: the contents plus their delimiters or trailing whitespace). They are valid after an operator or in Visual mode, but not on their own in Normal mode.

`i` `w` / `a` `w`	Word / word plus trailing space
`i` `W` / `a` `W`	WORD
`i` `s` / `a` `s`	Sentence
`i` `p` / `a` `p`	Paragraph
`i` `"` / `a` `"`	Inside / including the double quotes
`i` `'`, `i` `` ` ``	Likewise for other quote characters
`i` `(` or `i` `b`	Inside the parentheses
`i` `{` or `i` `B`	Inside the braces
`i` `[`, `i` `<`	Inside brackets, angle brackets
`i` `t` / `a` `t`	Inside / including an XML or HTML tag

Example 4.4.2. The cursor is anywhere inside the string on the line `print("hello world")`.

`c` `i` `"`	Replace the string, keeping the quotes
`d` `a` `"`	Delete the string *and* the quotes
`d` `i` `(`	Empty the call's argument list
`y` `a` `(`	Copy `("hello world")`
`d` `a` `p`	Delete the whole paragraph

Note that none of these depend on the cursor being at a particular column — that independence is the whole point.

Remark 4.4.3. Neovim's `mini.ai` plugin (shipped with LazyVim, Chapter 8) extends this set considerably: `i` `f` for a function body, `i` `a` for an argument, `i` `n` `(` for the *next* parenthesis pair, and Treesitter-aware objects that understand the actual syntax tree of the language. Same grammar, better nouns.

§4.5. Repeat: the Dot Command

`.`	Repeat the last change
`u`	Undo it
`;`	Repeat the last `f`/`t` motion
`n`	Repeat the last search

`.` repeats the last *change*, not the last motion, and it is the cheapest form of automation in the editor. The characteristic Vim workflow is **one change, then dot-and-next**:

Example 4.5.1. Rename every `oldName` in a function to `newName`:

(1) Put the cursor on one and press `*` to search for it.

(2) `c` `g` `n` `newName` `<Esc>` — change the next match. (`g` `n` is a text object meaning "the next search match".)

(3) Press `.` for each remaining occurrence, and `n` to skip one.

This beats `:%s/oldName/newName/g` whenever you want to inspect each site rather than trust a pattern.

Note 4.5.2 (Making changes dot-repeatable)**.** `.` replays a single Insert-mode session. So `c` `w` `foo` `<Esc>` repeats cleanly, but if you move with the arrow keys *during* Insert mode, Vim breaks the undo block and `.` will only replay the fragment after the move. Leave Insert mode to move.

Try it

Take a list of ten lines and put a semicolon at the end of each. Do it as `A` `;` `<Esc>` `j`, then `.` `j`, `.` `j`, … Then do the same thing with `q` `q` (Section 5.4) and compare.

---

§4.6. Composition Table

The point of the grammar is that you never learn this table — you derive it. It is printed once, here, so you can see the shape of the thing: the rows are operators, the columns are nouns, and every cell is a command you already know how to build.

	`w`	`$`	`i` `(`	`a` `p`	`G`
`d` delete	`d` `w`	`d` `$`	`d` `i` `(`	`d` `a` `p`	`d` `G`
`c` change	`c` `w`	`c` `$`	`c` `i` `(`	`c` `a` `p`	`c` `G`
`y` yank	`y` `w`	`y` `$`	`y` `i` `(`	`y` `a` `p`	`y` `G`
`>` indent	`>` `w`	—	`>` `i` `(`	`>` `a` `p`	`>` `G`
`g` `U` up	`g` `U` `w`	`g` `U` `$`	`g` `U` `i` `(`	`g` `U` `a` `p`	`g` `U` `G`

Five operators and five nouns give twenty-five commands. Add `e`, `}`, `i` `"`, `f` `x` and `%` to the columns and the same five operators give fifty. Learning `i` `t` (inside an HTML tag) one afternoon silently teaches you `d` `i` `t`, `c` `i` `t`, `y` `i` `t` and `>` `i` `t` at no extra cost — and that is the whole argument for spending a week on this chapter.

If you catch yourself hunting for "the command that does *X*", stop and ask instead: *what is the verb, and what is the noun?* Vim almost certainly has both already.

5. Editing in Practice

§5.1. Entering and Using Insert Mode

There are eight ways in, and choosing the right one saves a motion every time.

`i` / `a`	Insert before / after the cursor
`I` / `A`	Insert at the first non-blank / at end of line
`o` / `O`	Open a new line below / above, and insert
`c` `{motion}`	Change: delete the region, then insert
`s` / `S`	Substitute a character / the whole line
`g` `i`	Insert exactly where you last left Insert mode
`g` `I`	Insert in column 1

Example 5.1.1. To append a semicolon, `A` `;` `<Esc>` — not `$` `a` `;` `<Esc>`. To add a line under the current one, `o`, not `$` `a` `<CR>`. These savings look trivial written down and are not, because you make the edit several hundred times a day.

§5.1.i. Useful keys inside Insert mode

`<C-w>`	Delete the word before the cursor
`<C-u>`	Delete to the start of the line
`<C-h>`	Backspace
`<C-r>` `{reg}`	Insert the contents of a register (Section 5.3)
`<C-o>`	Run *one* Normal-mode command, then come back
`<C-n>` / `<C-p>`	Complete the current word from the buffer
`<C-t>` / `<C-d>`	Indent / dedent the current line

Note 5.1.2. `<C-o>` is the escape hatch that prevents mode-thrashing: `<C-o>` `z` `z` recentres the screen mid-sentence without leaving Insert mode, and `<C-o>` `D` deletes the rest of the line.

§5.2. Undo, Redo and the Undo Tree

`u` / `<C-r>`	Undo / redo
`U`	Undo all recent changes on one line
`:earlier 10m`	Revert the file to its state ten minutes ago
`:later 30s`	And forward again
`g` `-` / `g` `+`	Move through the undo tree in time order

Remark 5.2.1 (The undo tree)**.** Vim's undo history is a *tree*, not a stack. If you undo three changes and then type something new, the old branch is not destroyed — `g` `-` walks back into it chronologically. Setting `'undofile'` persists this tree to disk so that it survives closing the file, which is one of the options most worth switching on.

§5.3. Registers and the Clipboard

Vim does not have *a* clipboard; it has several dozen.

Definition 5.3.1 (Register)**.** A named slot holding text. Prefix any yank, delete or paste with `"x` to direct it at register `x`.

`" "`	The unnamed register: the target of every unprefixed yank and delete
`" 0`	The yank register: only `y` writes here, so it survives deletions
`" 1`–`" 9`	A ring of the last nine multi-line deletions
`" a`–`" z`	Named registers you control
`" A`–`" Z`	The same, but *appending* instead of replacing
`" +`	The system clipboard
`" *`	The X11 primary selection (middle-click paste)
`" _`	The black hole: writes here are discarded
`" %`	The current file name (read-only)
`" .`	The last inserted text (read-only)
`" :`	The last Ex command (read-only)
`" /`	The last search pattern

Example 5.3.2 (The classic frustration)**.** You yank a word with `y i w`, move to another word, delete it with `d i w`, and paste — and get back the word you just deleted. The delete overwrote the unnamed register. Three fixes:

(i) paste from the yank register instead: `" 0 p`;

(ii) delete into the black hole: `" _ d i w` then `p`;

(iii) select the target in Visual mode and press `p`, which replaces the selection with the register in one step.

`p` / `P`	Paste after / before the cursor
`g p`	Paste and leave the cursor after the pasted text
`y y p`	Duplicate the current line
`:registers`	Show every register's contents

Note 5.3.3 (Linewise vs. charwise)**.** A register remembers *how* it was filled. Text yanked with `y y` is linewise and `p` puts it on a new line below; text yanked with `y i w` is charwise and `p` inserts it after the cursor. This is why `y y p` duplicates a line cleanly.

Remark 5.3.4 (System clipboard)**.** `" + y` copies to the system clipboard and `" + p` pastes from it. Setting `vim.opt.clipboard = "unnamedplus"` makes every plain yank and paste use it, which is convenient but means deletes also clobber your clipboard — many people deliberately leave it off. Neovim needs a provider for this (`xclip`, `wl-clipboard`, or `pbcopy`); check with `:checkhealth provider`.

§5.4. Macros

A macro is a register holding keystrokes rather than text — which is why they share the same namespace.

`q {reg}`	Start recording into register `reg`
`q`	Stop recording
`@ {reg}`	Play it back
`@ @`	Replay the last macro
`5 @ q`	Play macro `q` five times
`:%normal @q`	Run macro `q` on every line in the file

Example 5.4.1. Turn a list of bare words into a LaTeX itemised list. With the cursor on the first line:

```
qq            # start recording into register q
I\item <Esc>  # insert the item macro at the start of the line
j             # move to the next line
q             # stop recording
9@q           # apply to the next nine lines
```

Note 5.4.2 (Writing macros that survive)**.** Finish every macro by moving to the start of the next unit of work (usually `j` `^`), so that repetition composes. Prefer `0`, `^`, `fx` and text objects over counting characters — a macro built from structural motions keeps working on lines of different lengths. And because a macro is just a register, `"` `q` `p` pastes it into the buffer for editing, and `"` `q` `y` `y` loads the edited line back.

Reach for `.` first, a macro when `.` is not enough, and `:g` (Section 5.7) when the change is described by a pattern rather than a position.

§5.5. Visual and Blockwise Editing

`v` / `V` / `<C-v>`	Start charwise / linewise / blockwise selection
`g v`	Reselect the previous selection
`o`	Jump to the other end of the selection
`i w`, `a p`, ...	Text objects extend the selection
`u` / `U`	Lowercase / uppercase the selection
`J`	Join the selected lines
`:`	Start an Ex command with the range `'<,'>` pre-filled

Blockwise Visual mode is the one feature with no Normal-mode equivalent:

`<C-v>` ... `I text <Esc>`	Insert `text` at the start of every selected line
`<C-v>` ... `A text <Esc>`	Append it at the end of every line
`<C-v>` ... `$ A`	Append at each line's own end, ragged right
`<C-v>` ... `d`	Delete a rectangle
`<C-v>` ... `r -`	Fill the rectangle with hyphens

Try it

Comment out five lines of Lua: put the cursor on the first, press `<C-v>` `4` `j` to select a zero-width column, then `I` `--` `<Esc>`. The insertion is replicated down the block when you leave Insert mode.

§5.6. Substitution

`:s/old/new/`	Replace the first match on this line
`:s/old/new/g`	Every match on this line
`:%s/old/new/g`	Every match in the file
`:%s/old/new/gc`	… confirming each one
`:%s/old/new/gi`	… case-insensitively
`:'<,'>s/old/new/g`	Within the last Visual selection
`:10,20s/old/new/g`	Within lines 10–20
`:.,+5s/old/new/g`	From here to five lines on
`:%s//new/g`	Reuse the last search pattern as `old`

In the replacement, `&` is the whole match, `\1` –`\9` are capture groups, `~` is the previous replacement, and `\u` / `\U` uppercase what follows.

Example 5.6.1. `:%s/\v(\w+)\.(\w+)/\2.\1/g` swaps the two halves of every dotted pair in the file. Under "very magic" (`\v`) the parentheses group without escaping.

Note 5.6.2. Leaving the pattern empty reuses the last search, so the safe workflow is: search with `/` until the highlighting shows exactly the matches you intend, then run `:%s//replacement/g`. You never have to type the pattern twice, and you never substitute blind.

§5.7. The Global Command

`:g` is Vim's answer to `grep` piped into an editor: *run an Ex command on every line matching a pattern.*

`:g/pat/d`	Delete every line matching `pat`
`:v/pat/d`	Delete every line *not* matching it
`:g/pat/normal A;`	Append a semicolon to matching lines
`:g/pat/t$`	Copy matching lines to the end of the file
`:g/pat/m0`	Move them to the top — reverses their order
`:g/^$/d`	Delete all blank lines
`:g/pat/normal @q`	Run macro `q` on every matching line

Example 5.7.1. `:g/TODO/normal gcc` comments out every line containing `TODO` (assuming the `g` `c` `c` comment mapping from `mini.comment`). `:g/\vfn \w+/normal O\/\/\/` inserts a doc-comment line above every function.

`:g` plus `normal` plus a macro is the most powerful line in Vim. When a refactor is describable as "for every line that looks like X, do Y", it is one command, not a script.

6. Files, Buffers, Windows and Tabs

§6.1. Buffers, Windows and Tab Pages

Vim's window model confuses newcomers because its three nouns do not mean what they mean elsewhere.

Definition 6.1.1 (Buffer, window, tab page)**.** A **buffer** is a file loaded into memory. A **window** is a viewport onto a buffer. A **tab page** is a *layout* of windows. So one buffer may be shown in several windows, a window always shows exactly one buffer, and a tab page is not "an open file" — it is a saved arrangement of splits.

> **Intuition .** Coming from a conventional editor, the instinct is to use one tab per file. Resist it. In Vim, *buffers* are your open files and tabs are workspaces: one tab for the code, one for the tests, each with its own splits.

§6.2. Working with Buffers

`:e file`	Edit a file, loading it into a new buffer
`:ls` / `:buffers`	List the buffers and their numbers
`:b 3`	Switch to buffer 3
`:b part`	Switch by (unique) substring of the name
`:bn` / `:bp`	Next / previous buffer
`<C-^>`	Toggle with the alternate (last) buffer
`:bd`	Delete (close) the buffer
`:ball`	Open every buffer in a split

Note 6.2.1. `<C-^>` — the same key as `<C-6>` — toggles between the two files you are actually working on. If you learn one command from this chapter, this is the one.

§6.3. Splits and Window Commands

All window commands live behind the `<C-w>` prefix.

`<C-w>` `s`	Split horizontally (`:split`)
`<C-w>` `v`	Split vertically (`:vsplit`)
`<C-w>` `h` `j` `k` `l`	Move to the window in that direction
`<C-w>` `w`	Cycle to the next window
`<C-w>` `c`	Close this window
`<C-w>` `o`	Close every *other* window
`<C-w>` `q`	Quit this window
`<C-w>` `H` `J` `K` `L`	Move the window itself to the far left/bottom/top/right
`<C-w>` `=`	Equalise all window sizes
`<C-w>` `_` / `<C-w>` `|`	Maximise height / width
`<C-w>` `+` `-` `<` `>`	Resize by one row/column
`<C-w>` `T`	Move this window into a new tab page

Example 6.3.1. To compare two parts of one long file, `:vsplit` and then scroll the two windows independently — both are views of the same buffer, so edits appear in both immediately. Add `:set scrollbind` in each to lock them together.

§6.4. Tab Pages

`:tabnew`	New tab page
`:tabe file`	Open a file in a new tab
`g` `t` / `g` `T`	Next / previous tab
`3` `g` `t`	Go to tab 3
`:tabclose`	Close this tab page
`:tabonly`	Close all others

§6.5. The Quickfix and Location Lists

The quickfix list is a buffer of file:line:message entries — compiler errors, grep hits, linter output — that you can step through with the cursor landing in the right place each time.

`:make`	Run `'makeprg'` and capture the errors
`:grep pat`	Run grep and capture the hits
`:copen` / `:cclose`	Open / close the quickfix window
`:cn` / `:cp`	Jump to the next / previous entry
`:cfirst` / `:clast`	First / last entry
`:cdo s/a/b/g`	Run a command on every entry's line
`:cfdo %s/a/b/ge | update`	Run it once per *file* and save

There is a second, window-local copy of the same machinery — the **location list** — with `:lopen`, `:lnext` and so on. Language servers typically put diagnostics there.

`:grep` → `:cdo s/.../.../g` → `:cfdo update` is a project-wide refactor in three commands, with each substitution reviewable before you save.

§6.6. Files, Filters and the Shell

`:r file`	Read a file in below the cursor
`:r !date`	Read the output of a shell command
`:!ls`	Run a shell command, show the output
`:%!sort`	Filter the whole buffer through `sort`
`:%!jq .`	… or pretty-print it as JSON
`:cd %:h`	Change directory to the current file's folder
`:sav newname`	Save as, and continue editing the new file
`:Explore` / `:Ex`	Open the built-in `netrw` file browser

Note 6.6.1 (Filename modifiers)**.** Inside an Ex command, `%` is the current file. Modifiers trim it: `%:h` is its directory, `%:t` the basename, `%:r` the name without extension, `%:e` the extension, `%:p` the absolute path. So `:!pdflatex %` compiles the file you are editing, and `:e %:r.pdf` opens the result.

§6.7. Diff Mode

`nvim -d a.txt b.txt`	Open two files side by side in diff mode
`:diffthis`	Add the current window to the diff
`:diffoff!`	Leave diff mode
`]` `c` / `[` `c`	Next / previous hunk
`d` `o`	**Obtain** the hunk from the other window
`d` `p`	**Put** this hunk into the other window
`:diffupdate`	Recompute the diff

Remark 6.7.1. Setting `nvim` as your Git merge tool gives you a three-way diff in the editor you already know. Neovim's `'diffopt'` defaults include the `linematch` algorithm, which aligns changed lines within a hunk far better than Vim's.

§6.8. Sessions

`:mksession! s.vim`	Save the current layout to a file
`nvim -S s.vim`	Restore it
`:source s.vim`	Restore it from inside Neovim

Note 6.8.1. The `persistence.nvim` plugin (part of LazyVim) does this automatically per directory: `<leader>` `q` `s` restores the session for the current project, so reopening a repository puts back every split and buffer.

7. Neovim

Everything up to this point is Vim, and works identically in Neovim. This chapter is about what Neovim adds.

§7.1. What Neovim Changes

Neovim began in 2014 as a fork of Vim aimed at a refactored, more hackable codebase. The user-visible consequences are these:

Lua	A real programming language for configuration and plugins, alongside Vimscript
Built-in LSP	A Language Server Protocol client in core: completion, go-to-definition, rename, diagnostics
Treesitter	Incremental parsing, giving syntax-aware highlighting, indentation and text objects
Terminal emulator	A real shell inside a buffer
Async jobs	Plugins can run external processes without blocking the UI
Remote UI	The editor is decoupled from its front end, via a documented RPC API
Better defaults	Sensible options on by default; see `:help nvim-defaults`

Remark 7.1.1. Vim has since gained its own versions of some of these (`+terminal`, `+job`, Vim9script). The practical difference today is the ecosystem: almost all new plugin development targets Neovim's Lua API.

Note 7.1.2 (Checking your installation)**.** `:version` prints the build. `:checkhealth` is more useful — it runs every installed module's self-test and reports missing providers, absent external tools, and misconfigured plugins. Run it first whenever something behaves strangely.

§7.2. Configuration in Lua

Neovim reads `~/.config/nvim/init.lua` at startup (or `init.vim`, but not both). Any Lua file under `~/.config/nvim/lua/` is importable as a module: `lua/config/options.lua` becomes `require("config.options")`.

§7.2.i. Options

```
vim.opt.number = true             -- :set number
vim.opt.relativenumber = true     -- :set relativenumber
vim.opt.shiftwidth = 2            -- :set shiftwidth=2
vim.opt.expandtab = true
vim.opt.ignorecase = true
vim.opt.smartcase = true
vim.opt.undofile = true
vim.opt.scrolloff = 8

-- list-like options support append/remove
vim.opt.clipboard:append("unnamedplus")
vim.opt.shortmess:append("c")

-- buffer- and window-local variants
vim.opt_local.conceallevel = 0
```

Note 7.2.1. `vim.opt` wraps options in a table-like object that understands lists and sets, which is why `:append()` works. `vim.o`, `vim.bo` and `vim.wo` are the plain scalar forms for global, buffer-local and window-local options respectively.

§7.3. Mappings and Autocommands

§7.3.i. Key mappings

```
-- vim.keymap.set(mode, lhs, rhs, opts)
vim.keymap.set("n", "<leader>w", "<cmd>write<cr>", { desc = "Save file" })
vim.keymap.set("n", "<C-h>", "<C-w>h", { desc = "Go to left window" })
vim.keymap.set("v", "<", "<gv", { desc = "Dedent and keep selection" })
vim.keymap.set("t", "<Esc><Esc>", "<C-\\><C-n>", { desc = "Exit terminal mode" })

-- a Lua function as the right-hand side
vim.keymap.set("n", "<leader>d", function()
  vim.diagnostic.open_float()
end, { desc = "Line diagnostics" })

-- delete a mapping
vim.keymap.del("n", "<leader>w")
```

The `mode` argument is `"n"` Normal, `"i"` Insert, `"v"` Visual, `"x"` Visual-only, `"t"` Terminal, `"c"` Command-line, or a table of several. Useful options are `desc` (shown by `which-key`), `silent`, `buffer = 0` to scope the mapping to the current buffer, and `remap = true` on the rare occasion you want the right-hand side itself remapped.

Definition 7.3.1 (Leader)**.** `<leader>` is a user-chosen prefix key, letting you invent mappings without colliding with built-ins. It is `<Space>` in most modern configurations (LazyVim included), with `<localleader>` — used for filetype-specific mappings such as VimTeX's — set to `\`. Set them *before* loading any plugin:

```
vim.g.mapleader = " "
vim.g.maplocalleader = "\\"
```

§7.3.ii. Autocommands

An **autocommand** runs code when an event fires — opening a file, changing filetype, writing, entering Insert mode.

```lua
-- Highlight the yanked region briefly
vim.api.nvim_create_autocmd("TextYankPost", {
  callback = function()
    vim.highlight.on_yank({ timeout = 200 })
  end,
})

-- Disable conceal in LaTeX buffers, so raw math delimiters stay visible
vim.api.nvim_create_autocmd("FileType", {
  pattern = "tex",
  callback = function()
    vim.opt_local.conceallevel = 0
  end,
})

-- Groups make a config reloadable without stacking duplicates
local group = vim.api.nvim_create_augroup("MyConfig", { clear = true })
vim.api.nvim_create_autocmd("BufWritePre", {
  group = group,
  pattern = "*.lua",
  callback = function() vim.lsp.buf.format() end,
})
```

Note 7.3.2. `:autocmd` with no argument lists everything currently registered, and `:verbose autocmd BufWritePre` says which file defined each one — the fastest way to find out which plugin is reformatting your code behind your back. The same `:verbose` trick works for options (`:verbose set shiftwidth?`) and mappings (`:verbose nmap <leader>w`).

§7.4. Talking to Vim from Lua

`vim.cmd("colorscheme x")`	Run an Ex command
`vim.cmd.colorscheme("x")`	The same, in method form
`vim.fn.expand("%:p")`	Call any Vimscript function
`vim.g.foo`	Global variables (`g:foo`)
`vim.b`, `vim.w`, `vim.t`	Buffer, window, tab variables
`vim.api.nvim_*`	The full RPC API — buffers, windows, extmarks
`vim.uv`	The libuv event loop: timers, filesystem, processes
`vim.system({"ls"})`	Run a process asynchronously
`vim.notify("msg")`	Emit a notification
`vim.inspect(tbl)`	Pretty-print a table

Example 7.4.1. At the command line, `:lua` runs a line of Lua and `:=` prints an expression: `:=vim.opt.shiftwidth:get()` or `:=vim.api.nvim_buf_get_name(0)`. For anything longer, `:Lazy` has a REPL-like scratch buffer, and `:source %` re-runs the Lua file you are editing.

§7.5. The Built-in Terminal

`:terminal`	Open a shell in the current window
`:split | terminal`	… in a horizontal split
`i` / `a`	Enter Terminal mode (keys go to the shell)
`<C-\>` `<C-n>`	Back to Normal mode
`:terminal make`	Run a command and keep its output in a buffer

A terminal buffer is an ordinary buffer once you are in Normal mode, so every motion, search and yank applies to the scrollback. That alone justifies using it over a separate terminal window when you want to copy a stack trace into your code.

Note 7.5.1. LazyVim binds `<C-/>` to a floating toggleable terminal, and `<leader>` `f` `t` to one rooted at the project directory.

§7.6. Treesitter

Definition 7.6.1 (Treesitter)**.** An incremental parser generator. Neovim keeps a real *syntax tree* of each buffer, updated on every keystroke, rather than the regex-based approximation that classical Vim syntax files use.

What that buys you:

Highlighting	Correct on nested and embedded languages — Lua inside Vimscript, code blocks inside Markdown
Indentation	Derived from the tree instead of heuristics
Text objects	`a` `f` for a function, `i` `c` for a class, via `nvim-treesitter-textobjects`
Incremental selection	Grow the Visual selection node by node up the tree
Folding	`'foldexpr'` set from the tree structure
Navigation	Jump to the next function or class, and to matching structural pairs

`:TSInstall lua`	Install a parser
`:TSUpdate`	Update every installed parser
`:InspectTree`	Show the live syntax tree for this buffer
`:Inspect`	Report every highlight group under the cursor

Example 7.6.2. `:Inspect` is the tool to reach for when a colour looks wrong: it names the Treesitter capture, the syntax group and the extmark responsible for the character under the cursor, which turns "my theme is broken" into a specific highlight group you can override.

§7.7. LSP: Language Servers

Definition 7.7.1 (Language Server Protocol)**.** A JSON-RPC protocol, originally from VS Code, that lets one language-analysis server (`lua_ls`, `pyright`, `texlab`, `clangd`) serve any editor that speaks it. Neovim ships the client; you supply the servers.

The default mappings once a server attaches to a buffer:

`K`	Hover documentation
`g` `d`	Go to definition
`g` `D`	Go to declaration
`g` `r`	List references
`g` `i`	Go to implementation
`<C-o>`	Jump back afterwards
`<leader>` `c` `a`	Code action (LazyVim binding)
`<leader>` `c` `r`	Rename symbol
`<C-s>` in Insert mode	Signature help

Diagnostics are handled by a separate module, `vim.diagnostic`, so they work with linters that are not language servers:

`]` `d` / `[` `d`	Next / previous diagnostic
`:=vim.diagnostic.open_float()`	Show the full message
`:=vim.diagnostic.setloclist()`	Send them to the location list
`:LspInfo`	Which servers are attached to this buffer
`:LspLog`	The client's log, for when they are not
`:Mason`	Install and manage servers, linters, formatters

A minimal hand-rolled setup, for reference — distributions do this for you:

```
require("lspconfig").lua_ls.setup({
  on_attach = function(_, bufnr)
    local map = function(keys, fn, desc)
      vim.keymap.set("n", keys, fn, { buffer = bufnr, desc = desc })
    end
    map("gd", vim.lsp.buf.definition, "Goto definition")
    map("K", vim.lsp.buf.hover, "Hover")
    map("<leader>rn", vim.lsp.buf.rename, "Rename")
    map("<leader>ca", vim.lsp.buf.code_action, "Code action")
  end,
  settings = { Lua = { diagnostics = { globals = { "vim" } } } },
})
```

Remark 7.7.2. Neovim 0.11 added `vim.lsp.config()` and `vim.lsp.enable()`, which remove the need for `nvim-lspconfig` as a plugin, and made `g` `r` `n` (rename), `g` `r` `a` (code action) and `g` `r` `r` (references) default mappings. The machine these notes were written on runs 0.9.5, where the `lspconfig` form above still applies — check `:version` before copying configuration off the internet, because this is the API that has moved fastest.

§7.8. Completion and Snippets

Neovim's built-in completion is `<C-n>` / `<C-p>` (words in the buffer), `<C-x>` `<C-f>` (filenames) and `<C-x>` `<C-o>` (omni-completion, which the LSP client provides). Most setups layer a plugin on top — `nvim-cmp` or `blink.cmp` — that merges LSP, buffer, path and snippet sources into one popup, and `LuaSnip` for expandable templates. Chapter 8 shows this configuration as it actually stands on this machine.

§7.9. Plugins and `lazy.nvim`

Definition 7.9.1 (`lazy.nvim`)**.** The current standard plugin manager. Plugins are declared as Lua tables — "specs" — and it handles cloning, updating, lockfiles and, as the name suggests, deferring a plugin's load until it is needed.

```lua
return {
  -- 1. minimal: just clone it
  { "tpope/vim-surround" },

  -- 2. with options; lazy.nvim calls require("gitsigns").setup(opts)
  { "lewis6991/gitsigns.nvim", opts = { current_line_blame = true } },

  -- 3. lazy-loaded on a filetype, with dependencies and keymaps
  {
    "nvim-telescope/telescope.nvim",
    dependencies = { "nvim-lua/plenary.nvim" },
    cmd = "Telescope",
    keys = {
      { "<leader>ff", "<cmd>Telescope find_files<cr>", desc = "Find files" },
    },
  },

  -- 4. disable something a distribution enabled
  { "folke/trouble.nvim", enabled = false },
}
```

The loading triggers are `event` (any autocommand event, such as `VeryLazy` or `BufReadPost`), `ft` (filetype), `cmd`, and `keys`. A plugin declaring none of them loads at startup.

`:Lazy`	The plugin dashboard
`:Lazy sync`	Install, clean and update to match the spec
`:Lazy update`	Update, writing `lazy-lock.json`
`:Lazy restore`	Roll every plugin back to the lockfile
`:Lazy profile`	Per-plugin startup timings
`:Lazy log`	What changed in the last update

Note 7.9.2. `lazy-lock.json` pins every plugin to an exact commit. Commit it to version control with your config: `:Lazy restore` then reproduces a working editor exactly, which is what you want after an update breaks something at an inconvenient moment.

Plugins are the fun part and the trap. Every one is code that runs on every keystroke and one more thing to debug at 2am. Add them one at a time, and only after you have felt the absence they fill.

8. The Setup on This Machine

The previous chapter described the pieces in the abstract. This one describes the Neovim installation these notes were written in, so that the abstractions have somewhere concrete to land. At the time of writing that is Neovim 0.9.5 running **LazyVim** with VimTeX and LuaSnip added on top.

§8.1. LazyVim

Definition 8.1.1 (Distribution)**.** A pre-assembled configuration: a plugin manager, a curated plugin set, and a consistent set of keybindings, packaged so it can be installed in one step and then overridden piecemeal. `LazyVim` [fol] is the most widely used one.

The layout it expects is small:

```
~/.config/nvim/
  init.lua                 # entry point; requires config.lazy
  lazy-lock.json           # pinned plugin commits
  lua/config/lazy.lua      # bootstraps lazy.nvim and imports LazyVim
  lua/config/options.lua   # your option overrides
  lua/config/keymaps.lua   # your mappings
  lua/config/autocmds.lua  # your autocommands
  lua/plugins/*.lua        # one file per plugin or topic; merged automatically
```

Every file under `lua/plugins/` is imported and its specs are merged with LazyVim's. Declaring the same plugin again *extends* the existing spec rather than replacing it, which is how you change one option without forking the whole configuration:

```lua
-- lua/plugins/ui.lua: adjust a plugin LazyVim already installs
return {
  {
    "folke/tokyonight.nvim",
    opts = { transparent = true },
  },
}
```

Note 8.1.2 (Discovering the keymap)**.** LazyVim binds several hundred keys and you are not meant to memorise them. Press `<Space>` and stop — `which-key` pops up a menu of every continuation, grouped and labelled. `<leader>` `s` `k` searches all mappings, and `:verbose nmap <keys>` says which file defined one.

§8.2. The Keymap Worth Learning First

Verified against the LazyVim installed here; the leader is `<Space>`.

`<leader>` `<Space>`	Find files in the project root
`<leader>` `/`	Grep across the project
`<leader>` `e`	File explorer
`<leader>` `,`	Switch buffer
`<S-h>` / `<S-l>`	Previous / next buffer
`<leader>` `‘`	Toggle with the alternate buffer
`<leader>` `b` `d`	Delete this buffer, keep the window
`<C-h>` `<C-j>` `<C-k>` `<C-l>`	Move between windows (no `<C-w>` prefix)
`<leader>` `-` / `<leader>` `|`	Split below / right
`<C-/>`	Toggle a floating terminal
`<C-s>`	Save, from any mode
`<leader>` `q` `q`	Quit everything
`<leader>` `q` `s`	Restore this directory’s session
`<leader>` `l`	Open the `:Lazy` dashboard
`<leader>` `u` `i`	Inspect highlight groups under the cursor

Code-related, active whenever a language server is attached:

`K`	Hover documentation
`g` `d` / `g` `D`	Definition / declaration
`g` `r`	References
`g` `I` / `g` `y`	Implementation / type definition
`g` `K`	Signature help
`<leader>` `c` `a`	Code action
`<leader>` `c` `r`	Rename symbol
`<leader>` `c` `f`	Format buffer or selection
`<leader>` `c` `d`	Line diagnostics in a float
`]` `d` / `[` `d`	Next / previous diagnostic
`]` `e` / `[` `e`	Next / previous *error*
`<leader>` `x` `x`	Diagnostics list (Trouble)
`<leader>` `s` `r`	Project-wide search and replace (grug-far)
`g` `c` `c` / `g` `c`	Comment a line / a motion
`<A-j>` / `<A-k>`	Move the current line or selection down / up
`s`	Flash jump: type two characters, then a label

Note 8.2.1. LazyVim also remaps `j` and `k` to `g` `j` and `g` `k` when no count is given, so cursor movement follows display lines in wrapped prose but `5` `j` still means five real lines. This is exactly the behaviour you want in a LaTeX buffer.

§8.3. VimTeX

`vimtex` [Ler] turns Neovim into a LaTeX IDE: background compilation, forward and inverse search with the PDF viewer, a document outline, LaTeX-aware motions and text objects, and error parsing into the quickfix list.

The configuration in use here, from `lua/plugins/latex.lua`:

```
{
  "lervag/vimtex",
  ft = { "tex" },                          -- load only for .tex buffers
  init = function()
    vim.g.vimtex_view_method = "zathura"    -- forward search into Zathura
    vim.g.vimtex_compiler_progname = "nvr" -- inverse search back into Neovim
    vim.g.vimtex_compiler_method = "latexmk"
    vim.g.vimtex_compiler_latexmk = {
      backend = "nvim",
      build_dir = "build",                 -- keep aux files out of the repo
      callback = 1,
      continuous = 1,                      -- recompile on every write
      executable = "latexmk",
      options = {
        "-xelatex",                        -- required: this template needs XeTeX
        "-shell-escape",
        "-interaction=nonstopmode",
        "-synctex=1",                      -- enables forward/inverse search
      },
    },
  end,
}
```

Note 8.3.1. VimTeX's options must be set *before* the plugin loads, which is why they go in `init` rather than `config` or `opts` — setting `vim.g.vimtex_*` after the fact silently does nothing. This is the single most common VimTeX configuration mistake.

§8.3.i. Commands

VimTeX's prefix is `<localleader>` `l`, and since LazyVim leaves `maplocalleader` at its default, that is `\` `l`.

`\` `l` `l`	Start / stop continuous compilation
`\` `l` `v`	Forward search: jump Zathura to the cursor's line
`\` `l` `k`	Stop the compiler
`\` `l` `e`	Show compilation errors
`\` `l` `q`	Open the raw log
`\` `l` `c`	Clean auxiliary files
`\` `l` `t`	Table of contents
`\` `l` `i`	Info about the current document
`\` `l` `s`	Toggle which file counts as the main one

With `continuous = 1`, `\` `l` `l` once per session is enough: every `:w` triggers a rebuild and Zathura reloads. In the other direction, `<C-click>` in Zathura jumps Neovim to the corresponding source line — that is what `nvr` is for.

§8.3.ii. LaTeX-aware editing

These are the reason to use VimTeX even if you compile in a separate terminal:

`i` `e` / `a` `e`	Inside / around an `environment`
`i` `$` / `a` `$`	Inside / around inline math
`i` `c` / `a` `c`	Inside / around a command
`i` `d` / `a` `d`	Inside / around a delimiter pair
`i` `m` / `a` `m`	Inside / around a math group
`c` `s` `e`	Change the surrounding environment's name
`d` `s` `e`	Delete the surrounding environment, keep the body
`c` `s` `c` / `d` `s` `c`	Change / delete the surrounding command
`c` `s` `$` / `d` `s` `$`	Change / delete surrounding math
`c` `s` `d` / `d` `s` `d`	Change / delete surrounding delimiters
`t` `s` `s`	Toggle the starred form of an environment
`t` `s` `f`	Toggle between `\frac` and `/`
`t` `s` `d`	Toggle `\left\right` on a delimiter pair
`]` `]` in Insert mode	Close the innermost open environment or delimiter
`%`	Extended to match `\begin`/`\end`, `\if`/`\fi`

Example 8.3.2. The cursor is somewhere inside a `theorem` environment that should have been a `lemma`. Press `c` `s` `e`, type `lemma`, `<CR>` — both the `\begin` and the `\end` are rewritten. To delete a stray `\textbf{...}` but keep the text inside it, `d` `s` `c`. To wrap the visual selection in a new environment, `<F6>`.

§8.4. LuaSnip

`LuaSnip` [L3M] expands abbreviations into templates with tab stops. The snippets here live in `lua/snippets/tex.lua` and are loaded by

```
require("luasnip.loaders.from_lua").load({ paths = "~/.config/nvim/lua/snippets" })
```

A snippet is a table of **nodes**: literal text, numbered insert points, and the formatting that stitches them together.

```
local ls = require("luasnip")
local s, t, i = ls.snippet, ls.text_node, ls.insert_node
local fmta = require("luasnip.extras.fmt").fmta

return {
  -- type "dm" -> a display-math block, cursor at <1>
  s({ trig = "dm", name = "display Math" }, fmta([[
\[
    <content>
\]<cursor>
]], { content = i(1), cursor = i(2, "$0") })),

  -- type "rmk" -> a remark environment
  s({ trig = "rmk" }, fmta([[
\begin{<remark>}
    <content>
\end{<remark>}
]], { remark = t("remark"), content = i(1) })),
}
```

`fmta` takes a template string in which `<name>` marks a hole — angle brackets rather than the usual braces, precisely so that LaTeX's own braces need no escaping. `i(1)`, `i(2)`, ... are the tab stops, visited in order; `<Tab>` moves forward and `<S-Tab>` back.

`<Tab>`	Expand the trigger, or jump to the next tab stop
`<S-Tab>`	Jump to the previous tab stop
`<C-e>`	Cycle through a choice node
`:LuaSnipListAvailable`	List what is loaded for this filetype

Note 8.4.1 (Reloading while you write snippets)**.** `from_lua` caches. After editing the snippet file, re-run the loader

```
:lua require("luasnip.loaders.from_lua").load({
  paths = "~/.config/nvim/lua/snippets" })
```

or just restart — otherwise you will be testing the old version and concluding your snippet is broken.

Remark 8.4.2. Three separate files in this configuration call a LuaSnip loader:

- `init.lua` — absolute path, works;
- `lua/plugins/luasnip-snippets.lua` — absolute path, works;
- `lua/plugins/latex.lua` — *relative* path, `./lua/snippets`.

The last one resolves against the current working directory rather than the config directory, so it finds nothing unless Neovim was started from `~/.config/nvim`. Harmless, since the other two use the absolute path, but worth collapsing into a single loader call the next time you touch the file.

§8.5. The LaTeX Workflow, End to End

(1) `nvim Vim.tex` — VimTeX loads on the `tex` filetype.

(2) `\` `l` `l` starts `latexmk` in continuous mode; the PDF appears in Zathura.

(3) Write. Every `:w` (or `<C-s>`) recompiles in the background; errors land in the quickfix list, reachable with `]` `q`.

(4) `\` `l` `v` jumps the viewer to the cursor; `<C-click>` in the viewer jumps the cursor back.

(5) `\` `l` `t` opens the table of contents to navigate between sections.

(6) `\` `l` `c` cleans up before committing.

Note 8.5.1. This template requires XeTeX or LuaTeX — it loads `fontspec` and `xeCJK` for the CJK fonts. That is why `vimtex_compiler_latexmk` passes `-xelatex`; with the default `pdflatex` the header aborts immediately with an explicit error.

A configuration is a tool you maintain, not a possession you collect. Read `:checkhealth` when something breaks, `:verbose` when something is mysterious, and delete anything you have not used in a month.

Part II.

The Unix Shell

The shell is a programming language you have already been typing into. Every command you run is a line of it, and the things that look like syntax errors — unexpected globbing, a variable that lost its spaces, a pipeline that silently swallowed an error — are what happens when you use a language without having learnt it.

The organising idea is composition through text. Each program does one narrow thing and speaks the same universal interface: lines of characters in, lines of characters out. Chapter 9 covers the language itself — how to read a transcript, and what the shell does to your line before any program ever sees it. Chapter 10 and Chapter 11 are the tools; Chapter 12 is the plumbing that makes them a language rather than a list; and by Chapter 14 the same constructs have become scripts you can commit.

9. The Shell and the Command Line

You have been programming in the shell since the first time you typed `ls`. Most people never find this out, and it costs them: they treat the command line as a place where incantations either work or mysteriously do not, rather than as a small language with rules that can be learnt in an afternoon.

This chapter is that language's syntax. Nothing here is a tool for getting work done — chapters 10 onwards are — but almost every surprising thing the shell will ever do to you is explained by Section 9.2 and Section 9.5.

§9.1. Reading Shell Transcripts

Part II shows a great deal of terminal output, and the boxes it appears in follow one convention throughout.

```
$ echo hello          # a line beginning with "$ " is what YOU type
hello                 # a line with no prompt is what the computer printed
$ whoami
konyi
```

`$`	The prompt of an ordinary user. Your real prompt probably shows a path and a Git branch; it is shortened here to keep the examples readable
`#`	The prompt of the `root` user. A transcript using it is doing something that requires administrative rights
`>`	A continuation prompt: the shell is waiting because the previous line was incomplete
`# ...`	After a command, a comment. The shell ignores it, and so may you

Note 9.1.1. Never type the `$`. It is the prompt, not part of the command. This is the most common mistake made when copying examples out of documentation, and the error you get — `command not found: $` — is at least a clear one.

Command syntax is described the way manual pages describe it, and that notation is worth ten minutes on its own:

`FILE`, `<file>`	A placeholder you must replace with something real
`[OPTION]`	Square brackets: optional
`a|b`	A vertical bar: choose one
`FILE...`	An ellipsis: may be repeated
`[FILE]...`	Optional, and repeatable — zero or more files

Example 9.1.2. The synopsis line `ls [OPTION]... [FILE]...` says: the command is `ls`; it takes any number of options, all optional; then any number of filenames, also optional. So `ls`, `ls -l`, `ls -l src/` and `ls -la a.txt b.txt` are all valid, and the manual told you so in eighteen characters.

> **Notation .** Inline, this note writes commands as `grep -n TODO`, file and directory paths as `/usr/local/bin`, and environment variables as `PATH`. Keystrokes keep the keycap convention of Chapter 1, so `<C-c>` means Control held with `c`. Longer transcripts appear in the framed boxes shown above.

§9.2. What a Shell Is

Definition 9.2.1 (Shell)**.** A program that reads a line of text, transforms it according to a fixed set of rules, and then runs whatever program the result names. It is a read–evaluate–print loop for the operating system.

The word *transforms* is doing all the work in that definition, and it is the single idea this part is built on.

> **Intuition .** The shell does not pass your command to a program. It **rewrites** your command, and passes the *result* to a program. When you type `ls *.txt`, the program `ls` never sees an asterisk — the shell has already replaced it with a list of filenames. Every shell surprise you will ever have is an expansion that happened when you did not expect it, or failed to happen when you did.

Example 9.2.2. `ls *.txt` in a directory holding `a.txt` and `b.txt` runs the program `ls` with two arguments, `a.txt` and `b.txt`. In an empty directory, Bash leaves the asterisk alone and `ls` is asked for a file literally named `*.txt`; Zsh instead refuses and reports `no matches found`. Same command, three outcomes, and none of them involve `ls` understanding wildcards.

The loop runs in four stages, and keeping them separate in your head is most of the battle:

Read	Take one line of input
Expand	Apply the expansions of Section 9.4, in a fixed order
Execute	Split the result into words; the first is the command, the rest are its arguments
Wait	Run it, collect its exit status, print the prompt again

Interactive, login, and script shells

The same binary behaves differently depending on how it was started, which is why a command can work when you type it and fail in a script.

Interactive	Reading from a terminal, printing a prompt. Loads `~/.bashrc` or `~/.zshrc`
Login	The first shell of a session. Loads `~/.profile`, `~/.bash_profile` or `~/.zprofile`
Non-interactive	Running a script. Loads almost nothing — no aliases, often a different `PATH`

Note 9.2.3. This is why an alias defined in `~/.bashrc` is invisible inside a script, and why a script that works in your terminal can fail from `cron`. Section 13.5 covers which file to put what in.

§9.3. Bash, Zsh, and Which One You Are Running

"The shell" is a family, not a program:

`sh`	The POSIX standard. On most Linux systems `/bin/sh` is really `dash`, a small fast shell with no extras
`bash`	The GNU shell; the default on most Linux distributions and the assumed target of most scripts online
`zsh`	Bash-compatible for most purposes, with much better interactive features. The default on macOS, and the login shell on this machine
`fish`	Friendlier still, but deliberately *not* POSIX-compatible: scripts written for it do not run elsewhere

```
$ echo $SHELL          # your login shell, from /etc/passwd
/bin/zsh
$ ps -p $$ -o comm=    # the shell you are actually in right now
zsh
$ zsh --version
zsh 5.9 (x86_64-ubuntu-linux-gnu)
```

Note 9.3.1. `$SHELL` and "the shell I am in" are different questions. `$SHELL` is a setting; `$$` is the current process. If you run `bash` from inside Zsh, `$SHELL` still says `zsh` and it is lying to you.

Remark 9.3.2 (Differences that will bite you)**.** Zsh is close enough to Bash that most of this part applies unchanged, but three divergences matter:

(i) Unquoted `$var` is *not* word-split in Zsh, so a script that accidentally relies on splitting works in Bash and breaks in Zsh — or, more often, the reverse.

(ii) A glob that matches nothing is an error in Zsh, but is passed through literally in Bash.

(iii) Arrays are indexed from 1 in Zsh and from 0 in Bash.

The fix is not to choose a side. It is to put `#!/bin/bash` at the top of every script (Section 14.1) so that what runs it is never in doubt.

§9.4. Anatomy of a Command

A command line is a list of **words** separated by whitespace. The first word names what to run; the rest are handed to it as arguments.

```
{command}  [{options}]  [{arguments}]
```

`-l`	A short option: one dash, one letter
`-la`	Bundled short options — identical to `-l -a`
`-all`	A long option: two dashes, a whole word
`-color=auto`	A long option with a value. `-color auto` usually works too
`-n 5`	A short option with a value, as a separate word
`--`	"No more options": everything after this is an argument, even if it starts with a dash

Example 9.4.1. To delete a file that is genuinely named `-rf`, `rm -rf` is a catastrophe and `rm -- -rf` is the fix. The same trick rescues `grep -- -v file.txt` when you are searching for a literal `-v`.

The order of expansions

Between reading and executing, the shell applies these transformations, in this order. The order is not trivia: it is why `echo {1..3}.txt` works on files that do not exist, and why an unquoted variable holding a `*` will glob.

1. Brace expansion	`{a,b}` → `a b` (Section 9.6)
2. Tilde expansion	`~` → your home directory
3. Parameter expansion	`$HOME` → its value
4. Command substitution	`$(date)` → that command's output
5. Arithmetic expansion	`$((2+2))` → `4`
6. Word splitting	The result is cut at spaces — *only* for unquoted expansions
7. Pathname expansion	`*` and friends become filenames
8. Quote removal	The quotes themselves are stripped before the program sees anything

Example 9.4.2. With `f="my file.txt"`, the line `rm $f` expands to `$f` at step 3, is split into *two* words at step 6, and deletes `my` and `file.txt`. `rm "$f"` skips step 6 entirely and deletes the one file you meant. This is the most consequential pair of quotation marks in this part.

§9.5. Quoting and Escaping

Quoting exists to switch expansions off. There are three mechanisms and they differ only in how much they disable.

`'single'`	Disables *everything*. Every character is literal; even a backslash. The only thing you cannot put inside is another single quote
`"double"`	Disables globbing and word splitting, but keeps `$`, `` ` ``, `\` and `!` working
`\x`	Disables whatever the next single character would have done
`$'...'`	ANSI-C quoting: like single quotes, but `\n`, `\t` become real newlines and tabs

```
$ name="Kon Yi"
$ echo $name        # unquoted: two words, but echo joins them so this looks fine
Kon Yi
$ touch $name       # unquoted: creates TWO files
$ ls
Kon  Yi
$ touch "$name"     # quoted: creates one
$ ls
Kon  Kon Yi  Yi
```

Example 9.5.1 (Single versus double)**.**

```
$ echo "today is $(date +%A)"
today is Monday
$ echo 'today is $(date +%A)'
today is $(date +%A)
```

Use single quotes whenever you mean the characters themselves — a regular expression, an `awk` program, a password — and double quotes whenever you want a variable expanded but nothing else.

Quote every variable expansion — `"$var"`, `"$@"`, `"$(cmd)"` — unless you have a specific reason not to. The unquoted form is not shorter; it is a different operation that happens to look the same most of the time.

§9.6. Globbing and Brace Expansion

Definition 9.6.1 (Glob)**.** A pattern that the shell replaces with a list of matching *existing* filenames. Globs are not regular expressions (Section 11.5) and the two use overlapping symbols to mean different things.

`*`	Any run of characters, including none — but never a leading dot
`?`	Exactly one character
`[abc]`	One character from the set
`[a-z]`	One character from the range
`[!abc]`	One character *not* in the set (`[^abc]` also works)
`**`	Any number of directory levels, where enabled

Note 9.6.2. `*` does not match a leading dot, so `rm *` leaves `.gitignore` alone and `ls *` does not list `.bashrc`. This is a deliberate safety feature and a frequent source of confusion. `**` needs `shopt -s globstar` in Bash; it is on by default in Zsh.

Brace expansion looks similar and is completely different: it is pure text generation, it happens first, and it does not care whether the files exist.

```
$ echo {a,b,c}.txt
a.txt b.txt c.txt
$ echo file{1..5}
file1 file2 file3 file4 file5
$ echo {01..10..3}
01 04 07 10
$ mkdir -p project/{src,test,docs}
$ cp config.yml{,.bak}        # expands to: cp config.yml config.yml.bak
```

Example 9.6.3. That last line is worth memorising. `config.yml{,.bak}` expands to two words — the original and the original with a suffix — so the file is backed up without typing its name twice. It generalises: `mv report.tex{,.old}`.

Globs are expanded by the *shell*, before the program runs. So `find . -name *.c` is a bug: the shell substitutes the `.c` files in your current directory and `find` receives something it was never meant to see. Write `find . -name '*.c'` and let `find` do its own matching.

§9.7. Line Editing and History

The shell's input line is itself a tiny editor, and it is the same editor everywhere: GNU Readline, in Emacs mode by default.

`<C-a>` / `<C-e>`	Start / end of line
`<C-w>`	Delete the word before the cursor
`<C-u>` / `<C-k>`	Delete to start / end of line
`<C-y>`	Paste back what you just deleted
`<C-r>`	Search backwards through history incrementally
`<C-l>`	Clear the screen
`<C-c>`	Abandon this line
`<C-d>`	End of input — on an empty line, log out
`<M-.>`	Insert the last argument of the previous command

Note 9.7.1. `<C-w>`, `<C-u>` and `<C-r>` also work at any password prompt and inside Vim's Insert mode (Section 5.1). They are Readline bindings, not shell commands, which is why they turn up everywhere.

History is addressable:

`history`	Print the history, numbered
`!!`	The previous command — `sudo !!` is the classic
`!$`	The last argument of the previous command
`!42`	Command number 42
`!grep`	The most recent command starting with `grep`
`^old^new`	Rerun the last command with the first `old` replaced

Try it

Run `mkdir -p /tmp/demo/a/b`, then `cd !$` — you are now in the directory you just made, without retyping the path. Then press `<C-r>` and type `mkdir`: the search finds the earlier command, and `<CR>` runs it again.

Remark 9.7.2 (Vi mode)**.** `set -o vi` switches Readline to modal editing, giving you `<Esc>` and the motions of Chapter 3 on the command line; `set -o emacs` switches back. Worth trying once Part I has become reflex — though the Emacs bindings above are so widespread that most people keep both.

§9.8. Getting Help

`man ls`	The manual page for `ls`
`man 5 passwd`	Section 5's `passwd` — the *file format*, not the command
`man -k compress`	Search every manual page description; also spelled `apropos`
`ls -help`	The short version, for GNU tools
`help cd`	For shell *builtins*, which have no man page of their own
`type ls`	What is this name: builtin, alias, function, or file?
`command -v rg`	Where would this run from? Script-friendly
`info coreutils`	The full GNU manuals, where man pages are only a summary

Manual pages are numbered by section, and the number matters:

1	User commands — `ls`, `grep`
2	System calls — `open`, `fork`
3	Library functions — `printf`, `malloc`
5	File formats — `/etc/passwd`, `crontab`
7	Conventions and overviews — `man 7 glob`, `man 7 signal`
8	Administration commands — `mount`, `systemctl`

Example 9.8.1. `man 7 glob` documents the globbing rules of Section 9.6 in one page, and `man 7 signal` lists every signal name for Section 13.3. The section-7 pages are the most under-read documentation on a Unix system.

Note 9.8.2. `type` is the one to reach for when a command behaves oddly. If `type ls` reports `ls is an alias for ls -color=auto`, that explains why the output looks different inside a pipe — and `\ls` or `command ls` bypasses the alias.

Man pages are terse rather than difficult. Read the synopsis, skim description, then jump straight to examples at the bottom — most GNU pages have them, and they answer the actual question about four times out of five.

10. The Filesystem

Unix presents everything as a file in one tree: your documents, your keyboard, your disks, the running kernel's own state. That claim is repeated so often it has stopped meaning anything, so here is the version that matters in practice: there is exactly one namespace, it starts at `/`, and the commands in this chapter are how you move through it.

§10.1. Paths

Definition 10.1.1 (Absolute and relative paths)**.** An **absolute** path begins at the root of the tree and so starts with `/`. A **relative** path begins wherever you currently are, and does not.

`/`	The root of the whole tree
`.`	The current directory
`..`	The parent directory
`~`	Your home directory; `~konyi` is someone else's
`-`	As an argument to `cd`, the previous directory

Example 10.1.2. From `/home/konyi/Documents`, the file `/home/konyi/Documents/LaTeX/CSTools.tex` can be named `LaTeX/CSTools.tex`, `./LaTeX/CSTools.tex`, or `~/Documents/LaTeX/CSTools.tex`. All three are the same file; which you use is a question of what you want to stay true when you move.

The parts of a path have names, because commands use them:

```
$ basename /home/konyi/notes.tex      # the last component
notes.tex
$ dirname /home/konyi/notes.tex       # everything before it
/home/konyi
$ realpath ../Documents               # resolve to an absolute path
/home/konyi/Documents
```

Note 10.1.3. A leading dot makes a file **hidden**: it is not listed by `ls` and not matched by `*`. This is not a permission or a security feature — it is a naming convention that `ls` honours, which is why your home directory looks empty until you run `ls -a`.

§10.2. Navigating

`pwd`	Print the working directory
`cd dir`	Change to `dir`
`cd`	With no argument, go home
`cd -`	Back to where you just were
`cd ..`	Up one level
`pushd dir` / `popd`	Change directory onto a stack, and come back
`dirs -v`	Show the directory stack

Note 10.2.1. `cd` must be a shell *builtin*, not a program. A child process cannot change its parent's working directory, so if `cd` were an ordinary command it would change its own directory and then exit, accomplishing nothing. `type cd` confirms it. The same reasoning explains `export` and `source`.

Try it

Run `cd /tmp`, then `cd ~`, then `cd -` a few times. The last one toggles, which makes it the `<C-^>` of the shell (Section 6.2) — the fastest way to bounce between two directories you are working in.

§10.3. Listing and Inspecting

`ls`	List the current directory
`ls -l`	Long form: permissions, owner, size, date
`ls -a`	Include hidden files
`ls -lh`	Human-readable sizes — `4.0K` rather than `4096`
`ls -lt`	Newest first; add `-r` to reverse
`ls -ld dir`	The directory itself, not its contents
`tree -L 2`	The tree, two levels deep
`stat file`	Everything the filesystem knows about one file
`file x`	Guess what a file actually *is*, ignoring its name
`du -sh dir`	How big is this directory, in total
`df -h`	How full are the mounted filesystems

Example 10.3.1 (Reading `ls -l`).

```
$ ls -l notes.tex
-rw-rw-r-- 1 konyi konyi 4096 Sep 14 17:23 notes.tex
```

Left to right: the type and permissions (Section 10.6); the link count; the owning user; the owning group; the size in bytes; the modification time; the name. The very first character is the type — `-` for a regular file, `d` for a directory, `l` for a symbolic link.

Note 10.3.2. `file` is the honest answer to "what is this". Extensions are a convention that nothing enforces, so `file mystery.txt` reporting `PNG image data` is not a contradiction — it is the point.

§10.4. Creating and Destroying

`touch f`	Create `f` if absent; update its timestamp if present
`mkdir d`	Make a directory
`mkdir -p a/b/c`	Make the whole chain, and do not complain if it exists
`cp a b`	Copy
`cp -r src dst`	Copy a directory and everything in it
`cp -i a b`	Ask before overwriting
`mv a b`	Move — which is also how you rename
`rm f`	Delete a file
`rm -r d`	Delete a directory and its contents
`rmdir d`	Delete a directory, but only if it is empty

Note 10.4.1 (There is no undo)**.** `rm` does not move anything to a wastebasket. The file is gone, and the only recovery is your backups — or, for a file you are tracking, Part V. Three habits make this survivable:

(i) Prefer `rm -i` while you are learning, or alias it so.

(ii) Run `ls` on a glob before you run `rm` on it. The shell expands both the same way, so what `ls` printed is exactly what `rm` will delete.

(iii) Never put a space in `rm -rf ./*`. `rm -rf . /*` is a different and much worse command.

`mv` is rename, copy-then-delete, and move-between-directories all at once, because on Unix a filename is just an entry in a directory pointing at the data. Renaming rewrites the entry and never touches the file.

§10.5. Hard and Symbolic Links

That last observation has a consequence: a file's data can have more than one name.

Definition 10.5.1 (Hard and symbolic links)**.** A **hard link** is an additional directory entry pointing at the same data. The file is deleted only when the last one goes. A **symbolic link** (or **symlink**) is a small file whose contents are a *path* — a signpost, which can point at something that does not exist.

`ln target name`	Create a hard link
`ln -s target name`	Create a symbolic link — the common case
`ls -l`	Symlinks show as `name -> target`
`readlink -f name`	Follow it to the real file
`unlink name`	Remove the link, never the target

Example 10.5.2. Dotfile management is the usual reason to care. Keep the real files in a Git repository and symlink them into place:

```
$ ln -s ~/dotfiles/nvim ~/.config/nvim
$ ls -l ~/.config/nvim
lrwxrwxrwx 1 konyi konyi 22 Jul 30 ~/.config/nvim -> /home/konyi/dotfiles/nvim
```

Now the configuration is versioned, but Neovim still finds it exactly where it expects.

Note 10.5.3. Use an *absolute* target for a symlink unless you have thought carefully. A relative target is resolved from the link's own directory, not from where you were standing when you created it, which is the cause of most broken symlinks.

§10.6. Permissions and Ownership

Every file carries an owner, a group, and nine permission bits: read, write and execute, for the user, the group and everyone else.

```
-    rw-    rw-    r--
```

type user group other

`r`	Read: see the contents. On a directory, list its entries
`w`	Write: change the contents. On a directory, create or delete entries
`x`	Execute: run it. On a directory, *enter* it or pass through it

Note 10.6.1. The directory cases are the ones that surprise people. `x` without `r` on a directory lets you open a file inside it if you already know the name, but not list what is there. And deleting a file needs `w` on the *directory*, not on the file — which is why you can delete something you have no permission to read.

`chmod u+x script.sh`	Give the owner execute permission
`chmod go-w file`	Take write away from group and others
`chmod 644 file`	Octal: `rw-r--r--`
`chmod 755 dir`	Octal: `rwxr-xr-x`
`chmod -R u+rwX dir`	Recursive; capital `X` means "`x` only where it makes sense"
`chown user:group file`	Change ownership — usually needs `sudo`
`umask`	Which permissions are *removed* from new files

The octal form is three digits, one per class, each the sum of `r` = 4, `w` = 2, `x` = 1:

600	`rw-------` — private. SSH keys must be this
644	`rw-r--r--` — the default for a file
755	`rwxr-xr-x` — the default for a directory or a program
777	Everything for everyone. Essentially always the wrong answer

When a script will not run, the question is almost never the code. `ls -l` it: if there is no `x`, `chmod +x` it; if there is, check the shebang (Section 14.1).

§10.7. Finding Files

`find` walks a directory tree and applies tests to everything it meets. Its syntax is unlike every other Unix command — long options with one dash, arguments that read as a sentence — and it is worth the adjustment.

```
find {where} {tests} [{action}]
```

`find . -name '*.tex'`	By name, case-sensitively. Quote the pattern!
`find . -iname '*.PDF'`	Case-insensitively
`find . -type f`	Regular files only (`d` for directories)
`find . -mtime -7`	Modified in the last seven days
`find . -size +10M`	Larger than ten megabytes
`find . -empty`	Empty files and directories
`find . -maxdepth 2`	Do not descend further than two levels
`find . -path '*/build/*' -prune -o -print`	Skip a subtree entirely

Tests combine, and the default between them is *and*:

```
$ find . -type f -name '*.log' -size +1M          # big log files
$ find . -type f -newer Makefile                  # changed since Makefile did
$ find . -name '*.aux' -delete                    # find and remove
$ find . -name '*.tex' -exec wc -l {} +           # run a command on the results
```

Note 10.7.1. In `-exec`, `{}` stands for each file found. Ending with `+` batches as many files as will fit into one invocation; ending with `\;` runs the command once per file. Prefer `+`: it is dramatically faster on large trees, for the same reason `xargs` exists (Section 12.4).

Always quote `find`'s patterns. `find . -name *.c` lets the shell expand the glob first (Section 9.6), and `find` then receives filenames where it expected a pattern — producing either the wrong answer or `paths must precede expression`, which is `find`'s way of saying exactly this.

Remark 10.7.2. `fd` is a modern alternative with saner defaults: `fd '\.tex$'` does what most `find` invocations are reaching for, respects `.gitignore`, and is faster. It is not installed here, and `find` is on every machine you will ever log into, so learn `find` first and treat `fd` as a luxury.

11. Viewing and Processing Text

Unix's founding bet was that if every program reads and writes plain text, then a small number of general tools can be combined to solve problems nobody anticipated. The tools in this chapter are that bet's payoff. Each does one narrow thing; Chapter 12 is where they become a language.

§11.1. Reading Files

`cat file`	Print the whole file
`cat -n file`	... with line numbers
`cat a b > c`	Concatenate — which is what the name is short for
`head file`	The first ten lines
`head -n 3 file`	The first three
`tail file`	The last ten lines
`tail -n +5 file`	From line five to the end
`tail -f log`	Print new lines as they are appended, forever
`less file`	Page through it interactively

`less` is a pager, and its keys will look familiar:

`<Space>` / `b`	Forward / back one screen
`j` `k`	Down / up one line
`g` / `G`	Start / end of file
`/pat`, `n`, `N`	Search, next match, previous match
`-N`	Toggle line numbers
`F`	Follow, like `tail -f`; `<C-c>` stops
`q`	Quit

Note 11.1.1. These are Vim's keys, not a coincidence: `less`, `man`, `git log` and most things that paginate use the same bindings. Part I has already taught you how to read a log file.

`cat file | grep x` works but wastes a process; `grep x file` is the same thing. This gets called "useless use of `cat`" often enough to be a running joke. It is harmless — but `cat` on a 2 GB log is not, and `less` is what you wanted.

§11.2. Slicing Lines

`wc -l file`	Count lines; `-w` words, `-c` bytes
`cut -d: -f1 /etc/passwd`	Field 1, with `:` as the delimiter
`cut -c1-10 file`	Characters 1 to 10 of every line
`paste a b`	Join two files side by side, line for line
`tr 'a-z' 'A-Z'`	Translate characters
`tr -d '\r'`	Delete characters — this one strips Windows line endings
`tr -s ' '`	Squeeze repeats into one
`column -t`	Align whitespace-separated input into columns
`nl file`	Number the lines
`rev`	Reverse each line's characters

Note 11.2.1. `tr` reads only from standard input — it takes no filename. This is deliberate: it is a **filter**, meant to sit in the middle of a pipeline (Section 12.3), and `tr -d '\r' < old > new` is how you feed it a file.

Example 11.2.2. The users on this system, one per line:

```
$ cut -d: -f1 /etc/passwd | head -3
root
daemon
bin
```

`cut` is the right tool exactly when fields are separated by a single character. When they are separated by *runs* of whitespace, it is the wrong tool and `awk` is the right one.

§11.3. Ordering

`sort file`	Sort lines alphabetically
`sort -n`	Numerically — `10` after `9`, not before
`sort -h`	By human-readable size: `2K`, `1M`, `3G`
`sort -r`	Reverse
`sort -u`	Sort and remove duplicates
`sort -k2`	By the second field
`sort -t: -k3 -n`	By field 3, colon-separated, numerically
`uniq`	Collapse *adjacent* duplicate lines
`uniq -c`	... and prefix each with its count
`uniq -d`	Only the lines that were duplicated
`shuf`	Shuffle; `shuf -n 1` picks one at random

Note 11.3.1. `uniq` only removes *adjacent* duplicates, so it is nearly always preceded by `sort`. This looks like a design flaw and is not: it makes `uniq` a streaming filter that needs no memory, and the sort you would have needed anyway is written explicitly.

Example 11.3.2 (The counting idiom)**.** This pipeline appears everywhere, and is worth recognising on sight:

```
$ sort access.log | uniq -c | sort -rn | head
   842 /index.html
   311 /about
    97 /contact
```

Sort so duplicates become adjacent; count them; sort those counts numerically, largest first; take the top few. *What are the most frequent lines?* — in four words.

§11.4. Comparing

`diff a b`	Line-by-line differences
`diff -u a b`	Unified format — what Git and `patch` both speak
`diff -r d1 d2`	Recursively, over two directories
`cmp a b`	Byte-for-byte; silent when identical
`comm -12 a b`	Lines common to two *sorted* files
`patch < fix.diff`	Apply a diff
`vimdiff a b`	The same comparison, editable (Section 6.7)

Example 11.4.1 (Reading a unified diff)**.**

```
$ diff -u old.txt new.txt
--- old.txt
+++ new.txt
@@ -1,3 +1,3 @@
 unchanged context line
-a line that was removed
+a line that was added
 unchanged context line
```

`---` is the old file and `+++` the new; `@@` gives the line ranges of the **hunk**; then `-` for removed, `+` for added, and a leading space for context. This is exactly what `git diff` prints, so Section 31.5 needs no further explanation.

§11.5. Regular Expressions

Definition 11.5.1 (Regular expression)**.** A pattern language for matching text. Unlike a glob (Section 9.6), which is expanded by the shell against filenames, a regular expression is interpreted by the *program* you hand it to, and matches anywhere inside a line.

The unhappy part is that there are three dialects in common use:

BRE	Basic. `grep` and `sed` by default. `+`, `?`, `|`, `(` must be backslash-escaped to have their special meaning
ERE	Extended. `grep -E`, `sed -E`, `awk`. Those characters work unescaped
PCRE	Perl-compatible. `grep -P`, `rg`, and most programming languages. Adds `\d`, `\w`, lookaround, non-greedy `*?`

Use `grep -E` and `sed -E` habitually. The escaping rules of BRE exist for compatibility with 1975 and cost you a backslash every few characters for no benefit.

The atoms are common to all three:

`.`	Any single character
`*`	Zero or more of the preceding item
`+`	One or more
`?`	Zero or one
`{2,5}`	Between two and five
`[abc]`, `[a-z]`	One character from a set or range
`[^abc]`	One character not in the set
`^`	Start of line
`$`	End of line
`(ab|cd)`	Group, and alternation
`\.`	A literal dot
`\b`	A word boundary

Note 11.5.2 (The same symbol, two meanings)**.** In a glob, `*` means "any run of characters". In a regular expression it means "any number of *the thing before me*". So the glob `*.txt` is the regular expression `.*\.txt$` — and `*.txt` as a regular expression means something else entirely. Mixing these up is the single most common regular-expression mistake.

Remark 11.5.3. Regular expressions are greedy: they match as much as they can. `<.*>` against `<a> and <b>` matches the whole string, not `<a>`. In PCRE, `<.*?>` is the non-greedy form; in ERE the usual fix is to exclude the delimiter, `<[^>]*>`.

§11.6. `grep`

`grep` prints the lines of its input that match a pattern. That is all it does, and it is probably the command you will run most often for the rest of your career.

`grep pat file`	Lines containing `pat`
`grep -i`	Ignore case
`grep -v`	Invert: lines that do *not* match
`grep -n`	Prefix each with its line number
`grep -r pat dir`	Recurse into a directory
`grep -l`	Print only the names of matching files
`grep -c`	Print only a count
`grep -w`	Match whole words only
`grep -F`	Fixed strings: no pattern language at all
`grep -E`	Extended regular expressions
`grep -A3 -B1`	Show 3 lines after and 1 before each match
`grep -o`	Print only the matching part, not the whole line

Example 11.6.1.

```
$ grep -rn 'TODO' --include='*.tex' .        # every TODO in the note, with line numbers
$ grep -c '^$' report.tex                    # how many blank lines?
$ grep -o '[a-z]*@[a-z.]*' contacts.txt      # pull out just the addresses
$ ps aux | grep -v grep | grep nvim          # find a process, minus grep itself
```

Note 11.6.2. `grep` sets its exit status to 0 if it matched and 1 if it did not, which makes it usable as a test: `if grep -q pat file; then ...` with `-q` for silence. Section 12.7 builds on this.

Remark 11.6.3. `rg` (ripgrep, installed here) is `grep -r` done properly: it is several times faster, recurses by default, skips anything in `.gitignore`, and uses PCRE-ish syntax throughout. For searching a source tree, prefer it. For a pipeline or a script that must run anywhere, use `grep` — it is on every Unix machine in existence.

§11.7. sed

`sed` is a **stream editor**: it reads a line, applies your commands, prints the result, and repeats. You will use perhaps three of its features ever, and one of them constantly.

```
sed [-E] [-n] [-i] '{address}{command}' {file}
```

`sed 's/old/new/'`	Replace the first match on each line
`sed 's/old/new/g'`	Replace every match
`sed 's/old/new/2'`	Replace only the second match on each line
`sed -E 's/(a)(b)/\2\1/'`	Swap two capture groups
`sed -n '5p'`	Print only line 5 (`-n` suppresses the default printing)
`sed -n '10,20p'`	Print lines 10 to 20
`sed '/pat/d'`	Delete matching lines
`sed '1d'`	Delete the first line — a header, usually
`sed -i.bak 's/a/b/g'`	Edit the file in place, keeping a backup

Note 11.7.1. If this looks familiar, it should: Vim's `:%s/old/new/g` (Section 5.6) and `:g/pat/d` (Section 5.7) are the same commands with the same syntax. Both inherited them from `ed`, and learning either teaches you the other.

Note 11.7.2 (`-i` is not portable)**.** GNU `sed` accepts `sed -i 's/a/b/'`; BSD and macOS `sed` require an argument, `sed -i '' 's/a/b/'`. Writing `sed -i.bak` works on both and leaves you a backup, which is the right answer for a command that edits files irreversibly.

Example 11.7.3. The delimiter need not be a slash — any character works, which spares you from escaping paths:

```
$ sed 's|/usr/local|/opt|g' config.ini
```

§11.8. awk

`awk` is a complete programming language, but you can get most of its value from one sentence: *for each line, split it into fields, and if the line matches a condition, do something.*

```
awk '{pattern} { {action} }' {file}
```

Either part may be omitted: with no pattern the action runs on every line; with no action the line is printed.

`$0`	The whole line
`$1`, `$2`	The first and second fields
`NF`	The number of fields — so `$NF` is the last
`NR`	The current line number
`-F:`	Use `:` as the field separator
`BEGIN{}`	Run before the first line
`END{}`	Run after the last

Example 11.8.1.

```
$ awk '{print $1}' access.log            # first field of every line
$ awk -F: '$3 >= 1000 {print $1}' /etc/passwd    # human users only
$ awk 'NF' file.txt                      # drop blank lines: NF is 0, which is false
$ awk '{s += $2} END {print s}' data     # sum the second column
$ awk 'length > 79' report.tex           # lines over 79 characters
```

Intuition . Choose between the three by what the input looks like. If you want *lines that match*, use `grep`. If you want *the same substitution everywhere*, use `sed`. If the input has *columns* — and you want to compare, compute with, or reorder them — use `awk`. Most one-liners that get uncomfortable in `sed` are two clear words of `awk`.

When an `awk` program grows past one line, stop and write it in a real language. The tools in this chapter are for the middle of a pipeline; they are not a place to build software.

12. Streams, Pipes and Redirection

Chapter 11 gave you a box of tools that each do one narrow thing. This chapter is the plumbing that turns them into a language — and it is the part of Unix that has been copied by every system built since.

§12.1. Standard Input, Output and Error

Definition 12.1.1 (Standard streams)**.** Every process starts with three open file descriptors. **Standard input** (fd 0) is where it reads from, **standard output** (fd 1) is where its results go, and **standard error** (fd 2) is where its complaints go. By default all three are your terminal.

> **Intuition .** A program does not know or care what is on the other end of those three descriptors. It might be your keyboard, a file, or another program — and the program is written identically either way. That indifference is the entire mechanism behind everything in this chapter.

Splitting output from errors looks fussy until the first time it saves you:

```
$ ls real.txt missing.txt
ls: cannot access 'missing.txt': No such file or directory     # this went to fd 2
real.txt                                                       # this went to fd 1
$ ls real.txt missing.txt 2>/dev/null
real.txt
$ ls real.txt missing.txt > out.txt
ls: cannot access 'missing.txt': No such file or directory     # errors still appear
```

Note 12.1.2. The second case is why error messages appear on screen even when you have redirected output to a file: they were never on fd 1. This is a feature — a pipeline's data stays clean while its diagnostics still reach you.

§12.2. Redirection

Redirection attaches a stream to a file instead of the terminal. It is handled by the shell before the program starts, so the program never sees the `>`.

`> file`	Send stdout to `file`, **destroying** its contents
`>> file`	Append to `file` instead
`< file`	Take stdin from `file`
`2> file`	Send stderr to `file`
`2>> file`	Append stderr
`2>&1`	Send stderr wherever stdout is currently going
`&> file`	Both streams to one file (Bash and Zsh)
`> /dev/null`	Discard — `/dev/null` accepts anything and keeps none
`n> file`	Redirect an arbitrary descriptor

Note 12.2.1 (Order matters for `2>&1`)**.** `cmd > out.txt 2>&1` sends both streams to the file. `cmd 2>&1 > out.txt` does *not*: redirections are applied left to right, so stderr is pointed at the terminal (where stdout is at that moment) and only then is stdout moved to the file. The second form is almost always a bug, and it is subtle enough to survive code review.

`>` truncates before the command even runs. So `sort file > file` empties the file and then sorts nothing, and there is no recovery. Write to a new name, or use a tool with an in-place option such as `sed -i`.

§12.3. Pipes and Filters

Definition 12.3.1 (Pipe)**.** `a | b` connects `a`'s standard output directly to `b`'s standard input. Both run *at the same time*, with the kernel buffering between them — no temporary file is created, and `b` can start producing output before `a` has finished.

Definition 12.3.2 (Filter)**.** A program that reads standard input and writes standard output, so that it can sit in the middle of a pipeline. `grep`, `sort`, `sed`, `tr`, `uniq`, `wc` and `awk` are all filters, which is why they combine.

Example 12.3.3. Reading a pipeline is a matter of reading it left to right, one arrow at a time:

```
$ ps aux | grep nvim | awk '{print $2}' | xargs kill
```

List every process; keep the lines mentioning `nvim`; take the second column of each, which is the process ID; hand those IDs to `kill`. Four single-purpose programs, no temporary files, no program aware of the others.

Note 12.3.4. The exit status of a pipeline is the status of its *last* command, so a failure in the middle is silently ignored. `set -o pipefail` (Section 14.7) changes this, and any script with a pipeline in it should set it.

Try it

Build this up one stage at a time, pressing `<CR>` after each addition, and watch the output narrow: `cut -d:  -f7 /etc/passwd`, then `| sort`, then `| uniq -c`, then `| sort -rn`. You have just answered "which login shells are in use here, and how popular is each".

§12.4. `tee` and `xargs`

Two commands exist to patch the two places where pipes fall short.

`tee` solves "I want this both on screen and in a file":

`cmd | tee log.txt`	Write to the file *and* pass through
`cmd | tee -a log.txt`	Append rather than overwrite
`cmd | tee /dev/tty | c2`	Show the intermediate stage of a pipeline
`echo x | sudo tee /etc/f`	Write to a root-owned file — `sudo cmd > file` does not work, because the shell opens the file, not `sudo`

`xargs` solves the opposite problem: many programs take filenames as *arguments*, not on standard input.

`... | xargs rm`	Turn input lines into arguments
`... | xargs -n 1 cmd`	One argument per invocation
`... | xargs -I{} cp {} /bak`	Place the argument somewhere specific
`... | xargs -P 4 cmd`	Run four invocations in parallel
`find . -print0 | xargs -0`	Separate with NUL bytes, not whitespace

Note 12.4.1 (Filenames with spaces)**.** By default `xargs` splits on whitespace, so a file named `my file.txt` arrives as two arguments and the wrong things get deleted. The fix is the `-print0`/`-0` pair: NUL is the one byte a filename cannot contain. Make it a habit — `find . -name '*.log' -print0 | xargs -0 rm` — or sidestep the issue entirely with `find -exec` (Section 10.7).

§12.5. Here-Documents and Here-Strings

When the input you want to pipe in is text you are writing right now, there is no need for a file.

```
# here-document: everything up to the terminator becomes stdin
cat > config.yml <<EOF
name: $PROJECT      # expanded, because EOF is unquoted
port: 8080
EOF

# quoted terminator: nothing is expanded
cat > script.sh <<'EOF'
echo "$HOME stays literal"
EOF

# <<- strips leading TABS, so the heredoc can be indented with its block
if true; then
    cat <<-EOF
    indented in the source, flush left in the output
    EOF
fi

# here-string: one line, no terminator needed
grep -q root <<< "$line"
```

Quote the terminator — `<<'EOF'` — whenever the body contains `$` signs you want to survive, which is every time you are generating a shell script, a Makefile, or anything else with variables of its own.

§12.6. Command and Process Substitution

`$(cmd)`	**Command substitution**: replaced by `cmd`'s output
`` `cmd` ``	The old spelling. Does not nest, and escaping inside it is unreadable — do not use it
`<(cmd)`	**Process substitution**: replaced by a *filename* that yields `cmd`'s output
`>(cmd)`	The same, for writing into
`$((expr))`	**Arithmetic expansion**

```
$ echo "built on $(date +%F) by $(whoami)"
built on 2026-09-14 by konyi
$ files=$(ls *.tex | wc -l)
$ echo $((files * 2))
16
```

Process substitution is the less known of the two, and it solves a real problem: comparing the output of two commands, when `diff` insists on filenames.

```
$ diff <(sort a.txt) <(sort b.txt)
$ diff <(ls dir1) <(ls dir2)
```

Note 12.6.1. Without it you would write two temporary files and remember to delete them. `<(cmd)` expands to something like `/dev/fd/63` — a real path, which `diff` opens and reads perfectly happily. It is a Bash and Zsh feature, not POSIX, so a script declaring `#!/bin/sh` cannot use it.

Always quote command substitution: `"$(cmd)"`. Unquoted, its output goes through word splitting and globbing like any other expansion (Section 9.4), and a filename with a space in it will take your script apart.

§12.7. Exit Status and the Logical Operators

Every command returns a number when it finishes. Zero means success; anything else is a failure, and which number often says which failure.

`$?`	The exit status of the last command
`0`	Success — the only value that means it
`1`	General failure
`2`	Usually a usage error
`126`	Found, but not executable
`127`	Command not found
`130`	Terminated by `<C-c>` — that is $128 + 2$, the signal number

Note 12.7.1. Zero-means-success looks backwards until you see why: there is one way to succeed and many ways to fail, so the failures get the numbers.

Statuses are what the logical operators test:

`a ; b`	Run `a`, then `b`, regardless
`a && b`	Run `b` only if `a` *succeeded*
`a || b`	Run `b` only if `a` *failed*
`! a`	Invert the status
`a &`	Run `a` in the background (Section 13.2)

Example 12.7.2.

```
$ mkdir build && cd build           # only cd if the mkdir worked
$ make || echo "build failed"       # report only on failure
$ grep -q TODO file && echo "still unfinished"
$ command -v rg >/dev/null || echo "ripgrep is not installed"
```

The last is the standard way for a script to check whether a tool exists before relying on it.

`&&` between commands is the cheapest error handling there is. In a script, `cd build && make` cannot run `make` in the wrong directory, whereas `cd build; make` happily will — and that difference has destroyed real work.

13. Processes and the Environment

Everything you have run so far became a **process**: a program in mid-flight, with its own memory, its own file descriptors, and its own copy of the environment. This chapter is about seeing them, controlling them, and understanding the environment they inherit — which is where a surprising share of "it works on my machine" lives.

§13.1. Processes

Definition 13.1.1 (Process)**.** A running instance of a program, identified by a **process ID** (PID). Every process but the first has a parent, and the parent–child relationship forms a tree: your terminal started your shell, and your shell starts everything you type.

`ps`	Your processes in this terminal
`ps aux`	Every process on the system, in BSD syntax
`ps -ef`	The same, in System V syntax
`ps -p $$`	Just this shell
`pstree -p`	The process tree, with PIDs
`pgrep nvim`	PIDs of processes whose name matches
`top` / `htop`	Live, sorted by resource use
`lsof -p 1234`	Which files a process has open
`lsof -i :8080`	Which process is holding a network port

`$$`	The PID of the current shell
`$!`	The PID of the most recent background command
`$?`	The exit status of the last command
`$0`	The name of the running shell or script

Example 13.1.2. The two most common uses, by a wide margin:

```
$ ps aux | grep '[n]vim'          # what is running, and as whom
$ lsof -i :3000                   # what is squatting on the port I want
```

The bracket trick in the first is a small classic: `[n]vim` matches the text `nvim`, but the `grep` process's own command line contains `[n]vim` rather than `nvim`, so it does not match itself.

§13.2. Job Control

A **job** is a shell-level handle on a pipeline you started. Job control lets you run several at once from one terminal without opening more of them.

`cmd &`	Start it in the background immediately
`<C-z>`	Suspend the foreground job
`bg`	Resume the suspended job, in the background
`fg`	Bring it back to the foreground
`fg %2`	Bring back job number 2
`jobs`	List this shell's jobs
`jobs -l`	... with PIDs
`<C-c>`	Interrupt the foreground job
`wait`	Block until all background jobs finish
`disown %1`	Detach a job, so it survives the shell exiting
`nohup cmd &`	Start it already immune to hangup

Example 13.2.1. You are editing, and need a shell for one command:

```
$ nvim report.tex
# press C-z
[1]+  Stopped                 nvim report.tex
$ make
$ fg
# back in Neovim, exactly where you left it
```

Faster than quitting and reopening, and your undo history survives. Neovim's own `:terminal` (Section 7.5) covers the same need from the other direction.

Note 13.2.2. Jobs belong to the shell that started them, and normally die when it exits. `nohup`, `disown` and `tmux` (Section 15.4) are three answers to that; over SSH, `tmux` is the one that also survives losing your connection.

§13.3. Signals

Definition 13.3.1 (Signal)**.** A short asynchronous message sent to a process. Most signals have a default behaviour — usually "terminate" — and most can be caught by the process and handled instead.

`SIGINT` (2)	Interrupt. What `<C-c>` sends. Catchable
`SIGTERM` (15)	Polite termination. `kill`'s default. Catchable
`SIGKILL` (9)	Immediate death. **Cannot** be caught or ignored
`SIGSTOP` (19)	Suspend. What `<C-z>` sends. Cannot be caught
`SIGCONT` (18)	Resume a stopped process
`SIGHUP` (1)	The terminal went away. Often means "reload config"
`SIGSEGV` (11)	Invalid memory access — see Section 28.1

`kill 1234`	Send `SIGTERM` to PID 1234
`kill -9 1234`	Send `SIGKILL`
`kill -HUP 1234`	By name; `kill -l` lists them all
`killall nvim`	By process name
`pkill -f 'python.*server'`	By matching the full command line

Try `kill` before `kill -9`. `SIGTERM` lets a program flush its buffers, remove its lock files and save your work; `SIGKILL` does not give it the chance, and the corrupted state it leaves behind is your problem. Reach for `-9` only when the polite version has already failed.

§13.4. Environment Variables

Definition 13.4.1 (Environment)**.** A set of name–value pairs that every process inherits from its parent, in a *copy*. A child can change its own copy, but nothing a child does can affect its parent's — which is why `cd` and `export` must be builtins.

`name=value`	A shell variable: visible here, not to children
`export name`	Promote it to an environment variable
`export n=v`	Both at once
`unset name`	Remove it
`env`	Print the whole environment
`echo "$PATH"`	Print one variable
`n=v cmd`	Set it for this one command only

`PATH`	Where to look for commands
`HOME`	Your home directory; what `~` expands to
`PWD`	The current directory
`USER`	Your login name
`SHELL`	Your login shell, from `/etc/passwd`
`EDITOR`	What Git, `crontab` and others open for you
`PAGER`	What `man` and `git log` pipe through
`LANG`	Language and character encoding
`TERM`	The terminal type, which decides what colours work

`PATH` deserves its own paragraph, because it is where "command not found" comes from. It is a colon-separated list of directories, searched left to right, and the first match wins:

```
$ echo $PATH
/home/konyi/.local/bin:/usr/local/bin:/usr/bin:/bin
$ which -a python3          # every match, in search order
/home/konyi/miniconda3/bin/python3
/usr/bin/python3
$ export PATH="$HOME/bin:$PATH"     # prepend: yours wins
```

Note 13.4.2. Prepend to put your own tools first; append to use them only as a fallback. Note that `.` is deliberately *not* on `PATH`: if it were, `cd`-ing into a directory containing a malicious file called `ls` would be enough to run it.

Example 13.4.3. Setting a variable for one command is the cleanest way to change behaviour temporarily, and leaves no trace afterwards:

```
$ EDITOR=nvim git commit
$ LANG=C sort names.txt        # byte order, not locale-dependent order
$ DEBUG=1 ./run.sh
```

§13.5. Startup Files

Which file runs when is the most-searched question about shell configuration, and the answer follows the distinction in Section 9.2.

`~/.zshrc`, `~/.bashrc`	Every *interactive* shell. Aliases, prompt, key bindings
`~/.zprofile`, `~/.bash_profile`	*Login* shells only. Environment variables, `PATH`
`~/.profile`	The POSIX fallback, read by many login shells and display managers
`~/.zshenv`	Zsh: read by *every* shell, including scripts
`/etc/profile`, `/etc/zsh/*`	System-wide versions, read first

Environment variables belong in the *login* file; aliases, functions and prompt settings belong in the *interactive* file. Put `PATH` in `.bashrc` and it is re-prepended every time you open a nested shell, which is how people end up with the same directory in `PATH` nine times.

`source ~/.zshrc`	Re-read it in the current shell
`. ~/.zshrc`	The same thing; `.` is POSIX
`exec zsh`	Replace this shell with a fresh one

Note 13.5.1. `source` runs the file *in the current shell*, so its variables and functions stick. Running `./script.sh` starts a child, and everything it sets vanishes when it exits. This is why activation scripts — virtualenvs, `conda` — always tell you to `source` them.

§13.6. Aliases and Functions

`alias ll='ls -lh'`	Define an alias
`alias`	List them all
`unalias ll`	Remove one
`\ls`	Run the real command, bypassing any alias
`command ls`	The same, and script-safe

Aliases do simple textual substitution and cannot take arguments in the middle. When you need that, write a function:

```
# make a directory and enter it
mkcd() {
  mkdir -p "$1" && cd "$1"
}

# find a file by name under the current directory
ff() {
  find . -iname "*$1*" -not -path '*/.git/*'
}

# extract any archive, without remembering the flags
extract() {
  case "$1" in
    *.tar.gz|*.tgz) tar xzf "$1" ;;
    *.tar.bz2)      tar xjf "$1" ;;
    *.zip)          unzip "$1"   ;;
    *) echo "don't know how to extract $1" >&2; return 1 ;;
  esac
}
```

Note 13.6.1. Aliases are not expanded inside scripts, so a script that relies on your `ll` will fail for everyone else, including future you. Functions defined in `.bashrc` have the same problem. Anything a script needs should be written out in full or defined in the script itself.

§13.7. Measuring

`time cmd`	How long did it take — real, user, and system time
`/usr/bin/time -v`	Much more detail, including peak memory
`top`, `htop`	Live CPU and memory, by process
`free -h`	Memory in use and available
`df -h`	Disk space per filesystem
`du -sh * | sort -h`	What is using the space here
`uptime`	How long since boot, and the load average
`nice -n 10 cmd`	Run at lower priority
`ulimit -a`	Resource limits for this shell

Example 13.7.1. `time` distinguishes three quantities, and the distinction is the whole point:

```
$ time make -j4
real    0m12.431s      # wall-clock time you waited
user    0m41.208s      # CPU time in your program, summed over all cores
sys     0m3.117s       # CPU time inside the kernel
```

`user` exceeding `real` means the work really was parallel. If `real` greatly exceeds `user + sys`, the program spent its time waiting — on disk, on the network, or on a lock — and buying a faster CPU will not help.

Try it

Run `du -sh * | sort -h` in your home directory. It combines Section 10.3, Section 11.3 and Section 12.3 into one line, and it is usually the fastest answer to "where did my disk space go".

14. Shell Scripting

A shell script is a file containing the commands you would have typed. That is its whole appeal — the step from "I did this by hand" to "this is automated" is copying your history into a file — and also its whole danger, since the shell was designed for interactive use, where a mistake is visible immediately. This chapter covers the language, and then the handful of settings that make it safe to leave running unattended.

§14.1. Shebangs and Executable Scripts

Definition 14.1.1 (Shebang)**.** The first line of a script, beginning `#!`, naming the interpreter that should run it. The kernel reads those two bytes, and runs the named program with the script as its argument.

```
#!/bin/bash           # this exact shell
#!/bin/sh             # POSIX only: no arrays, no [[ ]], no process substitution
#!/usr/bin/env bash   # the first bash on PATH -- more portable
#!/usr/bin/env python3
```

```
$ chmod +x deploy.sh     # without this you get "Permission denied"
$ ./deploy.sh            # the ./ is required: . is not on PATH
```

Note 14.1.2. `#!/usr/bin/env bash` is the portable form, because `bash` lives in `/bin` on Linux and `/opt/homebrew/bin` on an Apple Silicon Mac, while `env` is in `/usr/bin` everywhere. The cost is that it obeys `PATH`, so it picks up whichever `bash` the caller happens to have.

Declare `bash` if you use Bash features. Writing `#!/bin/sh` and then using `[[ ]]` or arrays produces a script that runs on your machine — where `/bin/sh` may be a symlink to `bash` — and fails on a Debian server where it is `dash`.

§14.2. Variables, Expansion and Arrays

```
name="Kon Yi"          # no spaces around = -- "name = x" tries to RUN name
readonly PI=3.14159    # cannot be changed afterwards
local tmp="$1"         # inside a function only: not visible outside

echo "${name}"         # braces: always safe, required before adjacent text
echo "${name}_final"   # without braces this looks for $name_final
```

Parameter expansion has a small sublanguage of its own, and it replaces a surprising amount of `sed`:

`${v:-default}`	Use `default` if `v` is unset or empty
`${v:=default}`	... and assign it as well
`${v:?message}`	Abort with `message` if unset — a cheap assertion
`${#v}`	Its length
`${v#prefix}`	Strip the shortest matching prefix
`${v##prefix}`	Strip the longest matching prefix
`${v%suffix}`	Strip the shortest matching suffix
`${v%%suffix}`	Strip the longest matching suffix
`${v/old/new}`	Replace the first match
`${v//old/new}`	Replace every match

Example 14.2.1.

```
$ f=/home/konyi/report.tex
$ echo "${f##*/}"        # everything after the last slash -> basename
report.tex
$ echo "${f%/*}"         # everything before the last slash -> dirname
/home/konyi
$ echo "${f%.tex}.pdf"   # swap the extension
/home/konyi/report.pdf
$ echo "${VERBOSE:-0}"   # a default for an unset variable
0
```

The mnemonic for the direction: on a US keyboard `#` is left of `$` and strips from the left; `%` is right of it and strips from the right.

Arrays are Bash-only, and indexed from zero:

```
files=(a.txt b.txt "with space.txt")
echo "${files[0]}"        # a.txt
echo "${#files[@]}"       # 3 -- the number of elements
for f in "${files[@]}"; do echo "$f"; done   # quoted [@]: one word each
```

`"${arr[@]}"` with the quotes is the only correct way to iterate an array. `${arr[*]}` joins everything into one string, and the unquoted forms split on spaces — so the element `with space.txt` becomes two iterations.

§14.3. Arguments

`$0`	The script's own name
`$1`, `$2`	The first and second arguments
`${10}`	The tenth — braces required past nine
`$#`	How many arguments were given
`"$@"`	All of them, *each as a separate word*
`"$*"`	All of them, joined into one word
`shift`	Discard `$1`; everything moves down one

Use "$@", never $*, and never either without quotes. The quoted "$@" is the only form that passes arguments through unchanged when one of them contains a space.

```
#!/usr/bin/env bash
# usage: backup.sh [-v] SOURCE DEST

verbose=0
while getopts "vh" opt; do
  case "$opt" in
    v) verbose=1 ;;
    h) echo "usage: $0 [-v] SOURCE DEST"; exit 0 ;;
    *) exit 2 ;;                # getopts already printed the error
  esac
done
shift $((OPTIND - 1))           # drop the options, leaving the operands

if [[ $# -ne 2 ]]; then
  echo "error: expected 2 arguments, got $#" >&2
  exit 2
fi

src=$1
dest=$2
(( verbose )) && echo "copying $src -> $dest"
cp -r -- "$src" "$dest"
```

Note 14.3.1. Usage errors go to standard error (`>&2`) and exit non-zero, so that a caller can tell success from failure and a pipeline is not polluted with your complaints. This is the same discipline every tool in Chapter 11 follows.

§14.4. Conditionals

`[ ... ]`	The POSIX test. It is a *command* named `[`, which is why the spaces are mandatory
`[[ ... ]]`	Bash's version: safer, supports `&&`, `||`, `=~`
`(( ... ))`	Arithmetic. `$` is not needed on variable names
`-f f`	`f` exists and is a regular file
`-d d`	`d` exists and is a directory
`-e p`	`p` exists, whatever it is
`-r`, `-w`, `-x`	... and is readable, writable, executable
`-s f`	`f` exists and is not empty
`-z "$v"`	The string is empty
`-n "$v"`	The string is not empty
`"$a" = "$b"`	Strings are equal
`-eq -ne -lt -le -gt -ge`	Numeric comparison
`"$v" =~ re`	Matches a regular expression (`[[ ]]` only)

```
if [[ -f "$config" ]]; then
  source "$config"
elif [[ -f /etc/defaults.conf ]]; then
  source /etc/defaults.conf
else
  echo "no configuration found" >&2
  exit 1
fi

case "$1" in
  start)        systemctl start app ;;
  stop)         systemctl stop app ;;
  restart)      "$0" stop && "$0" start ;;
  *.tar.gz)     tar xzf "$1" ;;        # patterns are GLOBS, not regexes
  *)            echo "unknown: $1" >&2; exit 2 ;;
esac
```

Note 14.4.1. Prefer `[[ ]]` in Bash scripts. Inside it, an unquoted empty variable does not fall apart — `[ $x = y ]` with `x` unset expands to `[ = y ]`, which is a syntax error, while `[[ $x = y ]]` is simply false. Quote anyway; but the brackets give you a second line of defence.

§14.5. Loops and Reading Input

```
for f in *.tex; do
  echo "processing $f"
done

for i in {1..5}; do echo "$i"; done
for (( i = 0; i < 5; i++ )); do echo "$i"; done

while read -r line; do            # -r: do not mangle backslashes
  echo "[$line]"
done < input.txt

# the safe way to loop over find's results
while IFS= read -r -d '' file; do
  echo "$file"
done < <(find . -name '*.log' -print0)

until ping -c1 -W1 example.com &>/dev/null; do
  echo "waiting for network..."
  sleep 2
done
```

Never write `for f in $(ls)`. The output of `ls` is split on whitespace, so any filename with a space becomes several iterations, and any name containing a `*` is then globbed. `for f in *` does the right thing, needs no subprocess, and is shorter.

Note 14.5.1. `IFS= read -r` is the incantation for reading a line *exactly*: clearing `IFS` stops leading and trailing whitespace being trimmed, and `-r` stops backslashes being interpreted. Use it every time; the bare `read` is almost never what you want.

§14.6. Functions

```
log() {
  printf '[%s] %s\n' "$(date +%T)" "$*" >&2
}

die() {
  log "FATAL: $*"
  exit 1
}

file_size() {
  local f=$1                    # local: invisible outside this function
  [[ -f $f ]] || return 1       # return status, not a value
  stat -c%s "$f"                # "return a value" by printing it
}

size=$(file_size report.pdf) || die "report.pdf is missing"
log "report.pdf is $size bytes"
```

Note 14.6.1. Shell functions do not return values, they return *statuses* — a number from 0 to 255, where 0 means success. To hand back data, print it and let the caller capture it with `$(...)`. And declare every internal variable `local`, or it silently becomes global and collides with the caller's.

§14.7. Writing Robust Scripts

By default the shell is relentlessly forgiving: a failing command is ignored, an unset variable expands to nothing, and a broken pipeline reports success. Those defaults are right for an interactive session, where you can see what happened, and wrong for everything else.

```
#!/usr/bin/env bash
set -euo pipefail
IFS=$'\n\t'
```

`set -e`	Exit as soon as any command fails
`set -u`	Treat an unset variable as an error, instead of an empty string
`set -o pipefail`	A pipeline fails if *any* stage failed, not just the last
`IFS=$'\n\t'`	Split fields on newlines and tabs only, never spaces

Example 14.7.1. What each one prevents:

```
cd /nonexistent      # without -e, the script carries on...
rm -rf ./*           # ...and deletes the contents of the WRONG directory

rm -rf "$PREFIX/lib" # with PREFIX unset and no -u, this is rm -rf /lib

grep pat file | sort # without pipefail, a missing file still "succeeds"
```

Note 14.7.2 (`set -e` is not magic)**.** It does not fire inside a condition, or on the left of `&&`, or anywhere its status is being tested — which is correct, but means it is a safety net rather than a guarantee. Check explicitly where it matters: `cmd || die "cmd failed"`.

Clean up after yourself with a trap:

```
tmp=$(mktemp -d)                        # never invent your own temp filenames
trap 'rm -rf "$tmp"' EXIT               # runs however the script ends
trap 'echo interrupted >&2; exit 130' INT TERM
```

Those four lines — the shebang, `set -euo pipefail`, `mktemp`, and a `trap ... EXIT` — are the difference between a script you run yourself and a script you can put in a cron job. Type them before you type anything else.

§14.8. Debugging Scripts

`bash -n script.sh`	Check the syntax without running anything
`bash -x script.sh`	Print every command, after expansion, as it runs
`set -x` / `set +x`	Turn tracing on and off around one suspect region
`PS4='+$LINENO: '`	Prefix trace lines with the line number
`shellcheck script.sh`	Static analysis. Catches most of this chapter's traps

Example 14.8.1. Tracing shows you the command *after* every expansion of Section 9.4 has been applied, which is exactly the information a quoting bug hides:

```
$ bash -x backup.sh a.txt "my dir"
+ src=a.txt
+ dest='my dir'
+ cp -r -- a.txt 'my dir'
```

The quotes in the trace are the shell telling you where the word boundaries really are.

Run `shellcheck` on every script you write. It is not installed here (`apt install shellcheck`), it takes a second, and it reliably finds the unquoted variable you were about to spend an hour on.

15. Everyday Tools

The previous chapters were about the shell as a language. This one is a short tour of the programs you will reach for around it — not because they are conceptually deep, but because not knowing them costs an hour every time they come up.

§15.1. Archives and Compression

Unix separates two jobs that most systems merge: `tar` bundles many files into one, and `gzip` makes one file smaller. That is why the common extension is `.tar.gz` — it went through both.

`tar czf out.tar.gz dir/`	**C**reate a gzipped archive
`tar xzf in.tar.gz`	E**x**tract one
`tar tzf in.tar.gz`	Lis**t** the contents without extracting
`tar xzf in.tar.gz -C dst`	Extract somewhere other than here
`tar cJf out.tar.xz dir/`	`xz` instead — smaller, slower
`gzip file` / `gunzip file.gz`	Compress or decompress in place
`zip -r out.zip dir/` / `unzip in.zip`	For talking to other operating systems

Note 15.1.1. Modern GNU `tar` detects the compression itself, so `tar xf anything` works. The letters are still worth knowing because you will read them in other people's scripts: `c`reate, extract, `t`est/list, with `z` for gzip, `j` for bzip2, `J` for xz, `v` for verbose and `f` for "the next word is the filename".

Always run `tar tzf` before `tar xzf` on an archive you did not make. A well-behaved archive contains a single top-level directory; a badly made one unpacks two hundred files into your current directory, and you will be cleaning up by hand.

§15.2. Remote Work

`ssh user@host`	Log in to a remote machine
`ssh user@host 'cmd'`	Run one command and come straight back
`ssh -p 2222 user@host`	A non-default port
`ssh-keygen -t ed25519`	Make a key pair — do this once
`ssh-copy-id user@host`	Install your public key there
`scp file user@host:~/`	Copy a file up
`scp user@host:remote.txt .`	… and back down
`rsync -avz src/ user@host:dst/`	Sync a tree, transferring only differences
`rsync -avzn src/ dst/`	`-n` is a dry run: show what would happen

Note 15.2.1 (The trailing slash)**.** `rsync -a src/ dst/` copies the *contents* of `src` into `dst`. `rsync -a src dst/` copies the directory itself, producing `dst/src`. One character, two quite different results, and this is the most-consulted line in `rsync`'s manual.

Example 15.2.2 (`~/.ssh/config`)**.** Configure a host once and stop typing its details:

```
Host lab
    HostName compute.university.edu
    User konyi
    Port 2222
    IdentityFile ~/.ssh/id_ed25519
```

After which `ssh lab`, `scp file lab:` and `rsync -a dir lab:` all work. Keys must be private: `chmod 600` on the key file, or SSH refuses to use it (Section 10.6).

Use keys, not passwords. `ssh-keygen` once and `ssh-copy-id` per host is five minutes of setup that removes a password prompt from every remote command you will run for years — and it is the same key GitHub will want in Section 34.2.

§15.3. Fetching Things

`curl URL`	Print the response to standard output
`curl -O URL`	Save it under its remote name
`curl -o out.json URL`	Save it under a name you choose
`curl -L URL`	Follow redirects — needed more often than not
`curl -I URL`	Headers only
`curl -fsSL URL`	Fail loudly, stay quiet otherwise: the flags every install script uses
`wget URL`	Save to a file by default
`wget -r -np URL`	Mirror a site, without climbing to the parent

Example 15.3.1. `curl` writes to standard output, so it composes with Chapter 12 like any other filter:

```
$ curl -s https://api.github.com/repos/neovim/neovim | jq -r '.stargazers_count'
86214
```

`jq` is to JSON what `awk` is to columns, and it is installed here.

Note 15.3.2. `curl -fsSL URL | bash` is how a great many projects tell you to install them. It runs code you have not read, as you, from a server you do not control. At minimum, drop the pipe and read the script first — the instruction to pipe it straight into a shell is a convenience for the project, not for you.

§15.4. Terminal Multiplexing

Definition 15.4.1 (Multiplexer)**.** A program that runs a terminal session inside itself, so that the session survives your disconnecting — and so that one window can hold many shells. `tmux` is the current standard; `screen` is its ancestor.

`tmux`	Start a new session
`tmux new -s work`	... with a name
`tmux ls`	List sessions
`tmux attach -t work`	Reattach to one
`<C-b>` `d`	Detach — the session keeps running
`<C-b>` `c`	New window
`<C-b>` `n` / `p`	Next / previous window
`<C-b>` `%` / `"`	Split vertically / horizontally
`<C-b>` arrow	Move between panes
`<C-b>` `[`	Copy mode: scroll back, with Vim keys

Example 15.4.2. The reason to care is SSH. Start `tmux` on the remote machine, launch a six-hour job, press `<C-b>` `d`, and close your laptop. Tomorrow, `ssh lab` and `tmux attach` put you back in front of it, output and all. Without a multiplexer, closing the connection sends `SIGHUP` and the job dies (Section 13.3).

Note 15.4.3. `<C-b>` is the *prefix*: press and release it, then press the command key. It is a two-key sequence, exactly like Vim's `<C-w>` window commands (Section 6.3), and many people remap it to `<C-a>` or `<C-Space>` for comfort.

§15.5. A Handful of One-Liners

Every one of these is built from pieces already covered. They are collected here because recognising the shape is most of the skill.

```
# the ten largest things in this directory
du -sh * | sort -rh | head

# how many lines of LaTeX have I written?
find . -name '*.tex' -not -path './build/*' | xargs wc -l | tail -1

# most frequent lines in a log, commonest first
sort access.log | uniq -c | sort -rn | head

# every TODO in the project, with file and line number
grep -rn 'TODO' --include='*.tex' .

# which files changed in the last hour
find . -type f -mmin -60

# rename every .jpeg to .jpg
for f in *.jpeg; do mv -- "$f" "${f%.jpeg}.jpg"; done

# strip Windows line endings from every file in a tree
find . -type f -name '*.txt' -exec sed -i 's/\r$//' {} +

# a quick backup, timestamped
cp report.tex "report.$(date +%F).tex"

# wait for a port to open, then carry on
until nc -z localhost 8080; do sleep 1; done

# what is using this port?
lsof -i :8080

# disk usage of every subdirectory, sorted, human-readable
du -h --max-depth=1 . | sort -h

# find files tracked by neither git nor .gitignore
git status --porcelain | grep '^??'
```

None of these were memorised; each was assembled from four or five commands that each do one thing. That is what Part II was for. When you next face a task like these, do not search for the incantation — name the steps, and the pipeline will write itself.

Try it

Take the first recipe and build it up one stage at a time, as in Section 12.3: `du -sh *`, then `| sort -rh`, then `| head`. Then change it to answer a question of your own — the ten largest *files* anywhere below here, rather than the ten largest entries in this directory. The answer is a `find` away, and you already know it.

Part III.

Make and Makefiles

`make` is older than most of the languages it builds, and it has outlived a long line of intended replacements. The reason is that it solves a problem that never went away: given a pile of files, some of which are derived from others, rebuild exactly what is out of date and nothing else.

That is a statement about a *dependency graph*, not about C. The same machinery compiles a program, typesets a document, or regenerates a figure from a data file — and the reason Makefiles have a reputation for being cryptic is that people learn the syntax before the graph. Chapter 16 does it in the other order, after fixing this part's notation in Section 16.1. Chapters 17 to 19 then build up the syntax until one rule covers every source file; Chapter 20 and Chapter 21 cover the conventions and the tooling; and Chapter 22 assembles three Makefiles worth stealing.

16. Why a Build System

`make` was written in 1976, is older than most of the languages it builds, and has outlived a long procession of intended replacements. That survival is not inertia. It solves a problem that never went away, and it solves it with one idea small enough to fit in a paragraph.

§16.1. Notation for Part III

Makefile excerpts appear in framed blocks, as shell transcripts did in Part II:

```
program: main.o util.o        # target: prerequisites
    gcc -o program main.o util.o  # recipe
```

Note 16.1.1 (Indentation in these listings)**.** Recipe lines are printed indented, as above. In a real Makefile that indentation must be a single literal tab character — not the spaces the page appears to show. Section 17.2 is about nothing else, and it is the first thing to check when a Makefile will not run.

Terminal transcripts keep the convention of Section 9.1: a line beginning `$` is what you type, and everything else is output.

```
$ make
gcc -c main.c
gcc -o program main.o util.o
```

`$(VAR)`	A make variable being expanded. Parentheses are conventional; `${VAR}` is identical
`$$`	A literal dollar sign, passed through to the shell
`{target}`, `[prereq]`	Placeholders, as in Section 9.1

Note 16.1.2. The `$` character means two different things two lines apart. In `$(CC)` it is *make* expanding a variable; in a recipe, `$$HOME` is what you write to get the *shell* a single `$HOME`. Make expands first and the shell second, and forgetting the order is the most common Makefile bug after the tab.

§16.2. Compiling by Hand, and Where It Breaks

A one-file program needs no build system:

```
$ gcc -Wall -o hello hello.c
```

Three files is still manageable, if tedious:

```
$ gcc -Wall -c main.c
$ gcc -Wall -c parser.c
$ gcc -Wall -c util.c
$ gcc -o app main.o parser.o util.o
```

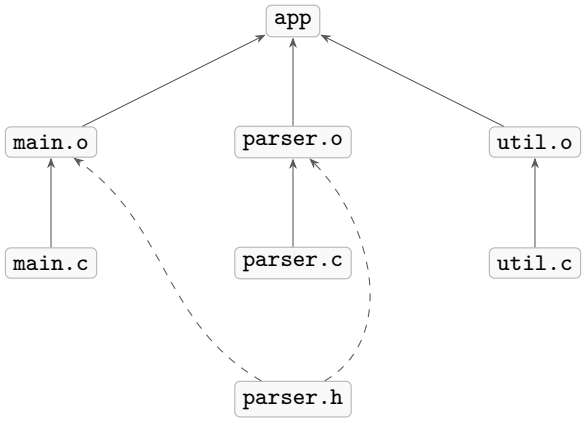
You could put those four lines in a shell script (Chapter 14) and call the problem solved. For a while it is. Then the project reaches forty source files, a full rebuild takes ninety seconds, and you are changing one line at a time — at which point the script is recompiling thirty-nine files that did not change, and you start editing the script to comment things out. That is the moment the problem stops being "run these commands" and becomes something else.

> **Intuition .** A build script answers *what commands do I run?* A build system answers *which of them do I still need to run?* The second question is the one that scales, and it cannot be answered without knowing what depends on what.

§16.3. Dependency Graphs and Incremental Builds

Definition 16.3.1 (Dependency graph)**.** A directed graph whose nodes are files and whose edges say "this file is built from that one". Building a target means visiting its prerequisites first, and an edge is **stale** when the prerequisite is newer than the thing derived from it.

For the three-file program above:



Solid edges are the obvious ones; the dashed pair is the shared header, which both `main.c` and `parser.c` include.

Now the question "what must I rebuild after editing `parser.c`?" has a mechanical answer: follow the arrows out of that node. `parser.o`, then `app`. Two commands, not four. Editing `parser.h` instead reaches `main.o` as well, because both source files include it — and *that* edge is the one a hand-written script always forgets.

Definition 16.3.2 (Incremental build)**.** Rebuilding only the parts of the graph that are out of date. The saving is not a constant factor: as a project grows, the full build grows with it while a typical incremental build stays roughly the same size.

> **Everything else in Part III is syntax. The content is this graph. When a Makefile misbehaves — rebuilding too much, or worse, too little — the question to ask is never "what is wrong with my rules" but "what does my graph actually say".**

§16.4. Make's Model

`make` implements exactly the above, with one simplifying decision that explains most of its behaviour and all of its failure modes.

Definition 16.4.1 (Make's rule)**.** Given a **target**, a list of **prerequisites**, and a **recipe** of shell commands: build each prerequisite first, then run the recipe if and only if the target does not exist, or any prerequisite has a *modification time* newer than the target's.

> **Intuition .** Make does not know what your compiler does, what a `.c` file is, or what changed inside it. It compares timestamps on files. That is the whole of its understanding, and it is why make works equally well for C, LaTeX, image conversion and data pipelines — and why it is fooled by anything that does not show up as a timestamp.

Note 16.4.2 (What timestamps cannot see)**.** Three consequences follow immediately, and all three will bite you eventually:

(i) Touching a file without changing it (`touch main.c`) forces a rebuild that achieves nothing.

(ii) Changing a *compiler flag* rebuilds nothing at all, because no file changed — your object files are now inconsistent and make is content.

(iii) A clock skew, or a file copied with its old timestamp, can make a prerequisite look older than its target, and the rebuild silently does not happen.

Section 19.5 fixes the first of these properly. For the second the answer is usually `make clean`.

§16.5. Where Make Sits

`make`	Rules and timestamps, by hand. Universal, tiny, and unopinionated
`CMake`	Generates Makefiles (or Ninja files) from a higher-level description; handles cross-platform and dependency discovery
`Ninja`	A make-like executor designed to be generated rather than written; much faster on large graphs
`latexmk`	A purpose-built builder for LaTeX that knows about reruns, `biber` and `makeindex` (Section 8.3)
`cargo`, `go build`	Language-specific; the dependency graph is derived from the source, not declared

Remark 16.5.1. CMake and Ninja have not replaced make so much as moved it. CMake's output is usually a Makefile, and reading it is the fastest way to appreciate why nobody writes those by hand. The reason to learn make anyway is that it is present on every Unix machine, needs no project, and is the right size for the jobs that are too small to justify a real build system — which is most of them.

Learn `make` for your own projects and for reading other people's. Reach for CMake when you need to build on Windows too, or when finding libraries becomes a project of its own.

17. Anatomy of a Makefile

A Makefile is a list of rules. Everything else — variables, functions, pattern matching, conditionals — exists to stop you from writing the same rule forty times.

§17.1. The Rule

```
{target}: [{prerequisites}]
→{recipe}
```

Definition 17.1.1 (Target, prerequisite, recipe)**.** The **target** is the file to be produced. The **prerequisites** are the files it is produced from. The **recipe** is the shell commands that produce it, each on its own line, each indented with a tab.

```
app: main.o parser.o util.o
    gcc -o app main.o parser.o util.o

main.o: main.c parser.h
    gcc -Wall -c main.c

parser.o: parser.c parser.h
    gcc -Wall -c parser.c

util.o: util.c
    gcc -Wall -c util.c
```

That is the graph of Section 16.3 written out. Running `make app` makes make read every rule, build the sub-graph rooted at `app`, and run only the recipes whose targets are out of date.

```
$ make app
gcc -Wall -c main.c
gcc -Wall -c parser.c
gcc -Wall -c util.c
gcc -o app main.o parser.o util.o
$ make app                      # nothing changed
make: 'app' is up to date.
$ touch parser.c
$ make app                      # only what the graph reaches
gcc -Wall -c parser.c
gcc -o app main.o parser.o util.o
```

Note 17.1.2. Rules may appear in any order — make reads the whole file before building anything, so a rule can name a prerequisite defined fifty lines later. The single exception is the *first* rule in the file, which is special (Section 17.3).

Note 17.1.3 (A target need not be a file)**.** Nothing checks that `app` is really produced. A rule whose target is never created just runs its recipe every time, which is how `clean` and `test` work — with a caveat that Section 20.1 spends its whole length on.

§17.2. The Tab Character

Every line of a recipe must begin with a literal tab. Not four spaces, not eight: a tab.

```
app:  main.o
↪  gcc -o app main.o
```

Note 17.2.1. The arrow marks the one `U+0009` byte that must begin that line. This is make's most notorious wart — its author called it a mistake within a few years, and by then it was too widely used to change.

Spaces instead of a tab produce make's least helpful error message:

```
$ make
Makefile:2: *** missing separator.  Stop.
```

`cat -A Makefile`	Show tabs as `^I` and line ends as `$`
`grep -P '\t' Makefile`	Which lines actually contain a tab
`sed -n 'l' Makefile`	The same, in POSIX
`<C-v>` `<Tab>`	In Vim's Insert mode: insert a literal tab even where `'expandtab'` is on

Note 17.2.2 (Editors that expand tabs)**.** Any editor configured to turn tabs into spaces will silently corrupt a Makefile. In Neovim, add a filetype rule (Section 7.3):

```
vim.api.nvim_create_autocmd("FileType", {
  pattern = "make",
  callback = function() vim.opt_local.expandtab = false end,
})
```

LazyVim already does this, and `:set list` makes the tabs visible so you can check.

Remark 17.2.3. GNU make since 3.82 can use another character instead. Setting the special variable `.RECIPEPREFIX` to `>` lets recipes be indented with `>` rather than a tab. It works, and almost nobody uses it: a Makefile that does is one more thing the next reader has to notice. Use the tab.

§17.3. The Default Goal

Running `make` with no arguments builds the **default goal**: the target of the first rule in the file.

```
all: app docs          # first rule -- so `make' means `make all'
app: main.o
    gcc -o app main.o
docs: manual.pdf
```

Make the first rule `all`, always, and make it depend on everything a plain `make` should produce. Otherwise the default goal is whatever rule happens to be at the top, and adding a rule in the wrong place quietly changes what `make` does.

Note 17.3.1. `.DEFAULT_GOAL := app` states it explicitly and is immune to reordering. It is worth using in a Makefile that other people will edit.

§17.4. How Recipes Are Run

Each line of a recipe runs in its *own* shell. Directory changes and variables do not carry to the next line.

```
broken:
    cd build          # this shell ends here...
    ls                # ...so this one is back where it started

works:
    cd build && ls    # one shell, one line

also-works:
    cd build; \
    ls                # a backslash joins them into one line
```

`@`	Do not echo this line before running it
`-`	Ignore a non-zero exit status; carry on
`+`	Run even under `make -n` (Section 21.1)

```
greet:
    @echo "building..."    # prints the message, not the echo command
clean:
    -rm -f *.o            # do not fail if there is nothing to remove
```

Note 17.4.1. Make aborts as soon as a recipe line exits non-zero, which is the behaviour you want — a build that carries on after a compile error produces a binary built from stale objects. The `-` prefix switches it off for one line, and `rm` in a `clean` rule is the main legitimate use. `rm -f` is usually better, since it succeeds on its own.

Remark 17.4.2. The shell is `/bin/sh`, not your login shell, so the Bash features of Chapter 14 — `[[ ]]`, arrays, process substitution — are unavailable unless you ask for them with `SHELL := /bin/bash`. A recipe that works when you paste it into your terminal and fails under make is almost always this.

§17.5. A First Makefile

Everything so far, in one file, for a three-source C program:

```
# ---- what to build -----------------------------------------------------
all: app

app: main.o parser.o util.o
    gcc -o app main.o parser.o util.o

main.o: main.c parser.h
    gcc -Wall -Wextra -g -c main.c

parser.o: parser.c parser.h
    gcc -Wall -Wextra -g -c parser.c

util.o: util.c util.h
    gcc -Wall -Wextra -g -c util.c

# ---- housekeeping ------------------------------------------------------
clean:
    rm -f app *.o

.PHONY: all clean
```

This is correct, and it is also the last time in Part III you will see a Makefile this repetitive. Three things about it are unsatisfying:

(i) The compiler and its flags are written out four times, so changing `-O2` means four edits and one you will miss.

(ii) The three `.o` rules are the same rule with different names.

(iii) The header dependencies are maintained by hand, so the first time you add an `#include` and forget to record it, make will confidently not rebuild.

Chapter 18 fixes the first, Chapter 19 the second and third. The result is about fifteen lines and works for any number of sources.

Try it

Create the three source files (any contents will do), type this Makefile — with real tabs — and run `make`, then `make` again, then `touch parser.h` and `make`. Watch which recipes run each time, and check them against the graph on page 90.

18. Variables and Expansion

Make has variables for the same reason every language does, and one extra: they are how you stop writing `gcc -Wall -Wextra -g` in four places and discovering later that one of them says `-Wall` only.

§18.1. Recursive versus Simple Assignment

There are two assignment operators and they are not interchangeable.

`=`	**Recursive**. The right-hand side is stored unexpanded and re-expanded at every use
`:=`	**Simple**. The right-hand side is expanded once, now, and the result is stored

```
# recursive: expanded when used
CFLAGS  = -Wall $(EXTRA)
EXTRA   = -O2
# $(CFLAGS) is "-Wall -O2" -- EXTRA was defined later, and that is fine

# simple: expanded here and now
SOURCES := $(wildcard *.c)
# the directory is scanned once, not once per reference
```

Note 18.1.1 (The trap)**.** Recursive assignment re-expands *every time*. Writing `SOURCES = $(shell find ...)` with `=` therefore runs `find` again at every reference, possibly dozens of times in one build. Worse, a recursive variable that refers to itself is a fatal error:

```
CFLAGS = $(CFLAGS) -g      # *** Recursive variable references itself.
CFLAGS := $(CFLAGS) -g     # fine: the right side is the OLD value
```

Use `:=` by default. Reach for `=` only when you genuinely need late binding — when the right-hand side mentions a variable that a later target-specific assignment will change.

§18.2. Conditional and Appending Assignment

`?=`	Assign only if the variable is not already set. Lets a caller override without editing the file
`+=`	Append, keeping the flavour of the original assignment
`!=`	Assign the output of a shell command (GNU make 4.0 and later)

```
CC      ?= gcc                  # `make CC=clang' wins; otherwise gcc
PREFIX  ?= /usr/local

CFLAGS  := -Wall -Wextra
CFLAGS  += -std=c17             # append
CFLAGS  += -g                   # append again

GIT_REV != git rev-parse --short HEAD
```

Note 18.2.1. `?=` is how a Makefile stays configurable. A user building your project can write `make CC=clang CFLAGS=-O3` and never touch the file — and packagers, who build thousands of projects, rely on exactly this.

§18.3. Automatic Variables

Inside a recipe, make defines a handful of variables describing the rule it is currently running. They are what make pattern rules possible at all, and they are the least readable part of the language.

`$@`	The target
`$<`	The *first* prerequisite
`$^`	*All* prerequisites, duplicates removed
`$+`	All prerequisites, duplicates kept, original order
`$?`	Only the prerequisites newer than the target
`$*`	The **stem**: what `%` matched (Section 19.2)
`$(@D)`, `$(@F)`	The target's directory and filename parts

Example 18.3.1. The same rule, spelled out and then with automatic variables:

```
app: main.o util.o
    gcc -o app main.o util.o

app: main.o util.o
    gcc -o $@ $^
```

Read it as *"make the target out of all the prerequisites"*. The second form cannot fall out of step with its own prerequisite list, which is the real argument for it.

Note 18.3.2. `$<` and `$^` differ in exactly the case that matters. Compiling one source into one object wants `$<`, because the prerequisites include headers and `gcc -c` must not be handed `parser.h`. Linking wants `$^`, because every object belongs on the command line.

```
main.o: main.c parser.h
    gcc -c -o $@ $<             # just main.c
app: main.o parser.o
    gcc -o $@ $^                # both objects
```

§18.4. Functions and Substitution References

Make has a small functional language for manipulating lists of words. A call is written as a dollar sign, a parenthesis, the function name, and its arguments; the arguments are almost always filenames.

`$(wildcard *.c)`	Expand a glob against the filesystem
`$(patsubst %.c,%.o,$(SRC))`	Pattern substitution over a list
`$(SRC:.c=.o)`	The same thing, in shorthand
`$(subst a,b,$(TXT))`	Plain textual substitution
`$(filter %.c,$(FILES))`	Keep the words that match
`$(filter-out main.o,$(OBJ))`	Drop the words that match
`$(sort $(LIST))`	Sort, and remove duplicates
`$(words $(LIST))`	How many words
`$(dir $(F))`, `$(notdir $(F))`	Directory and filename parts
`$(basename $(F))`, `$(suffix $(F))`	Without and only the extension
`$(addprefix build/,$(OBJ))`	Prepend to every word
`$(shell date +%F)`	Run a shell command, take its output
`$(foreach v,$(LIST),...)`	Loop over a list

Example 18.4.1 (The idiom you will use every time)**.**

```
SRC := $(wildcard src/*.c)
OBJ := $(SRC:.c=.o)
# or, equivalently and more explicitly:
OBJ := $(patsubst %.c,%.o,$(SRC))
```

Two lines, and the object list maintains itself. Adding a source file to `src/` needs no edit at all.

Note 18.4.2. `$(wildcard)` sees the filesystem *when the Makefile is read*. If a rule generates a `.c` file during the build, that file will not be in `$(SRC)` — make already made up its mind. Generated sources need their names written out, or a second invocation of make.

Remark 18.4.3. Make's functions are whitespace-sensitive in a way that catches everyone once: `$(filter %.c, $(FILES))` with a space after the comma passes *"space plus the list"* as the second argument. Do not put spaces after the commas in a function call.

§18.5. Conventional Variables

Make's built-in rules (Section 19.1) use a fixed set of variable names, and so does every Makefile anyone else will read. Use these names and a great deal works for free.

`CC`	The C compiler. Defaults to `cc`
`CXX`	The C++ compiler
`CFLAGS`	Flags for the C compiler
`CXXFLAGS`	Flags for the C++ compiler
`CPPFLAGS`	Preprocessor flags — `-I`, `-D`. Used by both
`LDFLAGS`	Linker flags — `-L`, `-Wl,...`
`LDLIBS`	Libraries to link — `-lm`, `-lpthread`
`ARFLAGS`	Flags for `ar`
`PREFIX`	Where `install` should put things

Note 18.5.1. `LDFLAGS` and `LDLIBS` are not the same, and the split is not pedantry. `LDFLAGS` says *where* to look and goes early on the command line; `LDLIBS` says *what* to link and must come after the object files, because the GNU linker resolves symbols in the order it is given them. Putting `-lm` in `LDFLAGS` produces `undefined reference to 'sqrt'` on a program that plainly links the maths library.

```
CC       := gcc
CPPFLAGS := -Iinclude -DNDEBUG
CFLAGS   := -Wall -Wextra -std=c17 -O2
LDFLAGS  := -Llib
LDLIBS   := -lm

app: $(OBJ)
    $(CC) $(LDFLAGS) -o $@ $^ $(LDLIBS)
```

§18.6. Overriding from Outside

Variables can be set in four places. Later entries win:

1. The Makefile — `CFLAGS := -O2`
2. The environment — `CFLAGS=-O3 make` — but only for variables not set in the file, unless `make -e`
3. The command line — `make CFLAGS=-O3` — beats the Makefile
4. `override` — `override CFLAGS += -g` — beats even the command line

Example 18.6.1.

```
$ make                          # CFLAGS as written in the file
$ make CFLAGS='-O0 -g'          # override for this build
$ make -e                       # let the environment win everywhere
$ make CC=clang -j8             # a different compiler, eight jobs
```

Note 18.6.2 (Target-specific variables)**.** A variable can be given a different value for one target and everything it depends on — which is how a debug build is usually expressed:

```
debug: CFLAGS += -O0 -g -fsanitize=address
debug: app
```

`make debug` then builds `app` with the extra flags. Note the caveat from Section 16.4: if the objects already exist from a normal build, make sees no changed timestamps and rebuilds nothing. Run `make clean` first, or give the debug build its own output directory.

Every value that someone might reasonably want to change — compiler, flags, install prefix, output directory — belongs in a variable at the top of the file, assigned with `?=`. It costs one line and saves the next person from editing your recipes.

19. Pattern and Implicit Rules

Section 17.5 ended with three complaints. This chapter answers two of them: how to write one rule that covers every source file, and how to stop maintaining header dependencies by hand.

§19.1. The Built-in Rule Database

Make ships knowing how to build some things already.

```
$ ls
main.c
$ make main.o                    # no Makefile at all
cc    -c -o main.o main.c
```

Definition 19.1.1 (Implicit rule)**.** A rule make already knows, matching targets by their suffix. Roughly a hundred are built in, covering C, C++, Fortran, Pascal, TEX, Yacc, Lex, and archive members.

`make -p`	Print the entire database, rules and variables
`make -p -f /dev/null`	Print only the built-ins, with no Makefile
`make -r`	Disable them all

The C rule is worth reading, because every Makefile in this chapter is a variation on it:

```
%.o: %.c
    $(CC) $(CPPFLAGS) $(CFLAGS) -c -o $@ $<
```

Note 19.1.2. That is why Section 18.5 insisted on the conventional variable names. Setting `CC` and `CFLAGS` alone changes how every `.o` is built, and you never write the rule. It is also why `-lm` belongs in `LDLIBS`: the built-in link rule puts `LDLIBS` last.

Remark 19.1.3. `make -r` (or `MAKEFLAGS += -r` inside the file) turns the database off. Large projects do this: searching a hundred irrelevant rules for every target costs measurable time, and an accidental match — make cheerfully building `foo` from a `foo.c` you forgot about — is worse than no match at all.

§19.2. Pattern Rules

Definition 19.2.1 (Pattern rule)**.** A rule whose target contains a `%`. The `%` matches any non-empty string — the **stem** — and the same stem is substituted into the prerequisites.

```
%.o: %.c
    $(CC) $(CPPFLAGS) $(CFLAGS) -c -o $@ $<
```

Asked for `parser.o`, make matches the stem `parser`, looks for `parser.c`, and runs the recipe with `$@` as `parser.o`, `$<` as `parser.c` and `$*` as `parser`.

Example 19.2.2. One pattern rule replaces all three of the `.o` rules in Section 17.5, and the whole Makefile becomes:

```
CC       := gcc
CPPFLAGS := -Iinclude
CFLAGS   := -Wall -Wextra -std=c17 -g
SRC      := $(wildcard *.c)
OBJ      := $(SRC:.c=.o)

all: app

app: $(OBJ)
    $(CC) $(LDFLAGS) -o $@ $^ $(LDLIBS)

%.o: %.c
    $(CC) $(CPPFLAGS) $(CFLAGS) -c -o $@ $<

clean:
    rm -f app $(OBJ)

.PHONY: all clean
```

Fifteen lines, and it handles any number of source files.

Patterns are not restricted to compilation:

```
%.pdf: %.tex
    latexmk -xelatex -interaction=nonstopmode $<

%.png: %.svg
    inkscape --export-type=png --export-filename=$@ $<

%.csv: %.json
    jq -r '.[] | [.name, .value] | @csv' $< > $@
```

Note 19.2.3. A pattern rule is used only when make needs the target and the prerequisite exists (or can itself be built). Several matching patterns are resolved by choosing the one with the shortest stem. If nothing matches, make reports `No rule to make target` — which nearly always means a typo in a filename rather than a missing rule.

Write pattern rules in terms of the automatic variables of Section 18.3, never in terms of concrete filenames. `$@` and `$<` are the only way a single rule can serve every file.

§19.3. Static Pattern Rules

A plain pattern rule applies to *anything* matching it. A static pattern rule applies only to a list you name.

```
{targets}: {target-pattern}: {prereq-patterns}
```

```
OBJ := main.o parser.o util.o

$(OBJ): %.o: %.c
    $(CC) $(CPPFLAGS) $(CFLAGS) -c -o $@ $<
```

Note 19.3.1. The difference matters when two kinds of file share a suffix. With a plain `%.o: %.c`, make will try to build *any* `.o` it hears about that way, including ones that should come from assembly or from a generator. Restricting the rule to a known list turns a silent wrong build into an honest `No rule to make target`.

Example 19.3.2. Static patterns are also how you build one list two ways:

```
RELEASE_OBJ := $(SRC:%.c=build/release/%.o)
DEBUG_OBJ   := $(SRC:%.c=build/debug/%.o)

$(RELEASE_OBJ): build/release/%.o: %.c
    $(CC) -O2 -DNDEBUG -c -o $@ $<

$(DEBUG_OBJ): build/debug/%.o: %.c
    $(CC) -O0 -g -c -o $@ $<
```

Two sets of objects from one set of sources, with no risk of the flags getting mixed — which, as Section 18.6 warned, is exactly what target-specific variables cannot guarantee.

§19.4. `VPATH` and Out-of-Tree Builds

Real projects do not keep sources, objects and the binary in one directory.

`VPATH = src:include`	A colon-separated list of directories to search for *any* prerequisite
`vpath %.c src`	The same, but only for files matching a pattern
`vpath %.h include`	… and headers somewhere else

```
vpath %.c src
vpath %.h include

BUILD := build
SRC   := $(wildcard src/*.c)
OBJ   := $(patsubst src/%.c,$(BUILD)/%.o,$(SRC))

$(BUILD)/%.o: %.c
    $(CC) $(CPPFLAGS) $(CFLAGS) -c -o $@ $<
```

Note 19.4.1. `vpath` with a pattern is almost always preferable to `VPATH`. `VPATH` searches those directories for everything, so a stale `build/parser.o` sitting in a searched directory can satisfy a prerequisite you meant to rebuild. Being specific about which patterns come from where avoids a whole class of confusing builds.

Note 19.4.2 (The directory must exist)**.** `gcc -o build/main.o` fails if `build/` does not exist, and make will not create it for you. The clean fix is an order-only prerequisite, which Section 20.3 covers.

§19.5. Automatic Dependency Generation

This section fixes the last complaint from Section 17.5, and it is the most valuable thing in the chapter.

The problem: `main.c` does `#include "parser.h"`, so editing that header must rebuild `main.o`. Make cannot know this — it does not read your source. Written by hand, the dependency is correct on the day you write it and wrong within a month.

The solution: the compiler already parses the includes, so ask it.

`-MMD`	Write a dependency file alongside each object, listing the headers it included. `-MD` also lists system headers
`-MP`	Add a dummy target for each header, so that *deleting* a header does not break the build

```
CPPFLAGS += -MMD -MP
OBJ      := $(SRC:.c=.o)
DEP      := $(OBJ:.o=.d)

app: $(OBJ)
    $(CC) $(LDFLAGS) -o $@ $^ $(LDLIBS)

clean:
    rm -f app $(OBJ) $(DEP)

-include $(DEP)          # the leading dash: do not complain on the first build
```

Compiling `main.c` now also writes `main.d`:

```
main.o: main.c parser.h util.h
parser.h:
util.h:
```

which is a Makefile fragment, and `-include` pulls it back in next time. The dependency graph now maintains itself.

Note 19.5.1. The leading dash in `-include` means "do not fail if these do not exist". On a clean tree there are no `.d` files yet, and without the dash the very first build stops with an error. The empty `parser.h:` rules come from `-MP` and exist so that a header you delete is treated as a target with no recipe rather than a missing prerequisite.

Three lines — `-MMD -MP`, a `DEP` list, and `-include` — and you never write a header dependency again. Any C or C++ Makefile without them is one refactor away from a build that is quietly wrong.

Try it

Add the three lines to the Makefile from Section 19.2, build, then `cat main.d`. Now add an `#include` to `main.c`, rebuild, and look again — the new header is there, and touching it rebuilds `main.o`. Nothing in the Makefile mentions it.

20. Phony Targets and Project Conventions

Not every target is a file. `make clean`, `make test` and `make install` name *actions*, and make's file-and-timestamp model needs to be told so.

§20.1. `.PHONY` and Why It Matters

```
clean:
    rm -f app *.o
```

This works until the day someone creates a file called `clean`:

```
$ touch clean
$ make clean
make: 'clean' is up to date.
```

Make did exactly what it promised. The target `clean` exists, it has no prerequisites, so nothing is out of date and the recipe does not run. The fix is to declare that this target is not a file:

```
.PHONY: all clean install test

clean:
    rm -f app *.o
```

Definition 20.1.1 (Phony target)**.** A target listed as a prerequisite of the special target `.PHONY`. Make then never checks the filesystem for it: the recipe runs unconditionally, and no timestamp comparison is made.

Note 20.1.2. The stray file is the story everyone tells, and it is rare. The reason to use `.PHONY` every time is the second, mundane one: it is faster — make skips the implicit-rule search entirely for phony targets — and it documents intent. A reader can tell at a glance which of your targets produce files.

Every target that does not name a file it creates goes in `.PHONY`. Declare it once, near the bottom, listing all of them.

§20.2. The Standard Target Names

These names are close to universal. Using them means anyone can build your project without reading the Makefile.

`all`	The default build. Should be the first rule (Section 17.3)
`clean`	Remove everything the build produced
`distclean`	… and the configuration too, back to a fresh checkout
`install`	Copy the results into `$(PREFIX)`
`uninstall`	Remove them again
`test`, `check`	Run the test suite
`dist`	Build a release tarball
`help`	Print the available targets

```
PREFIX ?= /usr/local
BINDIR := $(PREFIX)/bin

install: app
    install -d $(DESTDIR)$(BINDIR)
    install -m 755 app $(DESTDIR)$(BINDIR)/app

uninstall:
    rm -f $(DESTDIR)$(BINDIR)/app

test: app
    ./run-tests.sh

clean:
    rm -f app $(OBJ) $(DEP)

.PHONY: all install uninstall test clean
```

Note 20.2.1. `DESTDIR` is prepended to every installed path and is normally empty. Distribution packagers set it to a staging directory, so that `make install DESTDIR=/tmp/pkg` lays out a complete tree under `/tmp/pkg` without touching the real `/usr/local`. It costs nothing to support and packagers will thank you.

Example 20.2.2 (A self-documenting `help`)**.** A pleasant trick: annotate targets with a marked comment and let `awk` (Section 11.8) extract them.

```
help:  ## show this message
    @awk -F':.*##' '/^[a-z-]+:.*##/ {printf "  %-12s %s\n", $$1, $$2}' \
        $(MAKEFILE_LIST)

build: ## compile everything
test:  ## run the test suite
```

Note `$$1`: make expands `$` first, so the `awk` program needs its dollars doubled to survive — exactly the hazard named in Section 16.1.

§20.3. Order-Only Prerequisites and Directories

Section 19.4 left a problem: `build/` must exist before anything is written into it, but it must not trigger rebuilds.

The naive fix does not work:

```
$(BUILD)/%.o: %.c $(BUILD)
    $(CC) -c -o $@ $<
```

A directory's timestamp changes every time a file is written into it. So the directory becomes newer than the objects, and every object rebuilds on every run.

Definition 20.3.1 (Order-only prerequisite)**.** A prerequisite written after a `|` in the prerequisite list. It must be built before the target, but its timestamp is ignored when deciding whether the target is out of date.

```
$(BUILD)/%.o: %.c | $(BUILD)
    $(CC) $(CPPFLAGS) $(CFLAGS) -c -o $@ $<

$(BUILD):
    mkdir -p $@
```

Note 20.3.2. Read the `|` as "and, before any of that, make sure this exists". Creating output directories is the overwhelmingly common use; the other is a code generator that must run first but whose own timestamp should not force everything downstream to rebuild.

§20.4. Recursive Make, and Its Problems

A large project is often split into a Makefile per directory, each invoking make in its subdirectories:

```
SUBDIRS := src lib tests

all:
    for d in $(SUBDIRS); do $(MAKE) -C $$d || exit 1; done
```

Note 20.4.1. `$(MAKE)` rather than `make`: it carries the parent's flags through (including `-j` and `-n`) and lets make recognise the sub-build as its own. `-C dir` changes directory first. Writing `make` literally here breaks parallel builds and `make -n` alike.

This arrangement is common, and it is a known mistake. Peter Miller's 1997 paper *Recursive Make Considered Harmful* [Mil98] sets out why, and the argument has not weakened:

(i) **No sub-make sees the whole graph.** Each knows only its own directory, so a dependency crossing a directory boundary cannot be expressed — only guessed at by ordering the subdirectories by hand.

(ii) **The order is a lie.** `src` before `tests` is a hand-maintained approximation of a graph make could have computed exactly.

(iii) **Parallelism is crippled.** `make -j8` cannot overlap two subdirectories that a shell `for` loop runs in sequence, so the build is as slow as its longest chain.

(iv) **Builds can be wrong, not merely slow.** A missing cross-directory edge means make declares success with a stale object linked in — the worst failure a build system has.

Prefer one Makefile that `includes` fragments from each directory over one Makefile per directory that calls make. Each fragment contributes its rules to a single graph, make sees all of it, and `-j` works properly.

```
# top-level Makefile: one graph, several files
include src/module.mk
include lib/module.mk
include tests/module.mk

all: $(ALL_TARGETS)
```

Remark 20.4.2. This is also roughly what CMake and Ninja do: the description is written per directory, and the graph that gets executed is a single flat one. The idea did win — it just stopped being spelled `make -C`.

21. Running and Debugging Make

A Makefile that works is only half the job. The other half is knowing what make is doing when it does something you did not expect — which, given that its entire reasoning is invisible, needs tooling.

§21.1. Command-Line Options

`make`	Build the default goal (Section 17.3)
`make target`	Build one target
`make -j8`	Run up to eight recipes at once
`make -n`	**Dry run**: print the recipes, run none
`make -B`	Rebuild everything, ignoring timestamps
`make -k`	Keep going after an error, building what still can be
`make -C dir`	Change to `dir` first
`make -f build.mk`	Use a file not named `Makefile`
`make -s`	Silent: do not echo recipes
`make -r`	Drop the built-in rule database
`make -t`	**Touch** targets instead of building them
`make -q`	Ask only whether the target is up to date; say so in the exit status

`make -n` first, whenever a Makefile is unfamiliar or you are about to run a target called something like `deploy`. It prints exactly what would run and touches nothing.

Note 21.1.1. `make -B` is the blunt instrument for "it should have rebuilt and it did not". It is a diagnostic, not a fix: if `-B` produces a working binary and a plain `make` does not, you have a missing edge in the graph, and Section 19.5 is where to look.

§21.2. Parallel Builds

`make -j` is the reason to have written a correct dependency graph. Make can run any two recipes simultaneously when neither depends on the other, and on a modern machine that is most of them.

```
$ time make
real    0m48.2s
$ make clean && time make -j8
real    0m9.7s
```

`make -j`	Unlimited jobs. Rarely what you want — it will start hundreds
`make -j8`	Eight at a time
`make -j$(nproc)`	One per core
`make -l4`	Start new jobs only while the load average is below 4
`make -O`	Group each job's output, so the log stays readable

Parallel make is where an incomplete dependency graph finally shows itself. A serial build hides a missing edge, because the accidental order happens to be right; with `-j8` the order changes every run, and the build fails intermittently. *A Makefile that only works without `-j` is not working.*

Example 21.2.1 (The classic race)**.**

```
all: parser.o lexer.o        # both need parser.h, which is generated

parser.h parser.c: grammar.y
    bison -d -o parser.c $<
```

Serially, `parser.o` is built first and generates the header before `lexer.o` needs it. In parallel both start at once and one of them fails — sometimes. The fix is to state the edge that was always true:

```
lexer.o parser.o: parser.h
```

Note 21.2.2. Two recipes writing the same file is the other common parallel bug, and `-j1` will never reveal it. If a build fails only occasionally under `-j`, do not retry until it passes — the graph is wrong, and the same gap will eventually produce a silently stale binary.

§21.3. Printing from Inside a Makefile

`$(info ...)`	Print a message while reading the file; expansion continues
`$(warning ...)`	The same, prefixed with the file and line number
`$(error ...)`	Print and stop immediately

```
$(info SRC is [$(SRC)])
$(info OBJ is [$(OBJ)])

ifndef PREFIX
$(error PREFIX is not set -- try `make PREFIX=/usr/local')
endif
```

Note 21.3.1. These run at *read* time, not when a recipe runs, so they fire even for targets you did not ask for. That is what makes them the right tool for inspecting a variable: the square brackets in `[$(SRC)]` are there so you can see leading and trailing whitespace, which is the usual reason a list is not what you expected.

Example 21.3.2 (A general variable-printing rule)**.**

```
print-%:
    @echo '$* = [$($*)]'
```

```
$ make print-CFLAGS
CFLAGS = [-Wall -Wextra -std=c17 -g]
$ make print-OBJ
OBJ = [main.o parser.o util.o]
```

Worth keeping in every Makefile you maintain.

§21.4. Inspecting the Database

`make -p`	Every variable, rule and built-in, with their origins
`make -p | less`	… which is several thousand lines, so page it
`make -d`	Full decision trace: why each target was or was not rebuilt
`make -debug=b`	Basic tracing only — much more readable than `-d`
`make -debug=v`	Verbose: which files were considered
`make -trace`	Print each recipe with the line that caused it
`make -n -p`	Inspect without building anything

Example 21.4.1. `make -debug=b` answers the question that matters most often:

```
$ make --debug=b app
Considering target file 'main.o'.
 Prerequisite 'parser.h' is newer than target 'main.o'.
Must remake target 'main.o'.
```

or, in the case you are usually investigating:

```
Considering target file 'main.o'.
 Prerequisite 'main.c' is older than target 'main.o'.
No need to remake target 'main.o'.
```

The second output, when you were expecting a rebuild, says the graph is missing an edge rather than that make is broken.

Note 21.4.2. `make -trace` is the gentler everyday tool: it annotates each recipe with the Makefile line and the target that triggered it, which is usually enough to find the rule responsible without wading through `-d`.

§21.5. Error Messages You Will Meet

`missing separator`	A recipe line begins with spaces instead of a tab (Section 17.2)
`No rule to make target 'x'`	`x` is needed, does not exist, and no rule matches. Usually a misspelt filename
`nothing to be done for 'all'`	The target exists and is up to date, but its recipe is empty — often a missing tab again, or a phony target you forgot to declare
`'x' is up to date`	Make is right and you meant a phony target (Section 20.1)
`Recursive variable references itself`	`=` where you wanted `:=` (Section 18.1)
`commands commence before first target`	A tab-indented line before any rule — usually a continuation that lost its backslash
`Circular x <- y dependency dropped`	The graph has a cycle; make breaks it and carries on, so the result is unpredictable
`Error 1 (ignored)`	A recipe failed on a line prefixed with `-`
`warning: overriding recipe for target 'x'`	Two rules give `x` a recipe; the later one wins and the earlier is discarded

Note 21.5.1. `make: *** [Makefile:12: app] Error 1` is not a make error at all — it is make reporting that the command on line 12 exited non-zero. The real message is above it, from the compiler. Scroll up.

Nearly every confusing make session resolves to one of three questions: is it a tab? is the target phony? and is the edge actually in the graph? `cat -A`, `.PHONY`, and `make -debug=b` answer them in that order.

22. Worked Makefiles

Three Makefiles worth stealing. Each uses only what Part III has covered, and each is short enough to read in one sitting.

§22.1. A Multi-File C Project

Sources in `src/`, headers in `include/`, everything generated under `build/`, header dependencies maintained by the compiler.

```
# ---- configuration (override any of these on the command line) -----------
CC       ?= gcc
CPPFLAGS ?= -Iinclude -MMD -MP
CFLAGS   ?= -Wall -Wextra -Wpedantic -std=c17 -O2 -g
LDFLAGS  ?=
LDLIBS   ?= -lm
PREFIX   ?= /usr/local

TARGET   := app
BUILD    := build
SRC      := $(wildcard src/*.c)
OBJ      := $(patsubst src/%.c,$(BUILD)/%.o,$(SRC))
DEP      := $(OBJ:.o=.d)

# ---- build ---------------------------------------------------------------
all: $(TARGET)

$(TARGET): $(OBJ)
    $(CC) $(LDFLAGS) -o $@ $^ $(LDLIBS)

$(BUILD)/%.o: src/%.c | $(BUILD)
    $(CC) $(CPPFLAGS) $(CFLAGS) -c -o $@ $<

$(BUILD):
    mkdir -p $@

# ---- variants ------------------------------------------------------------
debug: CFLAGS := -Wall -Wextra -std=c17 -O0 -g -fsanitize=address,undefined
debug: LDFLAGS += -fsanitize=address,undefined
debug: clean all

# ---- housekeeping --------------------------------------------------------
run: $(TARGET)
    ./$(TARGET)

test: $(TARGET)
    ./tests/run.sh

install: $(TARGET)
    install -d $(DESTDIR)$(PREFIX)/bin
    install -m 755 $(TARGET) $(DESTDIR)$(PREFIX)/bin/$(TARGET)

clean:
    rm -rf $(BUILD) $(TARGET)

print-%:
    @echo '$* = [$($*)]'

-include $(DEP)

.PHONY: all debug run test install clean
```

Note 22.1.1. Every idea in Part III is here: `?=` for configurability (Section 18.2), automatic variables (Section 18.3), a pattern rule (Section 19.2), an order-only prerequisite for the output directory (Section 20.3), generated dependencies (Section 19.5) and `.PHONY` (Section 20.1). It works unchanged for one source file or fifty, and it is correct under `make -j`.

Note 22.1.2. `debug` depends on `clean` deliberately. As Section 18.6 warned, changing a flag changes no timestamp, so without the `clean` you would link sanitiser-instrumented objects against ordinary ones. The alternative — a separate output directory per variant, as in Section 19.3 — avoids the rebuild but costs more Makefile.

§22.2. A LaTeX Document

LaTeX is a bad fit for make's model. The number of runs needed is not known in advance: a document with cross-references needs at least two passes, three if the table of contents shifts a page, plus `biber` somewhere in the middle. Make cannot express "repeat until the `.aux` file stops changing".

`latexmk` can, so the honest Makefile delegates to it:

```
MAIN  := CSTools
ENGINE := -xelatex
FLAGS := -interaction=nonstopmode -synctex=1 -output-directory=build

all: $(MAIN).pdf

$(MAIN).pdf: $(wildcard *.tex) $(wildcard Chapters/*.tex) reference.bib
    latexmk $(ENGINE) $(FLAGS) $(MAIN).tex

watch:
    latexmk $(ENGINE) $(FLAGS) -pvc $(MAIN).tex

clean:
    latexmk -C -output-directory=build $(MAIN).tex

.PHONY: all watch clean
```

Note 22.2.1. The prerequisites are still worth listing, even though `latexmk` does its own dependency tracking: they let `make` skip the whole thing when nothing has changed, and they make `make -n` honest about whether a build is needed.

When a tool already solves the recurrence problem — `latexmk`, `cargo`, `go build` — make's job is to call it, not to reimplement it. A Makefile that runs `xelatex` three times and hopes is worse than the one line it replaced.

Remark 22.2.2. The one place a Makefile earns its keep in a LaTeX project is generated figures — diagrams compiled from source, plots produced by a script, images converted between formats. Those *are* a timestamp problem, and Section 22.3 is the shape to use.

§22.3. A Small Data Pipeline

Nothing about make is specific to compilers. Any process that turns files into other files fits, and a pipeline is where the incremental rebuild is most valuable — the steps are slow and you will run them repeatedly.

```
RAW   := $(wildcard data/raw/*.csv)
CLEAN := $(patsubst data/raw/%.csv,data/clean/%.csv,$(RAW))
FIGS  := $(patsubst data/clean/%.csv,figures/%.png,$(CLEAN))

all: report.pdf

# stage 1: clean each raw file
data/clean/%.csv: data/raw/%.csv scripts/clean.py | data/clean
    python3 scripts/clean.py $< $@

# stage 2: one figure per cleaned file
figures/%.png: data/clean/%.csv scripts/plot.py | figures
    python3 scripts/plot.py $< $@

# stage 3: everything feeds the report
report.pdf: report.tex $(FIGS) summary.csv
    latexmk -xelatex -interaction=nonstopmode report.tex

summary.csv: $(CLEAN) scripts/summarise.py
    python3 scripts/summarise.py $(CLEAN) > $@

data/clean figures:
    mkdir -p $@

clean:
    rm -rf data/clean figures summary.csv report.pdf

.PHONY: all clean
```

Note 22.3.1. Notice that `scripts/clean.py` is a prerequisite of the files it produces. Editing the script rebuilds everything downstream of it, which is exactly right and is the step people leave out. Without that edge, fixing a bug in the cleaning code leaves the old, wrong output in place and make reports success.

Example 22.3.2. Adding one raw file to `data/raw/` and running `make` cleans it, plots it, regenerates `summary.csv` — because the summary depends on all the cleaned files — and rebuilds the report. Nothing else runs. With `make -j4`, the per-file stages overlap.

If you find yourself keeping a note of which script to re-run after which change, you have a dependency graph in your head. Write it down as a Makefile and stop remembering it.

Try it

Take any multi-step process you currently run by hand — a build, a document, a data set — and write the graph before writing any rules. Name the files at each stage and draw the arrows between them, as on page 90. The Makefile is then a transcription, and it will take about ten minutes.

Part IV.

GDB

Almost everyone debugs by inserting print statements. It works, it is universal, and it is also the reason a great many people spend an afternoon on a bug that a debugger would have located in ninety seconds. The cost of `printf` debugging is invisible precisely because you never see the faster path.

GDB lets you stop a running program at any point and ask it questions: what is this variable, who called this function, what is at this address, what changed this value. Chapter 23 fixes the notation and gets you to a prompt; chapters 24 and 25 are about stopping in the right place, which on a real program is most of the skill; chapters 26 and 27 are what to ask once you are there. Chapter 28 then works through the failures you will actually meet — segfaults, corrupted memory, deadlocks, and code that misbehaves only when optimised — including the ones for which the right answer is a different tool entirely.

23. Preparing to Debug

Almost everyone debugs by inserting print statements. It is universal, it works, and it is also why a great many people spend an afternoon on a bug that a debugger would have located in ninety seconds. The cost is invisible precisely because you never see the faster path.

This chapter is about getting into position: what to compile, how to start, and how to ask GDB for help once you are there.

§23.1. Notation for Part IV

Two kinds of transcript appear in Part IV. Shell transcripts keep the convention of Section 9.1; GDB sessions use GDB's own prompt:

```
(gdb) break main          # a line beginning "(gdb)" is what YOU type
Breakpoint 1 at 0x1149: file main.c, line 5.
(gdb) run                 # everything else is GDB's output
Starting program: /home/konyi/demo/app
```

C source appears with line numbers, so that breakpoints can refer to them:

```
1  int main(void) {
2      int *p = NULL;
3      return *p;            /* this is line 3 */
4  }
```

`(gdb)`	GDB's prompt. Never type it
`$1`, `$2`	GDB's value history — results it numbered for you
`$rip`, `$sp`	A machine register
`$myvar`	A convenience variable you created
`{n}`, `[n]`	Placeholders, as in Section 9.1

Note 23.1.1. The dollar sign means something different again here. In Part II it introduced a shell variable, in Part III a make variable, and in GDB it marks a register, a convenience variable, or an entry in the value history. Each part's tools are unrelated; the character is just popular.

§23.2. Compiling with `-g`

```
$ gcc -g -O0 -Wall -o app main.c parser.c
```

`-g`	Emit debug information: line numbers, variable names, types
`-g3`	... and macro definitions, so `print` understands them
`-O0`	Do not optimise
`-Og`	Optimise, but only in ways that keep debugging sensible

Definition 23.2.1 (Debug information)**.** A section of the binary (DWARF, on Linux) mapping machine addresses back to source: which line produced which instruction, where each variable lives, and what every type looks like. Without it GDB can still run the program and read memory, but it cannot name anything.

Without `-g`, a backtrace is addresses:

```
#0  0x0000555555555149 in ?? ()
#1  0x00007ffff7db0d90 in ?? () from /lib/x86_64-linux-gnu/libc.so.6
```

With it, the same backtrace is a story:

```
#0  parse_line (s=0x0) at parser.c:42
#1  0x000055555555520a in main (argc=2, argv=0x7fffffffe3c8) at main.c:17
```

`-g` costs nothing at run time. It makes the binary larger, and that is all — it does not slow the program down and it does not change what the code does. Compile *everything* with `-g`, including release builds, and strip the symbols later if you must ship something small.

Note 23.2.2 (Why `-O0` matters)**.** Optimisation is what makes a debugger confusing. At `-O2` the compiler reorders statements, keeps variables in registers, merges identical code paths and inlines small functions out of existence. GDB then reports things that are true but useless:

`<optimized out>`	The variable exists in the source but nowhere at run time
Stepping jumps backwards	Instructions from several lines were interleaved
A breakpoint never fires	That code was inlined, or folded into another branch
`...(inlined)` frames	The call is real in the source and absent in the binary

`-Og` is the compromise: it applies the optimisations that do not destroy debug information. Use `-O0` while hunting a bug, and `-Og` for a build you want to be both fast and debuggable.

Remark 23.2.3. Sometimes the bug only appears at `-O2`, and then you have no choice. Section 26.5 covers reading optimised frames. But check first that the bug is really the optimiser's doing — more often the optimiser has merely exposed undefined behaviour that was always there, and Section 28.6 finds it faster than GDB will.

§23.3. Debug Symbols and Stripped Binaries

`file app`	Says `with debug_info` or `stripped`
`nm app | head`	List the symbols
`objdump -h app | grep debug`	Which DWARF sections are present
`strip app`	Remove the symbol table — irreversibly
`objcopy --only-keep-debug app app.debug`	Split the debug info into its own file

```
$ file app
app: ELF 64-bit LSB pie executable, ..., with debug_info, not stripped
$ strip app && file app
app: ELF 64-bit LSB pie executable, ..., stripped
```

Note 23.3.1. Distribution binaries are stripped, which is why a backtrace through a system library shows `??`. The fix is to install the matching debug package — `libc6-dbg` on Debian and Ubuntu, or the `debuginfod` service GDB will offer to use automatically. Do it once; a backtrace that names libc's frames is worth much more than one that does not.

§23.4. Core Dumps

Definition 23.4.1 (Core dump)**.** A snapshot of a process's memory written when it crashes. Loading it into GDB gives you the state at the moment of death — the stack, the variables, the faulting instruction — without having to reproduce the crash.

```
$ ulimit -c              # 0 means cores are disabled -- the usual default
0
$ ulimit -c unlimited    # for this shell only
$ ./app
Segmentation fault (core dumped)
$ gdb ./app core
```

Note 23.4.2. `ulimit -c unlimited` affects only the current shell and its children (Section 13.7). Where the file lands is set by `/proc/sys/kernel/core_pattern`, and on a modern desktop it is usually handed to `systemd-coredump` rather than written to the current directory — in which case `coredumpctl list` finds it and `coredumpctl debug` opens it in GDB directly.

Core dumps are the answer to "it crashed once, on the server, at three in the morning". Enabling them costs nothing and turns an unreproducible report into an ordinary debugging session.

§23.5. Ways to Start GDB

`gdb ./app`	Load the program; it is not running yet
`gdb --args ./app -v in.txt`	... with the arguments it should get
`gdb ./app core`	Post-mortem, from a core dump
`gdb -p 1234`	Attach to a process that is already running
`gdb -q`	Quiet: skip the licence banner
`gdb -x script.gdb ./app`	Run a file of GDB commands first
`gdb -ex run -ex bt ./app`	Run commands given on the command line
`gdb -tui ./app`	Start with the source window open (Section 29.1)

Example 23.5.1 (The one-line crash triage).

```
$ gdb -q -batch -ex run -ex bt --args ./app broken-input.txt
Program received signal SIGSEGV, Segmentation fault.
0x0000555555555185 in parse_line (s=0x0) at parser.c:42
42          while (*s != '\n') {
#0  parse_line (s=0x0) at parser.c:42
#1  0x00005555555551f9 in main (argc=2, argv=0x7fffffffe3b8) at
```

`-batch` runs the commands and exits, so this fits in a shell script or a CI job. In four seconds you know the function, the line, and that the argument was `NULL`.

Note 23.5.2 (Attaching). `gdb -p` is how you investigate a process that is hung rather than crashed: attach, `bt`, see what it is waiting on, then `detach`. On most Linux systems `ptrace` is restricted to child processes, so attaching to an unrelated process needs `sudo`, or `sudo sysctl kernel.yama.ptrace_scope=0` to lift the restriction for the session.

§23.6. The Prompt, Abbreviations and Help

GDB accepts any unambiguous prefix of a command, and the common ones have one-letter forms that everybody uses.

`b`	`break`
`r`	`run`
`c`	`continue`
`n`	`next`
`s`	`step`
`fin`	`finish`
`p`	`print`
`bt`	`backtrace`
`i`	`info`
`l`	`list`
`q`	`quit`

`<CR>` on an empty line	Repeat the previous command. Holding it down is how you step
`<Tab>`	Complete a command, a function name or a variable
`<C-c>`	Interrupt the running program and return to the prompt
`<C-d>`	Quit
`<C-r>`	Search the command history — the Readline bindings of Section 9.7 all work
`shell ls`	Run a shell command without leaving GDB
`!ls`	The same, abbreviated

Help is structured by topic, and is better than its reputation:

`help`	The list of command classes
`help breakpoints`	Every command in one class
`help break`	One command, in detail
`apropos watch`	Search all help text
`info`	What GDB knows: `info locals`, `info breakpoints`, ...
`show`	GDB's own settings: `show args`, `show language`
`set`	Change them: `set pagination off`

Note 23.6.1. `info` and `show` are easy to confuse and the distinction is worth learning: `info` asks about *the program* — its breakpoints, its stack, its variables. `show` asks about *GDB* — its own configuration. `info args` lists the current function's arguments; `show args` lists the arguments your program was started with.

Try it

Compile anything with `gcc -g -O0`, run `gdb -q ./app`, and type `start`. You are stopped on the first line of `main` with the whole of the rest of Part IV available. Then `help running` and read the list — it is one screen, and it is most of Chapter 24.

24. Controlling Execution

A debugger is a program that runs your program slowly enough for you to look at it. This chapter is the controls: start, stop, and advance by one step of whatever size you need.

§24.1. Starting and Stopping

`run`	Start the program from the beginning
`run arg1 arg2`	... with arguments
`run < in.txt > out.txt`	... with redirection, as in Section 12.2
`start`	`run`, with a temporary breakpoint on `main`
`continue`	Resume until the next stop
`continue 5`	Resume, ignoring this breakpoint the next 4 times
`kill`	Stop the program, keep the session
`quit`	Leave GDB
`set args -v in.txt`	Set the arguments for future runs
`show args`	What they currently are

Note 24.1.1. `start` is almost always the right way in: it gets you to the first line of `main` with the program loaded and the dynamic libraries mapped, which is the point at which most things become inspectable. Plain `run` with no breakpoints set simply runs to completion.

Note 24.1.2 (The program keeps its state)**.** `run` a second time restarts from scratch; breakpoints and watchpoints survive, convenience variables survive, and the value history survives. So the loop is: set breakpoints, `run`, look, adjust, `run` again — without ever leaving GDB.

§24.2. Stepping

`next` (`n`)	Run the next source line. **Step over** calls
`step` (`s`)	Run the next source line. **Step into** calls
`finish`	Run until the current function returns; print its value
`until`	Like `next`, but do not go backwards — runs a loop to completion
`until 60`	Run until line 60 is reached, or the frame returns
`advance parse`	Run until `parse` is reached, or the frame returns
`next 10`	Ten lines at once

> **Intuition .** `next` and `step` differ only at a function call. `next` treats the call as one line and runs the whole thing; `step` stops at its first line. Use `next` by default and `step` only when you have decided that the bug is inside *this* call — otherwise you will spend the afternoon inside `printf`.

Example 24.2.1.

```
(gdb) start
Temporary breakpoint 1, main () at main.c:12
12          int n = count_words(argv[1]);
(gdb) next                          # run count_words, stop after it
13          printf("%d\n", n);
(gdb) print n
$1 = 0                              # wrong: the file has 40 words
(gdb) run                           # start again, this time go inside
(gdb) step
count_words (path=0x7fffffffe5c1 "input.txt") at parser.c:8
8           FILE *f = fopen(path, "r");
```

The first `next` establishes *that* the answer is wrong; the `step` begins finding out why. Doing it in that order saves you from stepping through functions that were never the problem.

Note 24.2.2 (`step` into a library)**.** `step` will not enter a function with no debug information — it steps over it instead, which is what you want. If you have installed libc's debug symbols and now find yourself inside `malloc`, `finish` gets you out, and `skip file malloc.c` stops it happening again.

Example 24.2.3 (`until`, and why loops are annoying)**.** Stepping with `next` inside a loop walks the body once per iteration, so a thousand-iteration loop takes a thousand presses. `until` at the closing line of the body runs the loop out:

```
(gdb) next
14              for (int i = 0; i < n; i++) {
(gdb) next
15                  total += weight[i];
(gdb) until                         # run the loop to completion
17          printf("%d\n", total);
```

The other answer is a conditional breakpoint on the iteration you care about — Section 25.2.

`<CR>` repeats the last command, so stepping is: type `n` once, then hold Enter. This is the single most-used key sequence in GDB.

§24.3. Source Lines versus Instructions

Everything above steps by *source line*. Underneath, a line is several machine instructions, and sometimes that is the level you need.

`stepi` (`si`)	One machine instruction, entering calls
`nexti` (`ni`)	One instruction, stepping over calls
`disassemble`	The current function's instructions
`disassemble /s`	… interleaved with the source
`x/5i $pc`	The next five instructions from the program counter
`info registers`	Every register
`display/i $pc`	Show the next instruction after every stop

Example 24.3.1.

```
(gdb) disassemble /s main
Dump of assembler code for function main:
main.c:
5       int main(void) {
   0x1149 <+0>:     push   %rbp
   0x114a <+1>:     mov    %rsp,%rbp
6           int *p = NULL;
   0x114d <+4>:     movq   $0x0,-0x8(%rbp)
7           return *p;
   0x1155 <+12>:    mov    -0x8(%rbp),%rax
   0x1159 <+16>:    mov    (%rax),%eax        # <-- the fault is here
```

Note 24.3.2. You need this level in exactly three situations: the code was compiled without `-g`; the bug is in inline assembly or a compiler intrinsic; or you suspect the compiler, which is almost never the case. Otherwise stay at the source level — assembly is a way of being precise about the wrong thing.

§24.4. Changing the Flow

GDB can do more than watch. These commands alter a running program, which is occasionally exactly what an experiment needs.

`return`	Abandon the current function immediately
`return 0`	… making it return that value
`jump 42`	Continue from line 42 instead of here
`call parse(s)`	Call a function in the program, right now
`print parse(s)`	The same, and show the result

Example 24.4.1. Testing a fix without recompiling. A function is returning a null pointer and you suspect the caller mishandles it:

```
(gdb) break lookup
(gdb) run
Breakpoint 1, lookup (key=0x4006f4 "port") at config.c:31
(gdb) return 0                  # force the NULL return, right now
(gdb) continue
Program received signal SIGSEGV, Segmentation fault.
0x00005555555552a1 in main () at main.c:22
22          printf("%s\n", value);
```

Confirmed in twenty seconds: the caller does not check. You have reproduced a rare failure on demand, which is usually the hard part.

Note 24.4.2. `call` and `print` of a function actually run it in the program's own process, side effects and all. That is powerful — `print dump_state()` invokes your own debug printer — and dangerous: if the function crashes, it crashes the program you were debugging. GDB will tell you and offer to unwind.

`jump` skips the stack adjustment a real control transfer would do, so jumping out of one function into another is undefined nonsense. Jump within a function, or not at all.

25. Breakpoints and Watchpoints

Stepping from `main` is fine for a twenty-line program. For anything larger the skill is stopping in the right place — and, more often, stopping on the right *iteration* of the right place.

§25.1. Setting Breakpoints

`break main`	On entry to a function
`break parser.c:42`	On a line of a file
`break 42`	Line 42 of the current file
`break *0x401136`	At a machine address
`break +2`, `break -3`	Relative to the current line
`tbreak main`	Temporary: deleted once it fires
`rbreak ^parse_`	On every function matching a regular expression
`break parse if n > 100`	Conditional (Section 25.2)

```
(gdb) break parser.c:42
Breakpoint 1 at 0x1185: file parser.c, line 42.
(gdb) run
Breakpoint 1, parse_line (s=0x5555555592a0 "name=value") at parser.c:42
42          while (*s != '\n') {
```

Note 25.1.1. A breakpoint on a *function* lands after the prologue — after the stack frame is set up — so that arguments and locals are already readable. This is what you want, and it is why the reported line is usually the first statement rather than the opening brace.

Note 25.1.2 (Breakpoints before the program exists)**.** Setting a breakpoint in a shared library that has not been loaded yet gets you `Make breakpoint pending on future shared library load? (y or [n])`. Answer yes; GDB will set it when the library arrives. `set breakpoint pending on` in your `.gdbinit` (Section 29.2) stops it asking.

§25.2. Conditions and Ignore Counts

This is the section that makes breakpoints useful on real programs.

`break parse if i == 500`	Stop only when the condition holds
`condition 2 n > 100`	Add or change a condition on breakpoint 2
`condition 2`	Remove the condition
`ignore 2 499`	Skip the next 499 hits of breakpoint 2
`break f if $_streq(s, "bad")`	String comparison, via a convenience function
`break f if p == 0`	Stop when a pointer is null

Example 25.2.1. A loop of ten thousand iterations corrupts something around the five hundredth. Stepping is hopeless; a condition is instant:

```
(gdb) break process if i == 500
(gdb) run
Breakpoint 1, process (i=500, buf=0x5555555592a0) at work.c:88
88          buf[i] = table[index];
(gdb) print index
$1 = 1048577                    # the table has 1024 entries
```

Note 25.2.2. A condition is evaluated by GDB every time the breakpoint is hit, which means the program stops, GDB evaluates, and the program resumes — thousands of times. On a hot path this is genuinely slow. `ignore` is much faster when you simply want the nth hit, because GDB only counts.

When a bug happens on "some later iteration", do not step. Put a condition on the loop variable, or an `ignore` count on the breakpoint, and arrive at the failure directly.

§25.3. Listing, Disabling and Deleting

`info breakpoints`	List them all, with hit counts
`disable 2`	Keep it, stop using it
`enable 2`	Turn it back on
`enable once 2`	Fire once more, then disable itself
`delete 2`	Remove it
`delete`	Remove all of them, without asking
`clear parser.c:42`	Delete by location rather than number

```
(gdb) info breakpoints
Num     Type           Disp Enb Address            What
1       breakpoint     keep y   0x0000000000001185 in parse_line at parser.c:42
        stop only if n > 100
        breakpoint already hit 3 times
2       hw watchpoint  keep y                      total
3       breakpoint     del  y   0x0000000000001149 in main at main.c:5
```

Note 25.3.1. Prefer `disable` to `delete` while you are working. Numbers are never reused within a session, so a deleted breakpoint you want back has to be retyped, whereas a disabled one is `enable 2`. `info breakpoints` also shows hit counts, which answers "did that code run at all?" without any stepping.

§25.4. Watchpoints

Definition 25.4.1 (Watchpoint)**.** A breakpoint on *data* rather than on code: it stops the program whenever a given expression changes value, whatever line did it.

`watch total`	Stop when `total` changes
`watch *ptr`	Stop when the pointed-to memory changes
`watch -l total`	Watch the *location*, not the expression
`rwatch total`	Stop when it is read
`awatch total`	Stop on read or write
`watch a[i] if i > 5`	Conditional, like a breakpoint
`info watchpoints`	List them

```
(gdb) watch config.timeout
Hardware watchpoint 2: config.timeout
(gdb) continue
Hardware watchpoint 2: config.timeout
Old value = 30
New value = 0
reset_defaults (c=0x555555558040) at config.c:71
71          c->timeout = 0;
```

"This value is wrong and I have no idea who wrote it" is precisely what a watchpoint answers, and it answers it in one command. It is the most underused feature in GDB.

Note 25.4.2 (Hardware and software watchpoints)**.** Modern processors have a handful of debug registers — typically four — that watch an address in hardware at full speed. GDB uses them when it can, and says `Hardware watchpoint`. Exceed the supply, or watch something too large, and you get a *software* watchpoint: GDB single-steps the entire program and re-evaluates after every instruction. That is correct and perhaps a thousand times slower, so it is worth noticing which kind you got.

Note 25.4.3 (Scope)**.** A watchpoint on a local variable dies with its frame, and GDB tells you: `Watchpoint 2 deleted because the program has left the block`. To follow a specific object rather than a name, watch its address: `watch -l *&buf[3]`.

§25.5. Catchpoints

`catch throw`	Every C++ exception, where it is thrown
`catch catch`	… where it is caught
`catch syscall write`	A particular system call
`catch signal SIGPIPE`	A signal (Section 13.3)
`catch exec`, `catch fork`	Process creation
`catch load libfoo.so`	A shared library being loaded

Example 25.5.1. `catch throw` is the standard way to debug a C++ exception whose origin is unknown: the stack at the `throw` is intact, while by the time it reaches your handler the frames in between are gone. Similarly, `catch syscall openat` finds which file a program is failing to open without any `strace`.

§25.6. Breakpoint Command Lists

A breakpoint can carry a script, which turns GDB into an instrumentation tool that needs no recompilation.

```
(gdb) break parse_line
(gdb) commands
Type commands for breakpoint(s) 1, one per line.
End with a line saying just "end".
>silent
>printf "parse_line(%s) depth=%d\n", s, depth
>continue
>end
(gdb) run
parse_line(name=value) depth=0
parse_line(host=local) depth=1
parse_line((null)) depth=2
Program received signal SIGSEGV, Segmentation fault.
```

`silent`	Suppress GDB's usual "Breakpoint 1, ..." announcement
`printf`	C-style formatting, with the program's own variables
`continue`	Do not stop; carry straight on
`bt 3`	Print the top three frames and carry on
`end`	Finish the list

This is print-statement debugging without the recompile, and without leaving the statements in your source afterwards. When the edit–compile–run cycle is slow, or the program is already running in production, it is the fastest tool you have.

Try it

Take any program with a recursive function. Put a breakpoint on it with a command list of `silent`, a `printf` of its argument, and `continue`. You now have a call trace, produced without touching the source, and the last line before the crash is where to look.

26. The Stack and the Source

You are stopped. The first question is always the same: how did I get here?

§26.1. Reading a Backtrace

Definition 26.1.1 (Call stack)**.** The chain of function calls currently in progress. Each entry is a **frame**, holding that call's arguments, its local variables, and the address to return to. Frame `#0` is where the program is now; the numbers increase towards `main`.

`backtrace` (`bt`)	Print the whole stack
`bt 5`	The innermost five frames
`bt -5`	The outermost five
`bt full`	… with every frame's local variables
`bt -frame-arguments all`	Print full argument values, not summaries
`where`	A synonym for `bt`

```
(gdb) bt
#0  0x00005555555551a9 in parse_line (s=0x0) at parser.c:42
#1  0x0000555555555241 in read_config (path=0x7fffffffe5c1 "app.conf")
    at config.c:18
#2  0x00005555555552c8 in main (argc=2, argv=0x7fffffffe3b8) at main.c:11
```

Read it from the bottom. `main` called `read_config` with a filename; that called `parse_line` with `s=0x0`; and `0x0` is `NULL`, which is the bug. The arguments printed in each frame are usually enough to locate the fault without looking at a single line of source.

`bt` is the first command after any crash, and often the last. The frame that is *wrong* is rarely frame `#0` — frame `#0` is merely where the wrongness became fatal. Look for the frame that passed the bad value, which is usually one or two up.

Note 26.1.2 (Frames you cannot read)**.** `??` in a backtrace means no debug information for that frame — a stripped library (Section 23.3), or a corrupted stack. If *every* frame is `??` and the addresses look implausible, the stack itself has been overwritten, and Section 28.2 is the relevant section.

§26.2. Selecting Frames

The **selected frame** is the one that `print`, `info locals` and `list` refer to. Moving between frames is how you inspect the caller's variables.

`frame 2` (`f 2`)	Select frame 2
`up`	One frame outwards, towards `main`
`down`	One frame inwards
`up 3`	Three at once
`frame`	Which frame am I in?
`info frame`	Its addresses, saved registers, return address

Example 26.2.1.

```
(gdb) frame 1
#1  0x0000555555555241 in read_config (path=0x7fffffffe5c1 "app.conf")
    at config.c:18
18          parse_line(next_line(f));
(gdb) print f
$1 = (FILE *) 0x0
```

One `frame 1` and the real bug appears: `fopen` failed, nobody checked, and `next_line` was handed a null `FILE *`. The crash was two frames and one function away from its cause.

Note 26.2.2. Selecting a frame changes nothing about the program — it only changes which frame's names GDB resolves. `up` and `down` are safe to use freely, and `frame 0` returns you to the innermost.

§26.3. Viewing Source

`list` (`l`)	Ten lines around the current one
`list`	Again: the next ten
`list 40,60`	An explicit range
`list parse_line`	Around a function
`list parser.c:42`	Around a line of a file
`list -`	The ten lines before the last listing
`set listsize 20`	Show more at a time
`directory ../src`	Where to look for sources GDB cannot find
`info source`	Which file the current frame came from

Note 26.3.1. `No such file or directory` when listing means the binary records a path that no longer exists — you moved the tree, or built it in a container. `directory` adds a search path, and `set substitute-path /build /home/konyi/src` rewrites a prefix, which is the usual fix for a binary built elsewhere.

If you find yourself typing `list` more than twice, open the TUI instead (Section 29.1). `<C-x>` `a` turns on a source window that follows the program automatically, and the `list` command becomes unnecessary.

§26.4. Locals and Arguments

`info locals`	Every local variable in the selected frame
`info args`	Its arguments
`info all-registers`	Every register, including floating point
`bt full`	`info locals` for every frame at once
`info functions parse`	Functions whose names match
`info variables`	Global and static variables
`info scope parse_line`	What is in scope there, and where each lives

```
(gdb) info args
s = 0x0
len = 128
(gdb) info locals
i = 0
buf = "\000\000\000..."
p = 0x7fffffffe2a0 "name=value"
```

Note 26.4.1. `bt full` is the single most informative command for a crash you cannot reproduce: it captures every frame with every local in one block of text, which is exactly what to paste into a bug report. Pair it with `set pagination off` so it does not stop every screenful.

§26.5. Inlined and Optimised Frames

At `-O2` the stack GDB shows and the stack the machine has are no longer the same thing, and it is worth knowing how the difference presents itself.

`<optimized out>`	The variable has no storage at this point. It may be in a register that has since been reused, or eliminated entirely
`... (inlined)`	GDB reconstructed this frame from debug info; no real call happened
A missing frame	A tail call: the compiler reused the caller's frame
A line that "executes twice"	Instructions from several source lines interleaved

```
(gdb) bt
#0  hash (key=<optimized out>) at hash.c:12
#1  0x000055555555529a in lookup (t=0x5555555592a0, key=0x4006f4 "port")
    at table.c:30
#2  table_get (t=0x5555555592a0, key=0x4006f4 "port") at table.c:44
```

Note that frame `#1` and `#2` share an address: `lookup` was inlined into `table_get`, and GDB is showing you the source-level call that the machine no longer makes.

`info frame`	Is this a real frame or a reconstructed one?
`set print frame-info source-and-location`	More detail on each frame
`p $rdi`, `p $rsi`	Read arguments from registers when names fail
`disassemble /s`	What is actually executing

Rebuild at `-Og` or `-O0` before fighting an optimised backtrace. If the bug then disappears, that is not a dead end — it is strong evidence of undefined behaviour, and Section 28.6 will find it far faster than reading disassembly would.

27. Inspecting and Changing Data

You are stopped in the right frame. Now the actual work: finding out what the program thinks is true.

§27.1. `print` and Format Letters

`print` evaluates an expression in the language of the current frame — real C, with your program's variables, operators and functions.

`print n`	A variable
`print a[i]`	An array element
`print c->next->name`	A chain of dereferences
`print n * 2 + 1`	Arithmetic
`print s == NULL`	A comparison: prints 1 or 0
`print sizeof(struct node)`	Ask the type system
`print func(3)`	Call it (Section 24.4)

A slash and a letter changes the representation:

`/x`	Hexadecimal
`/d`	Signed decimal
`/u`	Unsigned decimal
`/o`	Octal
`/t`	Binary — for flags and masks
`/c`	As a character
`/f`	As a float
`/s`	As a string
`/a`	As an address, with the nearest symbol

```
(gdb) print flags
$1 = 37
(gdb) print/x flags
$2 = 0x25
(gdb) print/t flags
$3 = 100101
(gdb) print/a main
$4 = 0x1149 <main>
```

Note 27.1.1. `print/t` is the one to remember. Bit flags read as decimal are meaningless; in binary the answer is usually visible at a glance.

Several settings make output readable, and belong in your `.gdbinit`:

`set print pretty on`	One struct member per line, indented
`set print array on`	Arrays one element per line
`set print elements 0`	Do not truncate long arrays or strings
`set print null-stop on`	Stop printing a char array at the first NUL
`set print union off`	Do not expand unions inside structs
`set pagination off`	Do not stop every screenful

```
(gdb) set print pretty on
(gdb) print *cfg
$5 = {
  name = 0x555555556004 "server",
  port = 8080,
  timeout = 30,
  next = 0x0
}
```

§27.2. `x`: Examining Memory

`print` shows a *value*; `x` shows the *bytes at an address*, which is what you need when the value is not what its type claims.

```
x/{count}{format}{size} {address}
```

`x/16xb buf`	16 bytes, in hex
`x/8xw buf`	8 words (4 bytes each), in hex
`x/4xg $sp`	4 giants (8 bytes each) from the stack pointer
`x/s s`	As a NUL-terminated string
`x/10c buf`	10 characters
`x/5i $pc`	5 instructions
`x/20dw arr`	20 words as signed decimal

Sizes are `b`yte, `h`alfword (2), `w`ord (4) and `g`iant (8).

```
(gdb) x/16xb buf
0x7fffffffe2a0: 0x68 0x65 0x6c 0x6c 0x6f 0x00 0xff 0xff
0x7fffffffe2a8: 0xff 0xff 0xff 0xff 0x00 0x00 0x00 0x00
(gdb) x/s buf
0x7fffffffe2a0: "hello"
```

Note 27.2.1. `x` remembers its format, so a bare `x` after the above shows the next 16 bytes in the same layout — and `<CR>` repeats it. Walking a buffer is one command and then the Enter key.

Reach for `x` when you distrust the type. A `char *` that prints as garbage, a struct whose fields look implausible, a buffer you suspect was overrun — `x/32xb` shows what is really there, with no interpretation layered on top.

§27.3. Types

`ptype cfg`	The full definition of the variable's type
`ptype struct node`	... of a named type
`whatis cfg`	Just the type name, without expanding it
`ptype/o struct node`	With byte offsets and sizes — shows the padding
`print sizeof(cfg)`	Its size
`info types node`	Which types match a pattern

```
(gdb) ptype/o struct node
/* offset      |    size */  type = struct node {
/*      0      |       8 */    char *name;
/*      8      |       4 */    int port;
/* XXX  4-byte hole      */
/*     16      |       8 */    struct node *next;
                               /* total size (bytes):   24 */
                             }
```

Note 27.3.1. `ptype/o` is how you see padding, which matters when a struct is written to a file or sent over a socket. The four-byte hole above is why `sizeof(struct node)` is 24 and not 20 — and why writing the struct verbatim produces a format that differs between compilers.

§27.4. display

`display i`	Print `i` automatically at every stop
`display/x flags`	... with a format
`display/i $pc`	The next instruction, every time
`info display`	What is being displayed
`undisplay 1`	Stop displaying it

```
(gdb) display i
1: i = 0
(gdb) display total
2: total = 0
(gdb) next
15              total += weight[i];
1: i = 0
2: total = 0
(gdb) next
14          for (int i = 0; i < n; i++) {
1: i = 0
2: total = 12
```

Two or three `displays` turn stepping into a live view of the variables you care about. Set them once, then hold `<CR>` and watch — it is the closest GDB gets to a graphical debugger's watch pane, and it is faster to set up.

§27.5. Changing Variables and Registers

`set var n = 42`	Assign to a variable
`set var p = 0`	Make a pointer null, to test the error path
`set var s[0] = 'X'`	Modify one element
`set var $rax = 1`	Assign to a register
`print n = 42`	Assigns too, and shows the result

Note 27.5.1. Use `set var`, not bare `set`. `set width = 80` is GDB's own `set width` setting, not an assignment to your variable named `width`; `set var width = 80` is unambiguous. This bites people roughly once, memorably.

Example 27.5.2. Testing an error path that is hard to trigger for real:

```
(gdb) break write_output
(gdb) run
Breakpoint 1, write_output (f=0x5555555592a0, n=1024) at io.c:55
(gdb) set var f = 0            # pretend the file failed to open
(gdb) continue
error: cannot write output     # good -- the check works
```

§27.6. Pointers, Arrays and Structs

`print *p`	Dereference
`print p->field`	A member through a pointer
`print *arr@10`	Ten elements starting at `arr`
`print *p@n`	`n` elements — `n` may be a variable
`print arr[3]@4`	Elements 3 to 6
`print (struct node *)ptr`	Cast, when the type was lost
`print {int}0x7fff2a0`	Read that address as an `int`
`print *node->next->next`	Follow a list by hand

Definition 27.6.1 (The `@` operator)**.** `expr@n` means "treat this as the first of `n` consecutive elements". It exists because a pointer in C carries no length, so GDB cannot know how much of a dynamically allocated array to print.

```
(gdb) print arr
$1 = (int *) 0x5555555592a0        # just an address: no length information
(gdb) print *arr@5
$2 = {3, 1, 4, 1, 5}
(gdb) print *arr@n                 # n is the program's own length variable
$3 = {3, 1, 4, 1, 5, 9, 2, 6}
```

`*p@n` is the command that makes GDB usable on C. Without it every heap array prints as a bare address, and half the state of your program is invisible.

§27.7. Convenience Variables and Value History

`$1`, `$2`	Earlier results. Every `print` numbers its output
`$`	The most recent value
`$$`	The one before that
`$$3`	Three back
`set $i = 0`	A convenience variable — yours, not the program's
`set $p = head`	Useful for walking a data structure
`show values`	The last ten history entries

Example 27.7.1 (Walking a linked list).

```
(gdb) set $n = head
(gdb) print $n->name
$1 = 0x555555556004 "first"
(gdb) set $n = $n->next
(gdb) print $n->name
$2 = 0x55555555600a "second"
```

Now the last two lines repeat with `<CR>` — two keystrokes per node. For a longer list, a loop does it in one command:

```
(gdb) set $n = head
(gdb) while $n != 0
 >printf "%s -> ", $n->name
 >set $n = $n->next
 >end
first -> second -> third ->
```

Note 27.7.2. Convenience variables are typeless and persist across `run`, which makes them the right place to stash an address you want to keep watching across several runs of the program. They never collide with your program's own names, because your program cannot have a variable beginning with `$`.

28. Debugging Real Failures

The previous chapters were the instrument panel. This one is the flying: the handful of failures you will actually meet, and the shortest route through each.

§28.1. Segmentation Faults

Definition 28.1.1 (Segmentation fault)**.** The kernel's response to a memory access the process is not allowed to make: dereferencing a null or wild pointer, running off the end of the stack, or writing to read-only memory. The signal is `SIGSEGV` (Section 13.3).

The procedure is mechanical:

```
(gdb) run
Program received signal SIGSEGV, Segmentation fault.
0x00005555555551a9 in parse_line (s=0x0) at parser.c:42
42          while (*s != '\n') {
(gdb) bt                          # 1. how did I get here?
#0  parse_line (s=0x0) at parser.c:42
#1  0x0000555555555241 in read_config (path=0x7fff...) at config.c:18
#2  0x00005555555552c8 in main (argc=2, argv=0x7fff...) at main.c:11
(gdb) info args                   # 2. what was I given?
s = 0x0
(gdb) frame 1                     # 3. who gave it to me?
#1  read_config (path=0x7fffffffe5c1 "app.conf") at config.c:18
18          parse_line(next_line(f));
(gdb) print f
$1 = (FILE *) 0x0                 # 4. and why? fopen failed, unchecked
```

`bt`, `info args`, `up` until something looks wrong. Four commands solve the large majority of segfaults, and none of them requires understanding the program first.

`0x0`	A null pointer: something returned `NULL` unchecked
A small value like `0x10`	A null pointer plus a struct offset — `p->field` where `p` is null
`0xffffffffffffffff`	An uninitialised or already-freed pointer
An address just past a valid one	A buffer overrun
The address is fine, the write fails	Writing to a string literal or other read-only memory

Note 28.1.2. When the pointer expression is complicated, GDB will tell you the faulting address directly:

```
(gdb) print $_siginfo._sifields._sigfault.si_addr
$1 = (void *) 0x0
```

§28.2. Stack Overflow and Runaway Recursion

A crash whose backtrace is thousands of identical frames, or entirely `??`, is usually the stack rather than the heap.

```
(gdb) bt
#0  descend (n=0x5555555592a0) at tree.c:14
#1  0x0000555555555199 in descend (n=0x5555555592a0) at tree.c:17
#2  0x0000555555555199 in descend (n=0x5555555592a0) at tree.c:17
...
#65531 0x0000555555555199 in descend (n=0x5555555592a0) at tree.c:17
(gdb) bt -5                    # the OUTERMOST frames: where it started
(gdb) frame 65000
(gdb) print n == $_saved        # same node every time -- a cycle in the tree
```

`bt 10`, `bt -10`	The two ends of the stack; the middle is repetition
`set backtrace limit 20`	Stop GDB printing 60,000 frames
`info frame`	Where the stack pointer has reached
`ulimit -s`	The stack limit, usually 8 MB

Note 28.2.1. Infinite recursion and a merely deep recursion look identical in a backtrace. The distinguishing question is whether the argument changes between frames: if frame `#1` and frame `#500` have the same pointer, the data structure has a cycle. A very large stack array — `char buf[8*1024*1024]` — produces the same crash with a backtrace only one frame deep.

§28.3. Uninitialised Memory and Heap Corruption

These are the failures GDB is *worst* at, and it is important to know why.

> **Intuition .** A use-after-free or a buffer overrun does its damage at one moment and causes a crash at another, often far away and in unrelated code. GDB shows you the crash. It has no way to show you the write that caused it, because by then nothing is left to see.

What GDB can still do:

`watch -l *ptr`	Catch the write, if you can guess the address in advance
`x/32xb p-16`	Look at the bytes around a suspect block
`print *p`	Implausible field values suggest the block was reused
`p $_memeq(a, b, n)`	Compare two regions

```
(gdb) print *node
$1 = {name = 0x6161616161616161 <error: Cannot access memory>, port = 1633771873}
```

Note 28.3.1. `0x6161616161616161` is `"aaaaaaaa"` read as a pointer — a string has been written over a struct. Recognising ASCII in a pointer is a useful reflex: `0x41414141` is `"AAAA"`, `0xdeadbeef` and `0xcdcdcdcd` are debug fill patterns, and `0x0` is the honest one.

Do not hunt memory corruption in GDB. Rebuild with AddressSanitizer, which catches the bad access at the moment it happens and prints both the access and the allocation it belongs to. Then, if you still need to, come back to GDB knowing exactly where to put the breakpoint.

```
$ gcc -g -O0 -fsanitize=address,undefined -o app main.c
$ ./app
==12345==ERROR: AddressSanitizer: heap-use-after-free on address 0x60300000eff4
READ of size 4 at 0x60300000eff4 thread T0
    #0 0x55d in use_node table.c:88
freed by thread T0 here:
    #1 0x4f2 in release table.c:61
previously allocated by thread T0 here:
    #2 0x48a in make_node table.c:22
```

Note 28.3.2. Three stack traces — the bad access, the `free`, and the `malloc` — which is the entire life story of the bug. This is available on this machine (Section 28.6) and costs one compiler flag.

§28.4. Multi-Threaded Programs

`info threads`	List them; `*` marks the current one
`thread 3`	Switch to thread 3
`thread apply all bt`	A backtrace of every thread
`thread apply all bt full`	… with locals. The deadlock command
`break f thread 3`	A breakpoint only thread 3 honours
`set scheduler-locking on`	Step one thread; freeze the others
`set non-stop on`	Stop one thread while the rest keep running

Example 28.4.1 (Diagnosing a deadlock)**.** Attach to the hung process (Section 23.5) and ask every thread what it is doing:

```
(gdb) thread apply all bt

Thread 2 (Thread 0x7ffff7a3d700):
#0  futex_wait (...) at lowlevellock.c:52
#1  __lll_lock_wait (...)
#2  work (arg=0x0) at worker.c:44        # waiting for mutex B

Thread 1 (Thread 0x7ffff7fc0740):
#0  futex_wait (...) at lowlevellock.c:52
#2  main () at main.c:31                 # waiting for mutex A
```

Two threads, each blocked in `futex_wait`, each holding what the other wants: a lock-ordering deadlock. One command diagnosed it.

Note 28.4.2. `set scheduler-locking on` makes stepping comprehensible — otherwise other threads run while you step and the state changes under you. It also makes deadlock easy to cause yourself, so turn it off again before `continue`.

`thread apply all bt` is to concurrency what `bt` is to a crash. For a hang, it is very often the whole diagnosis.

§28.5. Handling Signals

GDB intercepts every signal before the program sees it, and you can change what it does with each.

`info signals`	The whole table
`handle SIGPIPE nostop noprint`	Ignore it entirely
`handle SIGUSR1 stop print`	Stop on it
`handle SIGSEGV nostop pass`	Let the program's own handler have it
`signal SIGINT`	Deliver a signal by hand
`continue`	Resume, delivering the pending signal

Note 28.5.1. The common need is `handle SIGPIPE nostop noprint pass`: a networking program may take `SIGPIPE` routinely, and GDB stopping every time makes the session unusable. The opposite case is a program with its own `SIGSEGV` handler — GDB stops first, which is what you want, and `pass` lets the handler run if you want to debug the handler instead.

§28.6. Companion Tools

GDB is an inspector, not a detector. These find bugs it can only witness.

AddressSanitizer	`-fsanitize=address`. Use-after-free, buffer overruns, leaks, with the allocating stack trace. A few times slower
UBSan	`-fsanitize=undefined`. Signed overflow, bad shifts, misaligned access, null dereference
ThreadSanitizer	`-fsanitize=thread`. Data races, which are otherwise nearly impossible to find
Valgrind	`valgrind ./app`. The same class of errors with no recompilation, considerably slower
rr	`rr record` then `rr replay`. Records an execution so you can step *backwards* in GDB
strace, **ltrace**	Every system or library call the program makes

Note 28.6.1. On this machine the sanitisers are available through GCC 13, and `strace` is present; `valgrind` and `rr` are not installed (`apt install valgrind`). The sanitisers are the better default anyway — they are faster and more precise — with Valgrind's advantage being that it needs no rebuild, which matters when you only have the binary.

Match the tool to the failure. A crash with a sensible backtrace: GDB. Memory corruption, a leak, or a crash whose backtrace is nonsense: a sanitiser first, then GDB. A hang: `gdb -p` and `thread apply all bt`. A wrong answer with no crash at all: a breakpoint and `display`.

29. Making GDB Comfortable

Out of the box GDB is a bare prompt with no source on screen and no memory of your preferences. Twenty lines of configuration fix both, and are worth writing once.

§29.1. The TUI

Definition 29.1.1 (TUI)**.** GDB's built-in Text User Interface: a split terminal with the source, the registers or the disassembly above and the command prompt below. It is part of GDB, needs nothing installed, and follows the program as it runs.

`gdb -tui ./app`	Start with it on
`<C-x>` `a`	Toggle it at any time
`<C-x>` `1`	One window: source
`<C-x>` `2`	Two windows: source and disassembly
`layout src`	Source only
`layout asm`	Disassembly
`layout split`	Both
`layout regs`	Add the registers
`<C-x>` `o`	Move focus between windows
`<C-l>`	Redraw, when the display gets corrupted
`focus cmd`	Give the arrow keys back to the command line
`tui reg general`	Which register group to show

Note 29.1.2. The first thing everyone hits: with focus in the source window, the arrow keys scroll the source instead of walking the command history. `focus cmd`, or `<C-x>` `o`, gives them back. The second thing: program output and the TUI share a terminal and will corrupt each other — `<C-l>` redraws, and `set inferior-tty /dev/pts/3` sends the program's output to a separate terminal for good.

If you do nothing else in this chapter, learn `<C-x>` `a`. Seeing the current line highlighted in context, updating as you step, is the difference between GDB feeling archaic and feeling like a debugger.

§29.2. `.gdbinit` and User Commands

GDB reads `~/.gdbinit` at startup, and then `./.gdbinit` from the current directory if it is trusted.

```
# ~/.gdbinit
set confirm off                 # do not ask "are you sure" for everything
set pagination off              # do not stop every screenful
set history save on             # remember commands between sessions
set history size 10000
set print pretty on             # structs one member per line
set print array on
set print elements 0            # never truncate
set disassembly-flavor intel    # if you prefer Intel syntax
set breakpoint pending on       # do not ask about future shared libraries
set backtrace limit 100         # tame runaway recursion

# a source window by default
tui enable
```

Note 29.2.1. A project-local `.gdbinit` is read only if its directory appears in GDB's auto-load safe path, which is a security measure — otherwise cloning a repository and running `gdb` in it would execute someone else's commands. Add `set auto-load safe-path /` to your global file if you trust everything you check out, and understand what you are turning off.

User-defined commands make a repeated inspection into one word:

```
define plist
  set $n = $arg0
  while $n != 0
    printf "%s\n", $n->name
    set $n = $n->next
  end
end
document plist
Walk a linked list from the given head, printing each name.
end

define bpl
  break $arg0
  commands
    silent
    printf "hit %s\n", "$arg0"
    bt 3
    continue
  end
end
```

`$arg0`, `$arg1`	The command's arguments
`$argc`	How many were given
`document name`	Text for `help name`
`source debug.gdb`	Load a file of commands mid-session
`define hook-run`	Run automatically before every `run`

Every project accumulates two or three inspections you do constantly — print this tree, dump that ring buffer, check those invariants. Put them in a `.gdbinit` beside the source and commit it. It is documentation that executes.

§29.3. Pretty-Printers and Python

GDB embeds Python, and the most visible use is **pretty-printers**: functions that decide how a type is displayed.

```
(gdb) print v                                   # without a pretty-printer
$1 = {_M_impl = {<std::allocator<int>> = {...}, _M_start = 0x5555555592a0,
  _M_finish = 0x5555555592ac, _M_end_of_storage = 0x5555555592b0}}
(gdb) print v                                   # with one
$2 = std::vector of length 3, capacity 4 = {1, 2, 3}
```

Note 29.3.1. For the C++ standard library these ship with GCC and are usually loaded automatically — `info pretty-printer` lists what is active, and `set print pretty-printer off` turns them off when you need to see the raw struct. For your own types, a printer is a short Python class registered against a type name; `help python` and the GDB manual's *Pretty Printing* chapter have the pattern.

`python print(gdb.parse_and_eval("n"))`	Read a program value from Python
`python gdb.execute("bt")`	Run a GDB command from Python
`info pretty-printer`	What is registered
`source printers.py`	Load a Python script

§29.4. Front Ends, and GDB from Neovim

`gdb -tui`	Built in, always available, no setup
`cgdb`	A curses front end with a better source window
`gdbgui`	A browser front end
`nvim-dap`	Debug Adapter Protocol client for Neovim
`termdebug`	Ships *with* Vim and Neovim — `:packadd termdebug`

Example 29.4.1 (`termdebug`)**.** The lowest-effort way to debug inside the editor of Part I: it is already installed, and needs no plugin.

```
:packadd termdebug
:Termdebug ./app
```

Neovim splits into a GDB window, a program-output window and your source. Then `:Break` sets a breakpoint on the current line, `:Run`, `:Step`, `:Over` and `:Finish` drive it, and `:Evaluate` prints the expression under the cursor — while the source buffer stays an ordinary Neovim buffer with all the motions of Chapter 3.

Note 29.4.2. `nvim-dap` is the fuller answer — a proper debug adapter with a variables pane, breakpoint signs in the gutter and per-language configuration — and it is a plugin to add through `lazy.nvim` (Section 7.9). It is not installed on this machine. Start with `termdebug`, which costs nothing, and add `nvim-dap` if you find yourself debugging daily.

§29.5. A Worked Debugging Session

Everything in Part IV, on one bug. The program counts words in a file and reports zero for a file that plainly has some.

```
int count_words(const char *path) {
    FILE *f = fopen(path, "r");
    char buf[64];
    int n = 0;
    while (fgets(buf, sizeof buf, f)) {
        for (char *p = strtok(buf, " \n"); p; p = strtok(NULL, " \n"))
            n++;
    }
    fclose(f);
    return n;
}
```

```
$ gcc -g -O0 -o wc main.c && gdb -q ./wc
(gdb) break count_words
(gdb) run input.txt
Breakpoint 1, count_words (path=0x7fffffffe5c9 "input.txt") at main.c:6
6           FILE *f = fopen(path, "r");
(gdb) next
8           int n = 0;
(gdb) print f
$1 = (FILE *) 0x5555555592a0          # the file opened fine
(gdb) display n                       # watch the counter as we go
1: n = 0
(gdb) break 10 if n > 0               # stop as soon as it counts anything
(gdb) continue
Continuing.
1: n = 0                              # ...it never does
[Inferior 1 (process 41233) exited normally]
```

The loop body never runs, so `fgets` returned `NULL` the first time. Second run, with the attention one line earlier:

```
(gdb) run input.txt
(gdb) break 9
(gdb) continue
Breakpoint 2, count_words (...) at main.c:9
9           while (fgets(buf, sizeof buf, f)) {
(gdb) print sizeof buf
$2 = 64
(gdb) print fgets(buf, 64, f)
$3 = 0x0                              # NULL on the very first read
(gdb) print ftell(f)
$4 = 0                                # ...yet we are at the start of the file
(gdb) print feof(f)
$5 = 0
(gdb) print ferror(f)
$6 = 1                                # the stream is in an error state
```

Note 29.5.1. `print fgets(...)` and `print ferror(f)` are *calling* those functions in the live process (Section 24.4). Being able to interrogate a C library from the prompt — with the program's own `FILE *` in hand — is what makes GDB more than a viewer.

The stream errored without reaching end of file, which points at the mode. `print path`

shows `"input.txt"`, and a look at the caller resolves it: the caller opened the same file earlier, closed it, and passed the stale name while an `fopen` failure elsewhere left `errno` set. The fix is the check that was never there:

```
FILE *f = fopen(path, "r");
if (!f) { perror(path); return -1; }
```

Note what the session did *not* involve: no recompiling with print statements, no guessing, and no reading of the whole program. Establish where it stops being right, look at the state there, and walk outwards one frame at a time. That loop is the entire method, and the commands in Part IV are only its vocabulary.

Try it

Write a ten-line C program with a deliberate null dereference. Compile it with `gcc -g -O0`, run `gdb -q ./a.out`, then `run`, `bt`, `info locals`. You have now done the thing that most people avoid for years, and it took under a minute.

Part V.

Git and GitHub

Nearly all confusion about Git is confusion about its model, not its commands. People who have memorised `git pull` and `git push` without ever being told what a commit *is* will eventually hit a merge conflict, a detached `HEAD` or a bad rebase, and have no way to reason about what they are looking at.

So Chapter 30 spends its whole length on the model — commits as snapshots, branches as movable pointers, the three trees — before running a single useful command. Everything afterwards follows from it: chapters 31 to 33 on daily work, Chapter 34 on the collaboration layer GitHub adds, and Chapter 36 on the fact that in Git almost nothing is ever really lost, provided you know where to look.

30. Version Control and Git's Model

Nearly all confusion about Git is confusion about its model, not its commands. People who have memorised `git pull` and `git push` without ever being told what a commit *is* will eventually hit a merge conflict, a detached `HEAD` or a bad rebase, and have no way to reason about what they are looking at.

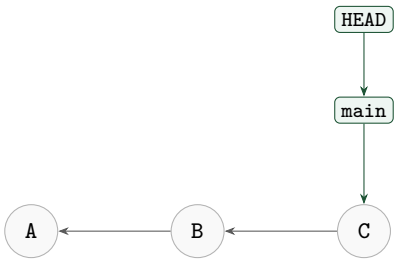
So this chapter runs no useful commands at all. It is about four ideas — snapshots, objects, the three trees, and refs — and everything in Part V follows from them.

§30.1. Notation for Part V

Git is driven from the shell, so transcripts keep the convention of Section 9.1: `$` is what you type, everything else is output.

```
$ git status
On branch main
nothing to commit, working tree clean
```

Commit graphs are drawn with the newest commit on the right, which is the opposite of how `git log` prints them and the same as how everyone draws them on a whiteboard:



`A`, `B`, `C`	Commits, oldest on the left
`main`, `feature`	Branches — names pointing at a commit
`HEAD`	Where you are now
`a1b2c3d`	An abbreviated commit hash
`<sha>`, `<branch>`	Placeholders, as in Section 9.1

Note 30.1.1. The arrows point from a commit to its *parent*, backwards in time. That is the direction the data actually goes: a commit records what came before it, and nothing records what comes after. Almost every surprising property of Git — why history is cheap to rewrite, why a branch is free, why a lost commit is still there — follows from that one arrow direction.

§30.2. Why Version Control

The obvious answer is backups, and it is the least interesting one. What a version control system actually gives you:

A safety net	Any change can be undone, including the one you made three weeks ago
A record	Every line has an author, a date, and a message saying why
Parallel work	Several people, or several of your own ideas, on the same files
Bisection	"It worked in March" becomes a mechanical search (Section 36.5)
A shared truth	One authoritative history, rather than `report_final_v3_FINAL.tex`

Intuition . The real change is psychological. With history you can afford to try things — delete a section, restructure a module, rewrite an algorithm — because the cost of being wrong is one command. Without it, every experiment is a negotiation with your own nerve.

§30.3. Snapshots, Not Diffs

This is the design decision that makes Git different from what came before.

Definition 30.3.1 (Commit)**.** A complete *snapshot* of every tracked file at one moment, together with its author, its date, its message, and a pointer to the commit or commits that came before it.

Older systems — CVS, Subversion — stored a list of files and, for each, the sequence of changes made to it. Git stores the whole tree, every time.

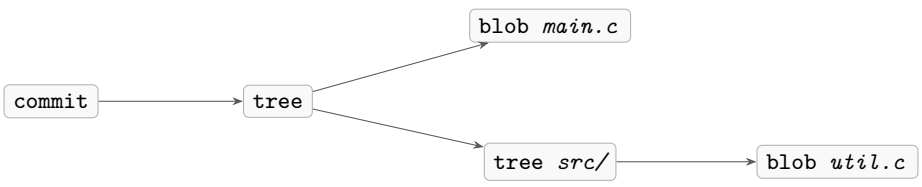
Intuition . That sounds wasteful and is not: a file that did not change between two commits is not stored twice, it is *referenced* twice, because Git identifies content by its hash. Storing snapshots rather than diffs means any commit can be reconstructed without replaying history, which is why `git checkout` of a five-year-old commit is instant.

Note 30.3.2. `git diff` shows you a diff, so it is easy to conclude that Git stores diffs. It does not: the diff is computed on demand by comparing two snapshots. This is why Git can show a diff between *any* two commits, related or not.

§30.4. The Object Database

Everything Git stores is one of four kinds of object, each named by the SHA-1 hash of its own contents.

blob	The contents of a file. No name, no permissions — just bytes
tree	A directory: a list of names, each with a mode and a pointer to a blob or another tree
commit	A pointer to one tree, plus parents, author, date and message
tag	An annotated name for a commit



Example 30.4.1. You can look at any of them directly:

```
$ git cat-file -t HEAD           # what kind of object is this?
commit
$ git cat-file -p HEAD           # print it
tree 4b825dc642cb6eb9a060e54bf8d69288fbee4904
parent 8f3a1c9e2b7d4a5f6c8e9d0a1b2c3d4e5f6a7b8c
author Kon Yi <you@example.com> 1757808000 +0800
committer Kon Yi <you@example.com> 1757808000 +0800

    Week1 Finish
$ git cat-file -p HEAD^{tree}    # the directory it points at
100644 blob a3f2e1d...    README.md
040000 tree 9c4b8a2...    Chapters
```

That is the entire format. A commit is a small text file naming a tree; a tree is a list naming blobs and other trees.

Definition 30.4.2 (Content addressing)**.** An object's name *is* the hash of its contents. Two identical files anywhere in history are one blob. Changing a byte changes the hash, which changes the tree that names it, which changes the commit — so a commit hash certifies the entire tree beneath it.

This is why history cannot be quietly altered: editing an old commit gives it a new hash, which changes every descendant's hash too. "Rewriting history" (Section 36.7) does not modify commits — it makes new ones and moves the branch.

§30.5. The Three Trees

Git manages three versions of your project at once, and almost every confusing Git message is about the boundary between two of them.



Working tree	The actual files in your directory. What your editor sees
Index (or **staging area**)	A draft of the next commit. `git add` copies from the working tree into it
HEAD	The commit you last committed or checked out

Intuition . The index is the feature people find pointless until they need it, and then cannot do without. It lets you decide *what* to commit separately from *what you have changed* — so a session that fixed a bug and also reformatted a function becomes two clean commits instead of one muddled one. `git add -p` (Section 31.3) is where this pays off.

Note 30.5.1. Now `git status` reads as a report on two boundaries: *"Changes to be committed"* is index versus `HEAD`; *"Changes not staged for commit"* is working tree versus index; and

"Untracked files" are in neither. `git diff` compares working tree with index, and `git diff --staged` compares index with `HEAD`. Nothing else needs to be memorised — it all falls out of the diagram.

§30.6. Refs, `HEAD`, and What a Branch Really Is

Definition 30.6.1 (Branch)**.** A file containing forty hex digits. That is not a simplification: a branch is a **ref** — a name pointing at one commit — stored in `.git/refs/heads/`, and committing moves the pointer forward.

```
$ cat .git/refs/heads/main
bc91a17e5f3d2c1b0a9876543210fedcba987654
$ cat .git/HEAD
ref: refs/heads/main
```

Branches are free. Creating one writes 41 bytes; deleting one removes them. Nothing is copied, no files move, and there is no reason to hesitate before making one. If you come from Subversion, where a branch was a server-side copy of the tree, this is the single biggest adjustment.

`HEAD` is one more level of indirection: it normally holds the *name* of a branch rather than a commit. So committing updates `HEAD`'s branch, and `git switch` rewrites `HEAD` to name a different one.

Definition 30.6.2 (Detached `HEAD`)**.** The state where `HEAD` holds a commit hash directly instead of a branch name — after `git checkout <sha>`, or checking out a tag. Commits made there belong to no branch, so moving away leaves them unreferenced and apparently lost.

Note 30.6.3. This is the state that alarms people, and it is harmless as long as you understand it. Nothing is broken; you are simply standing on a commit with no name attached. `git switch -c fix` gives the current position a name and you are back on ordinary ground — and even if you walk away first, Section 36.1 finds it again.

`HEAD`	Where you are
`HEAD~1`, `HEAD~3`	One, three commits back along first parents
`HEAD^`, `HEAD^2`	The first, second parent of a merge
`main@{yesterday}`	Where `main` pointed yesterday
`a1b2c3d`	A commit by hash; seven characters is usually enough
`v1.0`	A tag
`origin/main`	A **remote-tracking** branch: where `main` was on the remote when you last fetched

§30.7. First-Time Setup

```
$ git config --global user.name  "Kon Yi"
$ git config --global user.email "you@example.com"
$ git config --global init.defaultBranch main
$ git config --global core.editor "nvim"
$ git config --global pull.rebase true         # see sec. 33.4
$ git config --global --list                   # check what you have
```

`--global`	`~/.gitconfig` — all your repositories
`--local`	`.git/config` — this repository only. The default
`--system`	Every user on the machine
`--list --show-origin`	Every setting, and which file set it

Note 30.7.1. The name and email are baked into every commit you make and cannot be changed afterwards without rewriting history. Set them before your first commit. If you use a different address for work, a conditional include keeps them apart:

```
# ~/.gitconfig
[user]
    name = Kon Yi
    email = personal@example.com

[includeIf "gitdir:~/work/"]
    path = ~/.gitconfig-work
```

Note 30.7.2 (This machine)**.** Git 2.43 is installed, with `user.name` set to `Konyi0613` and credentials handled by the GitHub CLI — `gh auth git-credential` is registered as the credential helper for `github.com`, so pushing needs no password prompt. Section 34.2 covers what that arrangement is doing.

Try it

In any repository you have, run `git cat-file -p HEAD`, then `git cat-file -p HEAD^{tree}`, then `cat .git/HEAD`. Three commands, and the abstraction is gone: a commit really is a short text file naming a tree, and a branch really is a file with a hash in it.

31. The Everyday Loop

Ninety per cent of Git use is six commands in a cycle: change something, look at what changed, stage it, commit it, and occasionally undo. This chapter is that cycle.

§31.1. Starting a Repository

`git init`	Turn this directory into a repository
`git init -b main`	… with a named initial branch
`git clone <url>`	Copy an existing repository, history and all
`git clone <url> mydir`	… into a directory of your choosing
`git clone --depth 1 <url>`	A **shallow** clone: latest commit only
`git clone --branch v2 <url>`	Start on a particular branch or tag

Note 31.1.1. `git init` creates one directory, `.git/`, and changes nothing else. Deleting `.git/` turns the project back into ordinary files with no history — which is worth knowing both as reassurance and as a warning.

§31.2. `status` and the Index

```
$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.

Changes to be committed:            # index vs HEAD
        modified:   Chapters/git_1_model.tex

Changes not staged for commit:      # working tree vs index
        modified:   CSTools.tex

Untracked files:                    # in neither
        Chapters/git_2_loop.tex
```

Those three headings are exactly the three trees of Section 30.5, which is why that section came first.

`git status`	The full report
`git status -s`	Short: two columns, index then working tree
`git status -sb`	… with the branch line

`M`	Modified, staged
` M`	Modified, not staged
`MM`	Staged, then modified again
`A`	Newly added
`D`	Deleted
`R`	Renamed
`??`	Untracked

`git status` is the command to run when you do not know what to do next. It reports the state *and* suggests the command — which file to add, how to unstage, how to abort a merge. Read it rather than searching the web.

§31.3. Staging

`git add file.tex`	Stage one file
`git add Chapters/`	Stage a directory
`git add -A`	Stage everything, deletions included
`git add -u`	Stage changes to *tracked* files only
`git add -p`	Stage *parts* of files, interactively
`git rm file`	Delete it and stage the deletion
`git rm --cached file`	Stop tracking it, keep it on disk
`git mv old new`	Rename, and stage the rename

`git add -p` is the command that justifies the index existing at all. It walks through each change and asks:

```
$ git add -p
diff --git a/parser.c b/parser.c
@@ -40,7 +40,7 @@ int parse_line(const char *s)
-    while (*s != '\n') {
+    while (s && *s != '\n') {          # the bug fix
(1/2) Stage this hunk [y,n,q,a,d,s,e,?]? y
@@ -88,3 +88,6 @@
+/* TODO: rewrite this whole function */   # an unrelated note
(2/2) Stage this hunk [y,n,q,a,d,s,e,?]? n
```

`y`	Stage this hunk
`n`	Do not
`s`	Split it into smaller hunks
`e`	Edit the hunk by hand
`q`	Stop
`?`	Explain every key

One commit, one idea. `git add -p` is how you get there when an afternoon's work touched four unrelated things — and a history of single-idea commits is what makes Section 36.5 and `git revert` work at all.

§31.4. Committing

`git commit`	Open your editor for the message
`git commit -m "..."`	Message on the command line
`git commit -a`	Stage all tracked changes, then commit
`git commit --amend`	Replace the last commit (Section 36.3)
`git commit -v`	Show the diff in the editor while you write

Note 31.4.1. `git commit -v` puts the full staged diff below the message in your editor. It costs nothing, and it catches the debug `printf` you forgot to remove roughly once a month. Turn it on permanently with `git config -global commit.verbose true`.

A commit message has a standard shape, and it is worth following:

```
Fix crash when config file is missing

fopen's return value was never checked, so a missing app.conf
produced a NULL FILE* that parse_line dereferenced.

Add the check and report the error with perror.

Closes #42
```

First line	A summary under about 50 characters
Imperative mood	"Fix crash", not "Fixed" or "Fixes"
A blank line	Git relies on it to separate subject from body
The body	*Why*, not what — the diff already says what
Wrap at 72 characters	So that `git log` is readable in a terminal

The audience for a commit message is you, in eight months, running `git log -S` on a line that makes no sense. "Update" and "fix stuff" are not messages; they are the absence of one. The imperative mood is not fussiness — it makes the subject read as *what applying this commit does*, which is how `git revert` and `git cherry-pick` generate their own messages.

§31.5. `diff`: Three Comparisons

`git diff`	Working tree vs index — what is *not* staged
`git diff --staged`	Index vs `HEAD` — what *is* staged
`git diff HEAD`	Working tree vs `HEAD` — everything
`git diff main..feature`	Between two branches
`git diff a1b2c3d`	Against an old commit
`git diff -- file.tex`	Restricted to one file
`git diff --stat`	Just the summary of how much changed
`git diff -w`	Ignore whitespace. Invaluable after reformatting
`git diff --word-diff`	Word-level rather than line-level. Good for prose

Note 31.5.1. The output is the unified diff of Section 11.4, so nothing new has to be learnt to read it. `-word-diff` is worth remembering for LaTeX: a reflowed paragraph produces a line diff that shows everything as changed and a word diff that shows the three words you actually edited.

§31.6. Reading History

`git log`	The full history, newest first
`git log --oneline`	One line per commit
`git log --oneline --graph --all`	... with the branch topology
`git log -5`	The last five
`git log --stat`	With a summary of files changed
`git log -p`	With the full diff of each commit
`git log -- file.tex`	Only commits touching a file
`git log -S'fopen'`	Commits that added or removed that string
`git log --author=Kon`	By author
`git log --since='2 weeks'`	By date
`git log main..feature`	On `feature` but not on `main`
`git show <sha>`	One commit in full

Example 31.6.1. The invocation worth aliasing:

```
$ git log --oneline --graph --all --decorate
* 3f8a2b1 (HEAD -> feature) Add config validation
* 9c4e7d2 Extract parse_line
| * bc91a17 (origin/main, main) First day content only
|/
* b68711b Week1 Finish
```

The graph column shows where `feature` left `main`, and `-decorate` puts the branch names on the commits they point at — the diagram from Section 30.1, drawn in ASCII.

§31.7. .gitignore

```
# build products
build/
*.o
*.pdf

# LaTeX intermediates
*.aux
*.log
*.synctex.gz

# editor and OS noise
.DS_Store
*.swp

# but keep this one
!doc/manual.pdf
```

`build/`	A directory and everything in it
`*.o`	A glob, matched at any depth
`/secrets.txt`	Only at the repository root
`!keep.pdf`	A negation: track this despite an earlier rule
`doc/**/tmp`	Any depth between the two components

Note 31.7.1. `.gitignore` only affects *untracked* files. A file already committed goes on being tracked no matter what you add to the ignore list — `git rm -cached` removes it from tracking while leaving it on disk. `git check-ignore -v file` says which rule, in which file, is ignoring something.

Never commit build products, dependencies, or secrets. The first two make every diff unreadable; the third cannot be undone by deleting the file later, because the old commit still contains it (Section 36.8).

§31.8. Undoing

Four commands with four different meanings. Choosing between them is a matter of asking *which of the three trees do I want to change?*

`git restore file`	Working tree ← index. Discard unstaged edits
`git restore --staged file`	Index ← `HEAD`. Unstage, keep the edits
`git restore --source=HEAD~2 file`	Take a file from an old commit
`git reset --soft HEAD~`	Undo the commit, keep everything staged
`git reset HEAD~`	Undo the commit and unstage; files untouched
`git reset --hard HEAD~`	Undo the commit and **destroy** the changes
`git revert <sha>`	A *new* commit undoing an old one
`git clean -n` / `git clean -fd`	List / delete untracked files

Intuition . `reset` moves the branch pointer backwards and rewrites history; `revert` leaves history alone and adds a commit that cancels an earlier one. So `reset` is for mistakes nobody else has seen, and `revert` is for anything already pushed.

Note 31.8.1 (The only genuinely dangerous one)**.** `git reset -hard` and `git clean -fd` destroy uncommitted work, and that work was never in a commit, so Section 36.1 cannot bring it back. Every other command in this part is recoverable. Run `git clean -n` first, always — it prints exactly what `-f` would delete.

Example 31.8.2.

```
$ git restore CSTools.tex             # throw away today's edits to one file
$ git restore --staged CSTools.tex    # "I added the wrong file"
$ git reset --soft HEAD~              # "I committed too early"
$ git revert bc91a17                  # "that pushed commit was wrong"
$ git commit --amend                  # "the message had a typo"
```

Try it

In a scratch repository, make a change, run `git status`, stage it, run `git status` again, and match each heading to the diagram in Section 30.5. Then unstage with `git restore -staged` and watch the file move back across the boundary.

32. Branching and Merging

A branch is a file containing a hash (Section 30.6). Everything in this chapter follows from that, including the parts that frighten people.

§32.1. Creating and Switching

`git branch`	List local branches; * marks the current
`git branch -a`	Include remote-tracking branches
`git branch -v`	... with each one's latest commit
`git switch feature`	Move to an existing branch
`git switch -c feature`	Create it and move there
`git switch -c fix main`	... starting from `main` rather than here
`git switch -`	Back to the previous branch
`git branch -d feature`	Delete it — refuses if unmerged
`git branch -D feature`	Delete it anyway
`git branch -m old new`	Rename

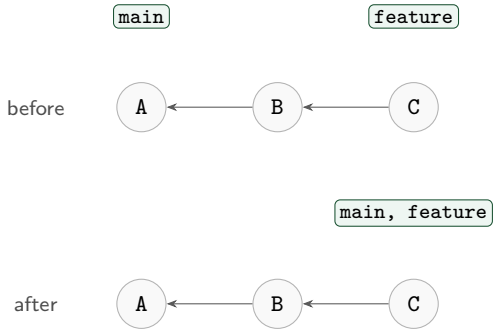
Note 32.1.1. `git switch` and `git restore` were introduced in Git 2.23 to split the two unrelated jobs that `git checkout` had been doing — moving between branches, and throwing away file changes. `git checkout` still works and you will see it everywhere; the newer pair is clearer, and much harder to use by accident.

Example 32.1.2. Branching costs nothing, so branch for everything:

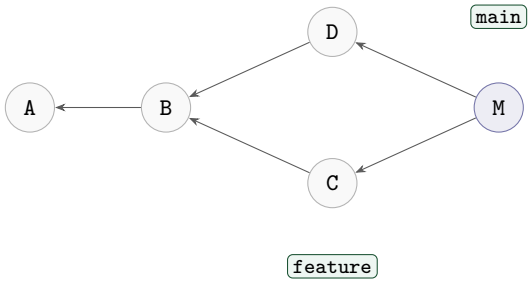
```
$ git switch -c fix-config-crash
Switched to a new branch 'fix-config-crash'
$ # ... edit, commit, edit, commit ...
$ git switch main
$ git merge fix-config-crash
$ git branch -d fix-config-crash
```

§32.2. Fast-Forward and Three-Way Merges

Definition 32.2.1 (Fast-forward)**.** When the target branch is a direct ancestor of the one being merged, there is nothing to combine: Git simply moves the pointer forward. No merge commit is created.



When both branches have moved on, Git makes a **merge commit** with two parents:



`git merge feature`	Merge `feature` into the current branch
`git merge --no-ff feature`	Always make a merge commit, even if it could fast-forward
`git merge --squash feature`	Take the changes as one uncommitted lump
`git merge --abort`	Give up and return to where you started

Note 32.2.2. `-no-ff` is worth knowing. A fast-forward loses the fact that a group of commits belonged to one branch; a merge commit records it, so `git log -graph` keeps showing the shape of the work. Many teams require it, and GitHub's "Create a merge commit" button does exactly this.

§32.3. Conflicts

Definition 32.3.1 (Conflict)**.** Both sides changed the same region of the same file, and Git will not guess. It writes both versions into the file with markers and stops, leaving you to produce the correct result.

```
<<<<<<< HEAD
    timeout = 30;
=======
    timeout = 60;
>>>>>>> feature
```

`<<<<<<< HEAD`	Everything below is *your* current branch
`=======`	The divider
`>>>>>>> feature`	Everything above is the branch being merged

The procedure is always the same:

(1) `git status` lists the conflicted files.

(2) Open each one, delete the markers, and leave the text you actually want — which may be either side, both, or something new.

(3) `git add` the file to say it is resolved.

(4) `git commit` when they are all done, or `git merge -abort` to walk away.

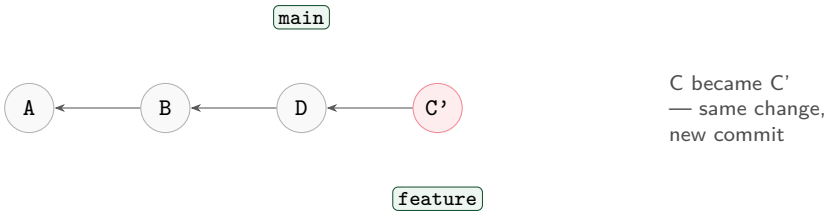
`git status`	Which files are still conflicted
`git diff`	The remaining conflicts, in a combined diff
`git checkout --ours file`	Take your side wholesale
`git checkout --theirs file`	Take theirs wholesale
`git merge --abort`	Undo the whole merge
`git mergetool`	Open a three-way merge tool — `nvim -d` by default with `merge.tool=vimdiff`

Note 32.3.2. "Ours" and "theirs" mean the opposite of what you expect during a *rebase*: rebasing replays your commits on top of the other branch, so `-ours` is the branch you are rebasing onto. Check with `git status`, which names both sides explicitly, rather than trusting the words.

A conflict is not an error and not a failure. It is Git declining to guess, which is the correct behaviour. The way to have fewer of them is to merge *more* often, not less: two branches that diverge for a month conflict badly, and two that sync daily barely conflict at all.

§32.4. Rebase

Definition 32.4.1 (Rebase)**.** Replay a series of commits onto a different base. Git takes each commit on your branch, computes its diff, and applies it to the new starting point — producing *new* commits with new hashes.



`git rebase main`	Replay this branch on top of `main`
`git rebase --continue`	After resolving a conflict
`git rebase --skip`	Drop the commit being applied
`git rebase --abort`	Return to where you started
`git pull --rebase`	Fetch, then rebase rather than merge

Intuition . Merge preserves what happened; rebase produces what you wish had happened. A merge records that two lines of work existed and came together. A rebase makes it look as though you had started from the latest `main` all along, giving a linear history that is far easier to read and to bisect.

[The golden rule] Never rebase commits that other people have. Rebasing creates new commits with new hashes; anyone who had the old ones now has a divergent history, and merging the two duplicates every commit. Rebase your own unpublished work freely; once it is pushed to a shared branch, merge instead.

§32.5. Interactive Rebase

`git rebase -i` is where a messy series of commits becomes a clean one.

```
$ git rebase -i main
```

Your editor opens with the commits to be replayed, oldest first:

```
pick 9c4e7d2 Extract parse_line
pick 3f8a2b1 fix typo
pick 7b1e4a9 Add config validation
pick 2d8c5f0 oops, forgot the header

# Commands:
#  p, pick   = use commit
#  r, reword = use commit, but edit the message
#  e, edit   = use commit, but stop for amending
#  s, squash = meld into previous commit, keep both messages
#  f, fixup  = meld into previous commit, discard this message
#  d, drop   = remove commit
#  (lines may be reordered; they are applied top to bottom)
```

Editing it into this:

```
pick 9c4e7d2 Extract parse_line
fixup 3f8a2b1 fix typo
pick 7b1e4a9 Add config validation
fixup 2d8c5f0 oops, forgot the header
```

turns four commits into two clean ones, each self-contained.

`reword`	Fix a message without touching the content
`squash`	Combine, and edit the joined message
`fixup`	Combine, discarding this commit's message. The common case
`drop`	Remove a commit entirely
`edit`	Stop there so you can amend or split it
Reorder lines	Reorder the commits

Note 32.5.1. `git commit -fixup=<sha>` marks a commit as a fix for an earlier one, and `git rebase -i -autosquash main` then pre-arranges the list for you. Set `git config -global rebase.autosquash true` and the workflow becomes: fix something, mark it, and squash the whole series before opening a pull request.

Commit early and often while working — including "wip" and "oops". Then tidy the series with an interactive rebase before anyone else sees it. You get both a safety net while you work and a readable history afterwards.

§32.6. `cherry-pick`

`git cherry-pick <sha>`	Apply one commit here
`git cherry-pick a1b2..c3d4`	A range
`git cherry-pick -n <sha>`	Apply it but do not commit
`git cherry-pick --abort`	Undo

Example 32.6.1. The standard use is a hotfix that belongs on two branches: commit it on `main`, then cherry-pick it onto the release branch. As with rebase, the result is a *new* commit with a different hash — the same change, recorded twice.

§32.7. Stashing

`git stash`	Put working-tree changes aside; clean the tree
`git stash -u`	Include untracked files
`git stash push -m "wip"`	… with a label
`git stash list`	What is stashed
`git stash pop`	Reapply the newest and remove it from the list
`git stash apply stash@{2}`	Reapply a particular one, keeping it
`git stash drop`	Discard one
`git stash show -p`	The diff it holds

Note 32.7.1. Stashing is for the interruption: you are half-way through something and must switch branches now. It is not storage — an unlabelled stash from three weeks ago is indistinguishable from the other four. If work is worth keeping for more than an hour, commit it on a branch instead; branches are free (Section 30.6) and they have names.

§32.8. Worktrees

`git worktree add ../hotfix main`	A second working directory, on another branch
`git worktree list`	Which ones exist
`git worktree remove ../hotfix`	Tidy up

A worktree is the better answer to "I need to look at `main` while I am in the middle of this". It gives you a second directory sharing one object database — no stash, no switching, and your build outputs stay where they are. For a project with a long compile, this alone is worth the chapter.

33. Remotes and Collaboration

Everything so far worked with no network. Git is **distributed**: your clone is a complete repository with the full history, and a remote is just another clone you have agreed to exchange commits with.

§33.1. Remotes

Definition 33.1.1 (Remote)**.** A named URL of another repository. `origin` is the conventional name for the one you cloned from; `upstream` is the convention for the original project when you are working from a fork.

`git remote -v`	List remotes and their URLs
`git remote add upstream <url>`	Add one
`git remote remove upstream`	Remove one
`git remote show origin`	What it has, and how your branches track it
`git remote set-url origin <url>`	Change where it points

```
$ git remote -v
origin    git@github.com:Konyi0613/LaTeX.git (fetch)
origin    git@github.com:Konyi0613/LaTeX.git (push)
```

Definition 33.1.2 (Remote-tracking branch)**.** A local, read-only record of where a branch stood on the remote when you last fetched — written `origin/main`. It is not the remote's branch, and it does not update by itself: it is a cached snapshot.

Note 33.1.3. The distinction explains *"Your branch is ahead of 'origin/main' by 2 commits"*. Git is comparing your `main` against its own cached `origin/main`, not against the server. If someone else pushed five minutes ago, Git does not know until you fetch.

§33.2. `fetch` versus `pull`

`git fetch`	Download new commits; change nothing else
`git fetch --all`	From every remote
`git fetch --prune`	. . . and delete remote-tracking branches that are gone
`git pull`	`fetch`, then `merge`
`git pull --rebase`	`fetch`, then `rebase`

`git fetch` is always safe. It updates your remote-tracking branches and touches neither your working tree nor your branch, so you can look before deciding. `git pull` is `fetch` plus an immediate merge, which is why it is the command that occasionally produces a conflict you were not expecting.

```
$ git fetch
$ git log --oneline HEAD..origin/main     # what did they do?
$ git diff HEAD origin/main               # and what does it change?
$ git merge origin/main                   # now decide
```

§33.3. `push` and Tracking

`git push`	Push the current branch to its upstream
`git push -u origin feature`	Push and set the upstream for next time
`git push origin --delete feature`	Delete a branch on the remote
`git push --tags`	Push tags, which `push` does not do
`git push --force-with-lease`	Overwrite the remote branch, but only if nobody else has pushed
`git push --force`	Overwrite unconditionally

Note 33.3.1 (Which force to use)**.** After an interactive rebase your local history no longer matches the remote and a plain push is rejected. `-force` overwrites whatever is there, including a colleague's commits pushed in the meantime. `-force-with-lease` refuses unless the remote is still where you last saw it — the same operation, with a check. There is no reason to use plain `-force` on a shared branch.

Note 33.3.2. `fatal: The current branch has no upstream branch` means Git does not know where to push. `git push -u origin feature` sets it once; thereafter plain `git push` works. Setting `git config -global push.autoSetupRemote true` makes Git do it for you.

§33.4. Merge or Rebase on Pull

Default `git pull` merges, and on a shared branch this litters history with merge commits saying nothing but "I synced":

```
Merge branch 'main' of github.com:Konyi0613/LaTeX
Merge branch 'main' of github.com:Konyi0613/LaTeX
Merge branch 'main' of github.com:Konyi0613/LaTeX
```

```
$ git config --global pull.rebase true       # rebase instead of merging
$ git config --global rebase.autoStash true  # stash and restore automatically
```

Rebase when pulling, merge when integrating. Pulling other people's commits onto your unpushed work is exactly the case the golden rule of Section 32.4 permits — your commits are not published yet. Bringing a finished branch into `main` is the case for a merge commit, because the fact that it was a branch is worth recording.

§33.5. Tags and Releases

Definition 33.5.1 (Tag)**.** A name permanently attached to one commit. Unlike a branch, it does not move. A **lightweight** tag is just a ref; an **annotated** tag is a real object with an

author, a date and a message, and is what releases should use.

`git tag`	List tags
`git tag -a v1.0 -m "Release 1.0"`	Create an annotated tag
`git tag v1.0-beta`	A lightweight tag
`git tag -a v0.9 a1b2c3d`	Tag an older commit
`git show v1.0`	The tag and the commit it names
`git push origin v1.0`	Push one tag
`git push --follow-tags`	Push commits and their annotated tags
`git tag -d v1.0`	Delete locally
`git describe --tags`	"v1.0-14-ga1b2c3d": the nearest tag, plus distance

Note 33.5.2. `git describe` is how build systems stamp a version into a binary without anyone maintaining a version number by hand. It reads as: fourteen commits after `v1.0`, at commit `a1b2c3d`.

§33.6. Submodules and Subtrees

Two ways to put one repository inside another.

`git submodule add <url> lib/foo`	Add a submodule
`git clone --recurse-submodules <url>`	Clone, fetching them too
`git submodule update --init --recursive`	Populate them after a plain clone
`git submodule update --remote`	Move them to their latest commits
`git subtree add --prefix=lib/foo <url> main --squash`	Add a subtree

> **Intuition .** A **submodule** records a *pointer*: the parent repository stores only the URL and one commit hash, and the files live in a separate repository. A **subtree** copies the other project's files into yours, so a plain clone gets everything.

Note 33.6.1. Submodules have a deserved reputation. A clone that omits `-recurse` leaves empty directories; a commit inside a submodule is invisible until the parent's pointer is also committed; and switching branches routinely leaves submodules on the wrong commit. They are right when the dependency is genuinely a separate project with its own history; for vendoring a library you do not develop, a subtree or a package manager causes less trouble.

Before reaching for either, ask whether your language's package manager solves it. Submodules are a version-control answer to a dependency-management question, and they are usually the wrong shape for it.

34. GitHub

Git is the version control system; GitHub is a website that hosts Git repositories and adds a collaboration layer on top — pull requests, issues, code review, continuous integration. The distinction matters: everything in this chapter is GitHub's invention, not Git's, and GitLab, Gitea and Bitbucket do the same things with different names.

§34.1. Repositories, Forks and Clones

Repository	A project on GitHub: the Git repository plus issues, pull requests and settings
Clone	A local copy on your machine. Pure Git
Fork	*Your* copy of someone else's repository, on GitHub. A GitHub concept

Intuition . Fork when you cannot push to the original — which is the normal situation for contributing to a project you do not maintain. You fork on GitHub, clone your fork, push branches to it, and open a pull request asking the original to take your changes. When you *can* push, skip the fork and branch directly.

```
$ git clone git@github.com:Konyi0613/LaTeX.git
$ cd LaTeX
$ git remote add upstream git@github.com:original/LaTeX.git   # for a fork
$ git remote -v
origin      git@github.com:Konyi0613/LaTeX.git (fetch)
upstream    git@github.com:original/LaTeX.git   (fetch)
```

§34.2. Authentication

GitHub stopped accepting passwords for Git operations in 2021. There are three replacements.

SSH key	A key pair; the public half goes to GitHub. No expiry, no typing
Personal access token	A password-shaped secret with scopes and an expiry date, used over HTTPS
`gh` CLI	Logs in once and acts as a credential helper for everything else

```
$ ssh-keygen -t ed25519 -C "you@example.com"    # see sec. 15.2
$ cat ~/.ssh/id_ed25519.pub                     # paste into GitHub settings
$ ssh -T git@github.com                         # check it worked
Hi Konyi0613! You've successfully authenticated.
```

Note 34.2.1 (This machine)**.** Authentication here is already handled by the third route: `gh` is registered as Git's credential helper for `github.com`, so HTTPS pushes find a token without prompting. `gh auth status` shows the account and scopes. An SSH key is still worth having — it works when `gh` is not installed, which is every server you will ever log into.

Never put a token in a URL, a script, or a commit. GitHub scans public pushes for its own token formats and revokes what it finds, which is a kindness rather than a solution — Section 36.8 covers what to do when a secret is already in the history.

§34.3. Pull Requests

Definition 34.3.1 (Pull request)**.** A request to merge one branch into another, with a description, a discussion, a running diff and whatever automated checks the project has configured. It is a GitHub object, not a Git one — underneath it is just two branches.

The full cycle:

```
$ git switch -c fix-config-crash        # 1. branch
$ # ... edit, commit ...
$ git push -u origin fix-config-crash   # 2. push the branch
$ gh pr create --fill                   # 3. open the PR
$ # ... review happens, you push more commits ...
$ gh pr merge --squash --delete-branch  # 4. merge and tidy up
```

Merge commit	Keeps every commit and records the branch. Full history
Squash merge	Combines the branch into one commit on `main`. Tidy history, loses the detail
Rebase merge	Replays each commit onto `main`. Linear history, no merge commit

Note 34.3.2. Squash merging pairs well with a habit of committing untidily while you work: the mess stays in the pull request, where it is reviewable, and `main` gets one clean commit per change. The cost is that `git log` on `main` can no longer show you how the work was built up. Most projects pick one of the three and apply it consistently; find out which before your first PR.

Note 34.3.3 (Draft pull requests)**.** Open a PR as a draft as soon as you push the branch, not when the work is finished. It gives the change a URL, runs CI on every push, and tells your colleagues what you are doing — with no implication that it is ready.

§34.4. Reviewing Code

Comment	Feedback without a verdict
Approve	Fine to merge
Request changes	Blocks merging until resolved
Suggestion	A concrete edit the author can apply with one click

`gh pr list`	Open pull requests
`gh pr view 42`	One, in the terminal
`gh pr view 42 --web`	. . . in the browser
`gh pr diff 42`	Its diff
`gh pr checkout 42`	Check the branch out locally — run it, don't just read it
`gh pr review 42 --approve`	Approve from the command line
`gh pr checks 42`	Whether CI passed

`gh pr checkout` is the command that makes reviews useful. Reading a diff catches style and obvious mistakes; checking the branch out, building it, and running it catches the things that matter. A review that never left the browser has only checked what the diff shows.

Note 34.4.1. Review the *change*, not the person, and say which comments are blocking and which are preferences. "Consider extracting this" and "this will deadlock" need to be distinguishable, or the author has to guess.

§34.5. Issues and Projects

`gh issue list`	Open issues
`gh issue create -t "..." -b "..."`	Create one
`gh issue view 17`	Read one
`gh issue close 17`	Close it
`gh issue list --label bug --assignee @me`	Filter

Note 34.5.1 (Closing keywords)**.** A commit message or pull request body containing `Closes #42`, `Fixes #42` or `Resolves #42` closes that issue automatically when the change lands on the default branch — and links the two permanently, so anyone reading the issue later finds the commit that fixed it. This is worth more than the automation: it is free documentation.

§34.6. The `gh` Command-Line Tool

`gh auth login`, `gh auth status`	Sign in; check who you are
`gh repo clone owner/name`	Clone without typing a URL
`gh repo create --public`	Create a repository from here
`gh repo view --web`	Open it in a browser
`gh pr create --fill`	Open a PR using the commits as the description
`gh pr status`	Your PRs, and those awaiting your review
`gh run list`, `gh run watch`	Actions runs; follow one live
`gh release create v1.0`	Cut a release
`gh browse`	Open the current file or line on GitHub

Note 34.6.1. `gh` 2.93 is installed here. `gh browse` deserves particular mention: run from a repository it opens the project page, and `gh browse Chapters/git_1_model.tex` opens that file — which is the fastest way to get a shareable link to a specific line of code.

§34.7. GitHub Actions

Definition 34.7.1 (Actions)**.** GitHub's continuous integration. A workflow is a YAML file, stored in the repository under `.github/workflows`, describing jobs to run on a hosted machine when something happens: a push, a pull request, a schedule, or a manual trigger.

```yaml
# .github/workflows/build.yml
name: Build the note

on:
  push:
    branches: [main]
  pull_request:

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Install TeX Live
        run: |
          sudo apt-get update
          sudo apt-get install -y texlive-xetex texlive-latex-extra latexmk

      - name: Compile
        run: latexmk -xelatex -interaction=nonstopmode -output-directory=build CSTools.tex

      - name: Upload the PDF
        uses: actions/upload-artifact@v4
        with:
          name: CSTools.pdf
          path: build/CSTools.pdf
```

`on`	What triggers the workflow
`jobs`	Independent units; they run in parallel by default
`runs-on`	Which hosted machine
`steps`	Run in order; each is a `run` command or a `uses` action
`uses`	A reusable action from the marketplace — pin the version

The first workflow worth writing for any project is the one that proves it still builds. For this note that is one `latexmk` invocation, and it means a broken LaTeX file is caught by the pull request rather than by you, three commits later.

§34.8. Pages, Releases and `README` Conventions

`README.md`	Shown on the repository's front page. What this is, how to build it, how to use it
`LICENSE`	Without one, nobody may legally reuse your work
`CONTRIBUTING.md`	How to submit a change
`.github/`	Workflows, issue templates, PR templates
Releases	Tagged versions with notes and attached binaries
Pages	Static hosting from a branch or an Actions job

Note 34.8.1. The `README` is the only documentation most people will ever read. Four things earn their space: what the project is in one sentence, how to install or build it, a minimal working example, and where to ask questions. Badges are decoration; the build instructions are

not.

35. Working on a Team

The commands do not change when a second person appears. What changes is that your history becomes something other people read, your mistakes become something other people inherit, and the conventions you follow stop being a matter of taste.

§35.1. Branching Models

Trunk-based	Everyone commits to `main`, behind feature flags, several times a day. Needs strong tests
GitHub Flow	One long-lived branch (`main`); short feature branches merged through pull requests. The common default
Git Flow	`main`, `develop`, plus `feature/`, `release/` and `hotfix/` branches. Designed for versioned, shipped software

GitHub Flow is the right default for almost every project: one protected `main` that always works, short-lived branches, and pull requests as the only way in. Git Flow was written in 2010 for software released in versions on a schedule; applied to a web service deployed twice a day it is pure overhead, and its own author has since said so.

Note 35.1.1. The variable that actually matters is branch *lifetime*. A branch alive for two days conflicts trivially; one alive for two months conflicts catastrophically, whatever the model is called. If you take one thing from this chapter, take that — merge early and often, in both directions.

§35.2. Commit Hygiene

Definition 35.2.1 (Atomic commit)**.** A commit containing exactly one logical change — complete, self-contained, and leaving the project in a working state.

One idea per commit	So that `revert` and `cherry-pick` work on it alone
It builds	So that `bisect` is not defeated by broken commits
Message says *why*	The diff already says what
No unrelated formatting	A whitespace change buried in a logic commit hides the logic
No generated files	Section 31.7

Note 35.2.2 (Conventional Commits)**.** Many projects use a structured subject line:

```
feat(parser): accept quoted values in config files
fix(io): check fopen's return value
docs: explain the three trees
refactor!: rename parse_line to parse_entry
```

The prefix is machine-readable, so tooling can generate changelogs and decide version numbers; `!` marks a breaking change. It is a convention, not a Git feature — adopt it if your project has, and do not impose it on a project that has not.

Reformatting and refactoring belong in their own commits, always. A reviewer faced with one commit that reindents four hundred lines and also changes a conditional cannot see the conditional — and neither will you, in a year, running `git log -S`.

§35.3. Protecting `main`

Require a pull request	No direct pushes to `main`
Require approvals	At least one reviewer who is not the author
Require status checks	CI must pass before merging
Require branches up to date	Rebase or merge `main` in before merging out
Forbid force pushes	History on `main` cannot be rewritten
Forbid deletion	The branch cannot be removed

Note 35.3.1. These are GitHub branch-protection rules, configured in the repository settings, not Git features — Git itself will happily let anyone force-push over anything. The rules are the mechanism that makes "never rewrite published history" (Section 32.4) enforceable rather than merely agreed.

Turn on "require status checks" before "require two approvals". A machine that reliably runs the tests catches more defects than a second human who is skimming, and it never gets tired on a Friday afternoon.

§35.4. Keeping a Fork in Sync

A fork does not update itself. The routine:

```
$ git remote add upstream git@github.com:original/project.git   # once
$ git fetch upstream
$ git switch main
$ git merge upstream/main          # or: git rebase upstream/main
$ git push origin main
```

`gh repo sync`	The same thing, in one command
`git fetch upstream`	Get their commits
`git rebase upstream/main`	Replay *your* branch on top of theirs

Never work on `main` in a fork. Keep it a clean mirror of upstream, and branch from it for every change. The moment `main` has your commits on it, syncing becomes a merge instead of a fast-forward, and stays awkward forever.

§35.5. Hooks

Definition 35.5.1 (Hook)**.** An executable script in `.git/hooks/` that Git runs at a defined moment. A non-zero exit status from a `pre-` hook cancels the operation.

`pre-commit`	Before a commit is made. Linting, formatting, quick tests
`commit-msg`	Check or rewrite the message
`pre-push`	Before pushing. The full test suite
`post-checkout`	After switching branches. Regenerate something
`pre-receive`	On the server, before accepting a push

```
#!/usr/bin/env bash
# .git/hooks/pre-commit -- refuse to commit a debug print
set -euo pipefail

if git diff --cached --name-only -z | xargs -0 grep -nE 'DEBUG_PRINT|TODO:REMOVE'
then
  echo "refusing: debug markers in staged changes" >&2
  exit 1
fi
```

Note 35.5.2. `.git/hooks/` is not part of the repository, so hooks are not shared and not versioned — a deliberate security decision, since cloning a repository must not execute its author's scripts. To share them, commit the scripts to a tracked directory and point Git at it: `git config core.hooksPath .githooks`. The `pre-commit` framework automates exactly this.

Keep hooks fast and keep them advisory. A `pre-commit` hook that takes thirty seconds trains everyone to use `git commit -no-verify`, at which point it is worse than nothing. Put the quick checks in a hook and the slow ones in CI, where waiting is free.

§35.6. Licences and Repository Hygiene

`LICENSE`	Without one, default copyright applies: nobody may legally reuse it
`README.md`	What it is, how to build, a minimal example
`CONTRIBUTING.md`	How to propose a change; what the review expects
`CODE_OF_CONDUCT.md`	Expected behaviour, and who to contact
`CHANGELOG.md`	What changed between versions, for humans
`.editorconfig`	Indentation and line endings, enforced by the editor

Note 35.6.1. "No licence" is not "public domain" — it is "all rights reserved". A public repository with no `LICENSE` cannot legally be used by anyone, which is almost never what the author intended. MIT and BSD-2 are permissive and short; the GPL family requires derived works to stay open; CC BY-SA suits prose and lecture notes better than any software licence does.

36. When Things Go Wrong

The most useful fact about Git is one nobody tells beginners: *almost nothing is ever really lost.* A commit that no branch points at is still in the object database, and Git keeps a log of every position every branch has held. This chapter is how to get things back, and how to find the commit that broke something.

§36.1. `reflog`: the Safety Net

Definition 36.1.1 (Reflog)**.** A local journal of every value `HEAD` and each branch has taken — every commit, checkout, merge, rebase and reset — with a timestamp. It is private to your clone and is never pushed.

```
$ git reflog
3f8a2b1 HEAD@{0}: reset: moving to HEAD~3
9c4e7d2 HEAD@{1}: commit: Add config validation
7b1e4a9 HEAD@{2}: commit: Extract parse_line
2d8c5f0 HEAD@{3}: checkout: moving from main to feature
bc91a17 HEAD@{4}: commit: First day content only
```

Every "I destroyed my work with `git reset --hard`" has the same answer: `git reflog`, find the hash from before the mistake, and `git reset --hard <that hash>`. The commits were never deleted — only the branch pointer moved, and the reflog remembers where it was.

`git reflog`	`HEAD`'s history
`git reflog show main`	One branch's history
`git reflog --date=iso`	With real timestamps
`git reset --hard HEAD@{2}`	Go back to where you were two moves ago
`git switch -c rescue HEAD@{1}`	Safer: give that position a new branch

Note 36.1.2. The one thing the reflog cannot recover is work that was never committed. `git reset --hard` with unstaged changes, or `git clean -fd`, destroys those permanently — which is why Section 31.8 singled them out. Reflog entries also expire, after 90 days by default.

§36.2. Recovering Lost Commits

`git reflog`	Almost always enough
`git fsck --lost-found`	Find unreachable objects the reflog missed
`git show <sha>`	Check you have the right one
`git switch -c rescue <sha>`	Give it a name and it is safe
`git cherry-pick <sha>`	Take just that commit onto your branch

Example 36.2.1 (A deleted branch)**.**

```
$ git branch -D feature
Deleted branch feature (was 9c4e7d2).
$ # ...five minutes later...
$ git switch -c feature 9c4e7d2      # the hash was in the message
$ git reflog | grep feature          # or find it here
```

Git printed the hash when it deleted the branch, precisely so this is possible.

§36.3. Fixing the Last Commit

`git commit --amend`	Redo the last commit with what is staged now
`git commit --amend -m "..."`	Change only the message
`git commit --amend --no-edit`	Add staged changes, keep the message
`git commit --amend --author="A <a@b.c>"`	Fix the author

Note 36.3.1. `-amend` does not edit the commit — it makes a new one with a new hash and moves the branch to it (Section 30.4). So it is subject to the golden rule: amending a pushed commit requires a force push, and anyone who already pulled has the old one.

§36.4. Splitting and Reordering

```
$ git rebase -i HEAD~4
```

Mark the commit to divide with `edit`; when the rebase stops there:

```
$ git reset HEAD~          # undo the commit, keep the changes unstaged
$ git add -p               # stage the first idea only
$ git commit -m "Extract parse_line"
$ git add -A               # stage the rest
$ git commit -m "Add config validation"
$ git rebase --continue
```

This is `git add -p` (Section 31.3) doing the same job after the fact. A commit you made in a hurry is not permanent — until you push it, the history is a draft.

§36.5. `bisect`

Definition 36.5.1 (Bisect)**.** A binary search over history. You tell Git one commit where the bug exists and one where it does not; Git checks out the midpoint, you test and report, and it halves the range each time. A thousand commits take about ten tests.

```
$ git bisect start
$ git bisect bad                  # the current commit is broken
$ git bisect good v1.0            # this tag was fine
Bisecting: 312 revisions left to test after this (roughly 8 steps)
[9c4e7d2...] Extract parse_line

$ make && ./run-tests.sh          # test this one
$ git bisect good                 # ...it passed
Bisecting: 156 revisions left to test after this (roughly 7 steps)

$ # ...repeat...
9c4e7d2f8a1b is the first bad commit
    Author: ...
    Date:   ...
    Extract parse_line

$ git bisect reset                # back to where you started
```

`git bisect start`	Begin
`git bisect bad` / `good`	Report the current commit
`git bisect skip`	Cannot test this one — it does not build
`git bisect run ./test.sh`	Automate it: exit 0 is good, non-zero is bad
`git bisect log`	What has been decided so far
`git bisect reset`	Stop and return

`git bisect run` is the whole thing in one command: write a script that exits 0 when the program behaves, and Git finds the guilty commit unattended. This is the payoff for the commit discipline of Section 35.2 — bisect is only as good as the guarantee that each commit builds.

§36.6. Archaeology

`git blame file.c`	Who last changed each line, and in which commit
`git blame -L 40,60 file.c`	Just those lines
`git blame -w -C file.c`	Ignore whitespace; follow moved code
`git log -S'fopen'`	Commits that added or removed that string
`git log -G'regex'`	... matching a regular expression
`git log -L 40,60:file.c`	The history of those lines specifically
`git log --follow file.c`	Across renames
`git show <sha>`	The commit blame pointed at

Note 36.6.1. `git blame` is badly named — its use is not finding someone to blame but finding the commit message that explains why a line is the way it is. The workflow is `blame` to get the hash, then `git show` to read the reasoning. This is the moment that repays every good commit message you ever wrote.

Note 36.6.2. `git log -S` (the "pickaxe") answers a question `blame` cannot: when did this string appear or disappear? If a call to `fopen` used to be checked and no longer is, `blame` shows only the current line, while `log -S'if (!f)'` finds the commit that removed the check.

§36.7. Rewriting History Safely

`git commit --amend`	The last commit
`git rebase -i <base>`	Any recent series
`git filter-repo`	Wholesale surgery across all history
`git push --force-with-lease`	Publish the rewrite, safely

[The golden rule, again] Rewriting is safe on commits nobody else has, and disruptive on commits they do. Before rewriting a shared branch: tell the team, do it, and have everyone run `git fetch && git reset --hard origin/main` — not `git pull`, which would merge the old history back in and duplicate every commit.

§36.8. Large Files and Secrets

Note 36.8.1 (Deleting is not enough)**.** Committing a password and then deleting it in the next commit removes it from the working tree only. The old commit still contains it, and `git log -p` still shows it. The same applies to a 500 MB file: every clone of the repository will download it forever.

`git filter-repo --path secrets.env --invert-paths`	Remove a file from all history
`git filter-repo --replace-text rules.txt`	Redact strings throughout
`git lfs track "*.psd"`	Large File Storage: keep a pointer, store the blob elsewhere
`git count-objects -vH`	How big is this repository, really

If a credential reached a remote, treat it as compromised and rotate it. Rewriting history is worth doing afterwards, but it does not un-distribute what other people already cloned, and it does not un-index what a scanner already found. Rotate first; clean second.

Note 36.8.2. `git filter-repo` replaced the old `filter-branch`, which was slow and notoriously easy to misuse. It is a separate install (`pip install git-filter-repo`). Whatever you use, do it on a fresh clone and keep the original until you have checked the result.

Try it

In a scratch repository, make three commits, then `git reset --hard HEAD 3` and confirm the work has apparently vanished. Now `git reflog`, find the hash, and `git reset --hard <it>`. Doing this once, deliberately, is what turns Git from something you are careful around into something you can experiment in.

Appendix

A. Vim Reference Card

§A.1. Modes

`<Esc>`	Return to Normal mode from anywhere
`i` `a` `I` `A` `o` `O`	Enter Insert mode
`v` `V` `<C-v>`	Charwise / linewise / blockwise Visual
`:`	Command-line mode
`R`	Replace mode
`<C-\>` `<C-n>`	Leave Terminal mode

§A.2. Motions

`h` `j` `k` `l`	Left, down, up, right
`w` `b` `e`	Word forward, back, end
`W` `B` `E`	Same, by whitespace-delimited WORD
`0` `^` `$`	Line start, first non-blank, line end
`f` `x` / `t` `x`	To / till the next `x`; `;` and `,` repeat
`g` `g` / `G`	Top / bottom of the file
`42` `G`	Line 42
`{` `}`	Previous / next paragraph
`%`	Matching bracket
`H` `M` `L`	Top / middle / bottom of the screen
`<C-d>` `<C-u>`	Half page down / up
`z` `z`	Centre the cursor line
`<C-o>` `<C-i>`	Back / forward in the jump list
`m` `a` then `‘` `a`	Set and return to mark `a`

§A.3. Operators and Text Objects

`d` `c` `y`	Delete, change, yank
`>` `<` `=`	Indent, dedent, re-indent
`g` `u` `g` `U` `g` `~`	Lower, upper, toggle case
`g` `q`	Reformat to `'textwidth'`
`d` `d` `y` `y` `c` `c`	Operate on the whole line
`D` `C` `Y`	Operate to the end of the line
`x` / `X`	Delete the character under / before the cursor
`r` `x`	Replace one character with `x`

`J`	Join with the next line
`i w` / `a w`	Inner / a word
`i p` / `a p`	Inner / a paragraph
`i " i ( i { i [ i t`	Inside quotes, parens, braces, brackets, tags
`a " a (` ...	The same, including the delimiters
`.`	Repeat the last change
`u` / `<C-r>`	Undo / redo

§A.4. Registers, Macros and Search

`" a y y`	Yank a line into register `a`
`" a p`	Paste register `a`
`" 0 p`	Paste the last *yank*, not the last delete
`" + y` / `" + p`	System clipboard
`" _ d`	Delete without touching any register
`:registers`	Show every register
`q a` ... `q`	Record a macro into register `a`
`@ a` / `@ @`	Play it / replay the last one
`/ pat <CR>`	Search forward; `n` and `N` repeat
`*` / `#`	Search for the word under the cursor
`:noh`	Clear search highlighting
`:%s/old/new/gc`	Substitute in the file, confirming each
`:g/pat/normal @q`	Run macro `q` on matching lines

§A.5. Files, Buffers and Windows

`:w :q :wq :q!`	Write, quit, both, force-quit
`:e file` / `:ls` / `:b N`	Open, list, switch buffer
`<C-^>`	Toggle with the alternate buffer
`<C-w> s` / `<C-w> v`	Split horizontally / vertically
`<C-w> h j k l`	Move between windows
`<C-w> c` / `<C-w> o`	Close this / all other windows
`:tabnew` / `g t`	New tab / next tab
`:copen` / `:cn` / `:cp`	Quickfix list: open, next, previous
`:%!cmd`	Filter the buffer through a shell command
`:help <topic>`	The manual

§A.6. Neovim

`:checkhealth`	Diagnose the whole installation
`:Lazy`	Plugin manager dashboard
`:Mason`	Install language servers and formatters
`:LspInfo`	Which servers serve this buffer
`:Inspect` / `:InspectTree`	Highlight groups / Treesitter tree
`:terminal`	Shell in a buffer
`:lua ...` / `:=expr`	Execute / print Lua
`:verbose set opt?`	Which file set this option
`:source %`	Re-run the Lua file being edited
`K` `g` `d` `g` `r`	LSP hover, definition, references
`]` `d` / `[` `d`	Next / previous diagnostic

B. Shell Reference Card

§B.1. Files and Navigation

`pwd`, `cd d`, `cd -`	Where am I; go there; go back
`ls -lh`, `ls -la`	Long listing; include hidden files
`tree -L 2`	Two levels of the tree
`mkdir -p a/b/c`	Make a whole chain of directories
`cp -r src dst`, `mv a b`	Copy a tree; move or rename
`rm -r d`, `rm -i f`	Delete a tree; ask first
`ln -s target name`	Symbolic link
`chmod +x f`, `chmod 644 f`	Make executable; set exact permissions
`du -sh *`, `df -h`	Size of each entry; free space
`stat f`, `file f`	Metadata; what it actually is
`find . -name '*.c' -type f`	Find by name — quote the pattern
`find . -mtime -7 -size +1M`	Recent and large
`find . -name '*.o' -delete`	Find and remove
`find . -name '*.c' -exec cmd {} +`	Run a command on the results

§B.2. Text Processing

`cat`, `head -n 3`, `tail -f`	Print all; first three; follow
`less f`	Page through it; Vim keys apply
`wc -l f`	Count lines
`grep -rn pat dir`	Search recursively, with line numbers
`grep -i`, `-v`, `-w`, `-c`	Ignore case; invert; whole word; count
`grep -E 'a|b'`	Extended regular expressions
`grep -q pat f`	Silent — for use as a test
`sed 's/old/new/g'`	Substitute everywhere on each line
`sed -n '10,20p'`	Print a range of lines
`sed -i.bak 's/a/b/g' f`	Edit in place, keeping a backup
`awk '{print $2}'`	The second field of every line
`awk -F: '$3 > 999 {print $1}'`	Field test with a custom separator

`sort -rn`, `sort -u`, `sort -h`	Numeric descending; unique; by size
`uniq -c`	Count adjacent duplicates
`cut -d:  -f1`, `tr -d '\r'`	Field one; strip carriage returns
`diff -u a b`	Unified diff — the format Git uses

§B.3. Redirection and Pipes

`> f`, `>> f`	Overwrite; append
`< f`	Read standard input from a file
`2> f`, `2>&1`, `&> f`	Errors to a file; errors to wherever output goes; both
`> /dev/null 2>&1`	Discard everything
`a | b`	Connect output to input
`cmd | tee log`	Save and pass through
`...  | xargs -0 rm`	Turn input into arguments, NUL-separated
`$(cmd)`	Command substitution — always quote it
`<(cmd)`	Process substitution: output as a filename
`<<'EOF' ...  EOF`	Here-document, nothing expanded
`a && b`, `a || b`	Run on success; run on failure
`$?`	Exit status of the last command

§B.4. Processes and Jobs

`ps aux`, `pgrep n`, `pstree`	All processes; by name; as a tree
`top`, `htop`	Live resource use
`lsof -i :8080`	What is holding this port
`cmd &`, `<C-z>`, `bg`, `fg`	Background; suspend; resume there; resume here
`jobs -l`	This shell's jobs, with PIDs
`kill PID`, `kill -9 PID`	Polite; last resort
`pkill -f pattern`	Kill by command line
`nohup cmd &`, `disown`	Survive the shell exiting
`export VAR=v`, `env`	Set for children; show the environment
`VAR=v cmd`	Set for one command only
`echo "$PATH"`, `which -a`	Search path; every match in order
`source f`	Run in the current shell, not a child
`type x`	Builtin, alias, function, or file?
`time cmd`	Wall, user and system time

§B.5. Scripting Idioms

`#!/usr/bin/env bash`	Portable shebang
`set -euo pipefail`	Fail on error, unset variable, or broken pipe
`trap 'rm -rf "$tmp"' EXIT`	Clean up however the script ends
`"$@"`	All arguments, each as its own word
`$#`, `shift`	How many; discard the first
`${v:-default}`	Fall back if unset or empty
`${f##*/}`, `${f%/*}`	Basename; dirname
`${f%.tex}.pdf`	Swap an extension
`[[ -f f ]]`, `[[ -z "$v" ]]`	File exists; string is empty
`[[ $a == $b ]]`, `[[ $n -lt 5 ]]`	String and numeric comparison
`for f in *.tex; do ...; done`	Loop over files — never over `ls`
`while IFS= read -r line`	Read lines exactly as written
`local v=$1`	Keep a function's variables to itself
`mktemp -d`	A temporary directory, safely named
`bash -x script.sh`	Trace every command after expansion
`shellcheck script.sh`	Static analysis

C. Make Reference Card

§C.1. Rule Syntax

`target: prereqs`	A rule; the recipe follows, tab-indented
`→recipe`	Recipe lines begin with a literal TAB
`target: prereqs | order-only`	After `|`: built first, timestamp ignored
`%.o: %.c`	Pattern rule; `%` is the stem
`$(OBJ): %.o: %.c`	Static pattern rule, restricted to a list
`.PHONY: all clean`	Targets that are not files
`.DEFAULT_GOAL := app`	What a bare `make` builds
`vpath %.c src`	Where to look for matching prerequisites
`-include $(DEP)`	Include a file, silently if it is missing
`@cmd`	Do not echo this recipe line
`-cmd`	Ignore a non-zero exit status
`cmd1 && cmd2`	One shell: each recipe line gets its own
`SHELL := /bin/bash`	Use Bash instead of `/bin/sh`

§C.2. Variables

`VAR = value`	Recursive: re-expanded at every use
`VAR := value`	Simple: expanded once, here. Prefer this
`VAR ?= value`	Only if not already set — lets callers override
`VAR += value`	Append
`VAR != cmd`	Assign a shell command's output
`target: VAR += x`	Target-specific value, inherited by prerequisites
`override VAR += x`	Wins even over the command line
`$$`	A literal `$`, passed on to the shell
`CC CFLAGS CPPFLAGS`	Compiler, its flags, preprocessor flags
`LDFLAGS LDLIBS`	Where to look; what to link. `-lm` goes in `LDLIBS`
`PREFIX DESTDIR`	Install location; packager's staging root
`$(MAKE)`	Recursive invocation — never write `make`

§C.3. Automatic Variables

`$@`	The target
`$<`	The first prerequisite — use for `-c`

`$^`	All prerequisites, deduplicated — use for linking
`$+`	All prerequisites, duplicates kept
`$?`	Only those newer than the target
`$*`	The stem matched by `%`
`$(@D)`, `$(@F)`	The target's directory and filename

§C.4. Built-in Functions

`$(wildcard *.c)`	Expand a glob now
`$(patsubst %.c,%.o,$(SRC))`	Pattern substitution
`$(SRC:.c=.o)`	The same, in shorthand
`$(subst a,b,$(T))`	Plain text substitution
`$(filter %.c,$(F))`	Keep matching words
`$(filter-out main.o,$(O))`	Drop matching words
`$(sort $(L))`	Sort and deduplicate
`$(dir $(F))`, `$(notdir $(F))`	Directory, filename
`$(basename $(F))`, `$(suffix $(F))`	Without, only the extension
`$(addprefix build/,$(O))`	Prepend to every word
`$(shell date +%F)`	Run a shell command
`$(info ...)`	Print while reading the file
`$(error ...)`	Print and stop

§C.5. Command-Line Options

`make -n`	Dry run: print the recipes, run none
`make -j$(nproc)`	One job per core
`make -B`	Rebuild everything regardless of timestamps
`make -k`	Carry on after errors
`make -C dir`	Change directory first
`make -f other.mk`	Use a different file
`make -r`	Drop the built-in rule database
`make -s`	Do not echo recipes
`make -debug=b`	Say why each target was or was not rebuilt
`make -trace`	Annotate each recipe with its Makefile line
`make -p`	Dump every rule and variable
`make CC=clang`	Override a variable for this build

§C.6. Diagnosing a Makefile

`missing separator`	Spaces where a tab belongs. `cat -A Makefile`
`No rule to make target`	Misspelt filename, or a missing generator rule
`'x' is up to date`	`x` should have been in `.PHONY`
`Recursive variable ...`	`=` where `:=` was meant
Fails only under `-j`	A missing edge in the graph, not a make bug
Rebuilds every time	A directory used as a normal prerequisite; make it order-only
Does not rebuild on a header edit	Add `-MMD -MP` and `-include $(DEP)`
`print-%: @echo '$* = [$($*)]'`	A rule for printing any variable

D. GDB Reference Card

§D.1. Starting and Stopping

`gcc -g -O0 -o app *.c`	Compile so the binary can be debugged at all
`gdb ./app`	Load it; nothing runs yet
`gdb --args ./app -v f`	... with arguments
`gdb ./app core`	Post-mortem from a core dump
`gdb -p 1234`	Attach to a running process
`gdb -q -batch -ex run -ex bt --args ./app`	One-line crash triage
`start`	Run, stopping at the first line of `main`
`run` / `r`	Run from the beginning
`continue` / `c`	Resume
`kill`, `quit`	Stop the program; leave GDB
`set args ...`	Arguments for the next `run`
`<CR>`	Repeat the previous command
`<C-c>`	Interrupt the running program
`help break`, `apropos watch`	Help on one command; search all help

§D.2. Breakpoints and Watchpoints

`break main`	On a function
`break parser.c:42`	On a line
`break parse if i == 500`	Conditional — the one that saves the day
`tbreak main`	Temporary: deleted after it fires
`ignore 2 499`	Skip the next 499 hits. Faster than a condition
`condition 2 n > 100`	Add or change a condition
`info breakpoints`	List them, with hit counts
`disable 2`, `enable 2`, `delete 2`	Manage one
`watch total`	Stop when a value changes
`watch -l *ptr`	Watch a location, not a name
`rwatch`, `awatch`	Stop on read; on read or write
`catch throw`, `catch syscall write`	Exceptions; system calls
`commands ... end`	Script to run at a breakpoint
`silent` / `printf` / `continue`	The usual trio inside one

§D.3. Stepping and Frames

`next` / `n`	Next source line, stepping *over* calls
`step` / `s`	Next source line, stepping *into* calls
`finish`	Run until this function returns
`until`	Like `next`, but run a loop to completion
`until 60`, `advance f`	Run to a line, or to a function
`stepi`, `nexti`	One machine instruction
`return 0`	Abandon the frame, forcing a return value
`jump 42`	Continue from another line in this function
`backtrace` / `bt`	The call stack. The first command after a crash
`bt full`	… with every frame's locals
`frame 2`, `up`, `down`	Select a frame
`info locals`, `info args`	The selected frame's variables
`list`, `list parse`	Show source
`disassemble /s`	Instructions, interleaved with source

§D.4. Examining Data

`print n` / `p n`	Evaluate an expression in the current frame
`print/x`, `/t`, `/c`, `/s`	Hex, binary, character, string
`print *p@n`	`n` elements from a pointer. Essential in C
`print *cfg`	Dereference a struct pointer
`print func(3)`	Call a function in the live process
`x/16xb buf`	16 bytes of memory, in hex
`x/s s`, `x/5i $pc`	As a string; as instructions
`ptype cfg`, `whatis cfg`	The type, expanded or named
`ptype/o struct node`	… with offsets and padding
`display i`	Print it automatically at every stop
`set var n = 42`	Assign — note `var`, not bare `set`
`set $p = head`	A convenience variable
`$1`, `$`	Earlier results; the last one
`info registers`	Every register

§D.5. Threads, Signals and Comfort

`info threads`	List them
`thread apply all bt`	Every thread's stack. The deadlock command
`thread 3`	Switch
`set scheduler-locking on`	Step one thread, freeze the others

`handle SIGPIPE nostop noprint pass`	Stop GDB interrupting on a routine signal
`info signals`	The whole signal table
`<C-x>` `a`	Toggle the TUI
`layout src`, `focus cmd`	Choose a layout; give the arrows back
`set print pretty on`	Readable structs
`set pagination off`	No “—Type <return> to continue—”
`define ... end`	A user-defined command
`source debug.gdb`	Load a file of commands
`:packadd termdebug` then `:Termdebug ./app`	GDB inside Neovim

§D.6. Choosing the Right Tool

Crash, sensible backtrace	GDB: `bt`, `info args`, `up`
Crash, backtrace is `??`	The stack is corrupt — a sanitiser first
Use-after-free, buffer overrun	`gcc -fsanitize=address`
Signed overflow, bad shift	`gcc -fsanitize=undefined`
Data race	`gcc -fsanitize=thread`
Hang or deadlock	`gdb -p` then `thread apply all bt`
Wrong answer, no crash	A breakpoint and `display`
Wrong value, unknown writer	`watch`
Happens on iteration 500	`break f if i == 500`
Only fails at `-O2`	Suspect undefined behaviour, not the compiler
Crashed once, on a server	`ulimit -c unlimited`, then `gdb ./app core`

E. Git Reference Card

§E.1. The Everyday Loop

`git init`, `git clone <url>`	Start a repository; copy one
`git status -sb`	What changed, in two columns
`git add file`, `git add -A`	Stage one file; stage everything
`git add -p`	Stage parts of files, interactively
`git rm --cached file`	Stop tracking, keep on disk
`git commit -v`	Commit, with the diff in the editor
`git commit -m "..."`	Commit with a one-line message
`git commit --amend`	Redo the last commit
`git diff`	Working tree vs index (unstaged)
`git diff --staged`	Index vs `HEAD` (staged)
`git diff HEAD`	Everything not committed
`git diff -w`, `--word-diff`	Ignore whitespace; diff by word

§E.2. Branching and Merging

`git switch -c feature`	Create a branch and move there
`git switch main`, `git switch -`	Move; move back
`git branch -v`, `git branch -d f`	List with commits; delete
`git merge feature`	Merge into the current branch
`git merge --no-ff feature`	Always record a merge commit
`git merge --abort`	Undo an in-progress merge
`git rebase main`	Replay this branch on top of `main`
`git rebase -i main`	… editing the series first
`git rebase --continue` / `--abort`	After a conflict; give up
`pick` / `reword` / `squash` / `fixup` / `drop`	The interactive-rebase verbs
`git commit --fixup=<sha>`	Mark a fix, for `--autosquash`
`git cherry-pick <sha>`	Apply one commit here
`git stash`, `git stash pop`	Set work aside; bring it back
`git worktree add ../hotfix main`	A second directory, another branch

§E.3. Remotes

`git remote -v`	Which remotes, and their URLs
`git remote add upstream <url>`	Add one — the fork convention
`git fetch`, `git fetch --prune`	Download; forget dead branches
`git log HEAD..origin/main`	What they did that you do not have
`git pull --rebase`	Fetch, then replay your commits on top
`git push -u origin feature`	Push and set the upstream
`git push --force-with-lease`	Overwrite, but only if nobody else pushed
`git push origin --delete f`	Delete a remote branch
`git tag -a v1.0 -m "..."`	An annotated tag
`git push --follow-tags`	Push commits with their tags
`git describe --tags`	Nearest tag, plus distance
`gh pr create --fill`	Open a pull request
`gh pr checkout 42`	Check out someone's PR to actually run it
`gh pr checks 42`, `gh run watch`	Did CI pass; follow a run live

§E.4. Inspecting History

`git log --oneline --graph --all --decorate`	The one to alias
`git log -p -- file.tex`	Full diffs, one file
`git log -S'fopen'`	Commits that added or removed a string
`git log -L 40,60:file.c`	The history of specific lines
`git log main..feature`	On one branch but not the other
`git show <sha>`	One commit in full
`git blame -w -C file.c`	Who last touched each line
`git bisect start / bad / good`	Binary-search for the breaking commit
`git bisect run ./test.sh`	... unattended
`git cat-file -p HEAD`	The raw commit object
`git count-objects -vH`	How large the repository really is

§E.5. Undo and Recovery

`git restore file`	Discard unstaged edits to a file
`git restore --staged file`	Unstage, keeping the edits
`git restore --source=HEAD~2 f`	Take a file from an old commit
`git reset --soft HEAD~`	Undo the commit, keep it staged
`git reset HEAD~`	Undo the commit and unstage

`git reset --hard HEAD~`	**Destroys** the changes
`git revert <sha>`	A new commit undoing an old one
`git clean -n`, `git clean -fd`	List, then delete untracked files
`git reflog`	Every position `HEAD` has held
`git reset --hard HEAD@{2}`	Go back to before the mistake
`git switch -c rescue <sha>`	Rescue a commit no branch points at
`git fsck --lost-found`	Unreachable objects the reflog missed
`git filter-repo --path f --invert-paths`	Remove a file from all history

§E.6. Diagnosing Git

"Detached `HEAD`"	Harmless. `git switch -c name` to keep the work
"Your branch is ahead by 2"	You have unpushed commits
"no upstream branch"	`git push -u origin <branch>`
Push rejected, non-fast-forward	Fetch and rebase; then `--force-with-lease` if you rewrote
Conflict markers in a file	Edit, `git add`, `git commit`; or `--abort`
"I reset and lost everything"	`git reflog`
Uncommitted work lost to `--hard`	Not recoverable. The one real hazard
`.gitignore` ignored	The file is already tracked: `git rm --cached`
Every commit duplicated after a pull	Someone rewrote a shared branch (Section 36.7)
A secret was pushed	Rotate it first; rewrite history second

F. Configuration Files

§F.1. A Minimal `init.lua`

A self-contained starting point, for when a distribution is more than you want. Put it in `~/.config/nvim/init.lua`; it bootstraps its own plugin manager on first launch.

```lua
-- Leaders must be set before any plugin loads
vim.g.mapleader = " "
vim.g.maplocalleader = "\\"

-- Options
local opt = vim.opt
opt.number = true
opt.relativenumber = true
opt.expandtab = true
opt.shiftwidth = 2
opt.tabstop = 2
opt.smartindent = true
opt.ignorecase = true
opt.smartcase = true
opt.undofile = true          -- persistent undo tree
opt.scrolloff = 8
opt.signcolumn = "yes"       -- stop the text jumping when a sign appears
opt.splitright = true
opt.splitbelow = true
opt.termguicolors = true
```

Mappings and autocommands next. `desc` is what `which-key` and `:map` will show you later.

```lua
-- Mappings
local map = vim.keymap.set
map("n", "<leader>w", "<cmd>write<cr>", { desc = "Save" })
map("n", "<leader>h", "<cmd>nohlsearch<cr>", { desc = "Clear highlight" })
map("n", "<C-h>", "<C-w>h", { desc = "Left window" })
map("n", "<C-j>", "<C-w>j", { desc = "Lower window" })
map("n", "<C-k>", "<C-w>k", { desc = "Upper window" })
map("n", "<C-l>", "<C-w>l", { desc = "Right window" })
map("v", "<", "<gv", { desc = "Dedent, keep selection" })
map("v", ">", ">gv", { desc = "Indent, keep selection" })
map("t", "<Esc><Esc>", "<C-\\><C-n>", { desc = "Leave terminal mode" })

-- Autocommands
vim.api.nvim_create_autocmd("TextYankPost", {
  group = vim.api.nvim_create_augroup("yank_highlight", { clear = true }),
  callback = function() vim.highlight.on_yank({ timeout = 200 }) end,
})
```

Finally the plugin manager, which installs itself the first time Neovim starts.

```
-- Plugins (lazy.nvim, bootstrapped on first run)
local lazypath = vim.fn.stdpath("data") .. "/lazy/lazy.nvim"
if not (vim.uv or vim.loop).fs_stat(lazypath) then
  vim.fn.system({ "git", "clone", "--filter=blob:none", "--branch=stable",
    "https://github.com/folke/lazy.nvim.git", lazypath })
end
vim.opt.rtp:prepend(lazypath)

require("lazy").setup({
  { "folke/tokyonight.nvim", priority = 1000,
    config = function() vim.cmd.colorscheme("tokyonight") end },
  { "nvim-treesitter/nvim-treesitter", build = ":TSUpdate",
    opts = { highlight = { enable = true }, indent = { enable = true } },
    config = function(_, opts)
      require("nvim-treesitter.configs").setup(opts)
    end },
  { "neovim/nvim-lspconfig" },
  { "lervag/vimtex", ft = "tex", init = function()
      vim.g.vimtex_view_method = "zathura"
    end },
})
```

§F.2. A Reusable Makefile Skeleton

§F.3. A `.gdbinit` Worth Having

```
# ~/.gdbinit
set confirm off                 # stop asking "are you sure"
set pagination off              # no "---Type <return> to continue---"
set history save on             # remember commands between sessions
set history size 10000
set print pretty on             # structs one member per line
set print array on
set print elements 0            # never truncate an array or string
set print null-stop on          # stop a char array at the first NUL
set backtrace limit 100         # tame runaway recursion
set breakpoint pending on       # do not ask about unloaded libraries
set disassembly-flavor intel

# Walk a linked list: plist head
define plist
  set $n = $arg0
  while $n != 0
    printf "%s\n", $n->name
    set $n = $n->next
  end
end
document plist
Walk a linked list from the given head, printing each name.
end

# Trace every call to a function without recompiling: trace parse_line
define trace
  break $arg0
  commands
    silent
    bt 1
    continue
  end
end
document trace
Set a breakpoint that prints one stack frame and carries on.
end
```

§F.4. A `.gitconfig` Worth Having

```
# ~/.gitconfig
[user]
    name  = Your Name
    email = you@example.com

[init]
    defaultBranch = main

[core]
    editor = nvim
    autocrlf = input          # never store CRLF in the repository

[pull]
    rebase = true             # no "Merge branch 'main' of ..." commits
[rebase]
    autosquash = true         # arrange --fixup commits automatically
    autostash  = true         # stash and restore around a rebase
[push]
    autoSetupRemote = true    # plain `git push' works on a new branch
    followTags = true         # push annotated tags with their commits

[commit]
    verbose = true            # show the diff while writing the message

[diff]
    algorithm = histogram     # better diffs on reordered code
    colorMoved = zebra        # distinguish moved lines from changed ones
[merge]
    conflictStyle = zdiff3    # show the common ancestor in conflict markers

[alias]
    s    = status -sb
    l    = log --oneline --graph --all --decorate
    ll   = log --stat
    d    = diff
    ds   = diff --staged
    co   = switch
    cob  = switch -c
    amend = commit --amend --no-edit
    unstage = restore --staged
    last = log -1 HEAD --stat

# use a different identity for work repositories
[includeIf "gitdir:~/work/"]
    path = ~/.gitconfig-work
```

Note F.4.1. `merge.conflictStyle = zdiff3` is the least known and most useful of these: conflict markers then include the *common ancestor* as well as both sides, so you can see what each side actually changed rather than guessing from two final states.

G. This Machine

Every command in this note was run on one ordinary Linux desktop. Versions matter more in some places than others — `grep` has behaved the same way for thirty years, while Neovim's LSP API moved twice in two releases — so they are recorded here rather than repeated in the text.

§G.1. Versions

Operating system	Linux Mint 22.1, kernel 6.8.0
`nvim`	0.9.5 — note Section 7.7: 0.11 changed the LSP configuration API
`zsh`	5.9 (the login shell)
`bash`	5.2.21
GNU coreutils	9.4
`sed`	GNU sed 4.9 — `-i` without an argument is GNU-only
`awk`	GNU Awk 5.2.1
`make`	GNU Make 4.3
`gcc`	13.3.0
`gdb`	15.0.50
`git`	2.43.0 — `git switch` and `git restore` need 2.23
`gh`	2.93.0
XeTEX	3.141592653, TeX Live 2023

§G.2. Installed and Not Installed

Several sections recommend a tool that is not present here. They are named so that you know what is a recommendation and what was actually exercised.

Present	`rg`, `jq`, `tree`, `htop`, `tmux`, `curl`, `wget`, `rsync`, `xclip`, `zathura`, `nvr`, `latexmk`
Present	GCC's sanitisers — `-fsanitize=address,undefined,thread`
Absent	`fd` (Section 10.7), `shellcheck` (Section 14.8), `tldr`
Absent	`valgrind` and `rr` (Section 28.6)
Absent	`nvim-dap` (Section 29.4); `termdebug` ships with Neovim and is used instead
Note	`grep` here is `ugrep` in GNU-compatible mode, not GNU grep
Note	Core dumps are disabled by default — `ulimit -c` reports 0 (Section 23.4)

§G.3. The Editor Configuration

Chapter 8 describes this in full. In summary: LazyVim on `lazy.nvim`, with `vimtex` (Zathura for viewing, `nvr` for inverse search, `latexmk` with `-xelatex` into a `build/` directory) and `LuaSnip` loading snippets from `~/.config/nvim/lua/snippets`. The leader is `<Space>` and the local leader is `\`, so VimTeX's prefix is `\ l`.

Note G.3.1. This note is itself built by the workflow it describes: XeTEX because the template loads `fontspec`, driven by `latexmk` from inside Neovim, tracked in Git, and typeset from a Makefile of the shape given in Section 22.2.

Bibliography

[CS14] Scott Chacon and Ben Straub. *Pro Git*. 2nd ed. Apress, 2014. ISBN: 9781484200773. URL: https://git-scm.com/book.

[fol] folke. *LazyVim: a Neovim config for the lazy*. https://www.lazyvim.org/. Accessed 2026-09-14.

[L3M] L3MON4D3. *LuaSnip: snippet engine for Neovim written in Lua*. https://github.com/L3MON4D3/LuaSnip. Accessed 2026-09-14.

[Ler] Karl Yngve Lervåg. *VimTeX: a modern Vim and Neovim filetype plugin for LaTeX files*. https://github.com/lervag/vimtex. Accessed 2026-09-14.

[Mil98] Peter Miller. "Recursive Make Considered Harmful". In: *AUUGN Journal of AUUG Inc.* 19.1 (1998), pp. 14–25. URL: https://accu.org/journals/overload/14/71/miller_2004/.

[Nei15] Drew Neil. *Practical Vim: Edit Text at the Speed of Thought*. 2nd ed. Pragmatic Bookshelf, 2015. ISBN: 9781680501278.

[Neo] Neovim Project. *Neovim documentation*. https://neovim.io/doc/user/. Accessed 2026-09-14.

[SMS] Richard M. Stallman, Roland McGrath, and Paul D. Smith. *GNU Make Manual*. Accessed 2026-09-14. Free Software Foundation. URL: https://www.gnu.org/software/make/manual/.

[SPS] Richard M. Stallman, Roland Pesch, and Stan Shebs. *Debugging with GDB: The GNU Source-Level Debugger*. Accessed 2026-09-14. Free Software Foundation. URL: https://sourceware.org/gdb/current/onlinedocs/gdb/.

[Vim] Vim Project. *Vim documentation*. https://vimhelp.org/. Accessed 2026-09-14.